Snowflake Interview Master Study Material

Purpose: A single, ordered Snowflake interview-preparation guide built from the four uploaded documents. It keeps the original theory, examples, SQL/code, screenshots/diagrams, troubleshooting notes, and interview questions. The complete converted source material is preserved in the appendices so source content is not intentionally omitted.

How to use this guide

Study in the sequence below. For each topic, prepare four things: **definition → how it works → SQL/example → interview scenario**.

Recommended Preparation Order

1. Snowflake fundamentals and cloud concepts
2. Snowflake architecture and query processing
3. Virtual warehouses, scaling, concurrency, and cost
4. Databases, schemas, table types, views, and constraints
5. Micro-partitions, pruning, clustering, and clustering depth
6. Snowflake caching and query lifecycle
7. Stages and file formats
8. Bulk loading with COPY INTO
9. Loading from Azure Blob / ADLS Gen2 using storage integration
10. Snowpipe and continuous ingestion
11. Streams, CDC, tasks, task graphs, and pipelines
12. Time Travel, Fail-safe, and Zero-Copy Cloning
13. SQL interview preparation: joins, CTE, window functions, QUALIFY, MERGE
14. Semi-structured data: VARIANT, JSON, FLATTEN, STRIP_OUTER_ARRAY
15. Stored procedures, UDFs, SnowSQL, Python connector
16. Security: roles, grants, masking, row access policies, tags
17. Monitoring, load history, query history, task history, troubleshooting
18. Cost controls: resource monitors, budgets, warehouse strategy
19. External tables
20. Apache Iceberg tables
21. Hybrid tables
22. Query Acceleration Service
23. Search Optimization Service
24. Dynamic Tables
25. Snowpark
26. Replication / database copy / DR
27. Data unloading

28. Data warehouse fundamentals
29. Dimensional modeling: facts, dimensions, grain, surrogate keys
30. Star / Snowflake / Galaxy schemas
31. SCD and CDC concepts
32. Migration questions: Oracle / Teradata / Hive / Athena comparisons
33. Production scenarios and troubleshooting
34. Rapid-fire Snowflake interview questions
35. Lead/architect scenario questions

Part I — Core Snowflake Interview Notes

1. Snowflake Fundamentals

What is Snowflake?

Snowflake is a cloud-native data platform/data warehouse delivered as SaaS. A central interview point in the uploaded material is that **compute and storage are decoupled**, allowing them to scale independently.

Key characteristics

- SaaS: infrastructure is managed by Snowflake.
- Runs on major cloud infrastructure.
- Separates storage from compute.
- Uses virtual warehouses for compute.
- Supports structured and semi-structured data.
- Provides built-in capabilities such as Time Travel, cloning, data sharing, streams/tasks, governance, and serverless services.

Interview question

Q: What makes Snowflake different from a traditional on-premises warehouse?
A: Snowflake separates compute, storage, and cloud-service responsibilities. Compute can be resized or multiplied without copying the underlying data, while Snowflake manages storage format, metadata, optimization, and infrastructure as a service.

---

2. Snowflake Architecture

Remember the three layers:

Cloud Services Layer
↓
Virtual Warehouse / Compute Layer

↓
Centralized Storage Layer

Storage layer

- Data is stored in Snowflake-managed cloud storage.
- Snowflake reorganizes loaded data into optimized compressed columnar storage.
- Data is divided automatically into micro-partitions.
- Storage is independent of warehouses.

Compute layer

- Query execution is performed by virtual warehouses.
- A warehouse is an MPP compute cluster.
- Multiple warehouses can access the same underlying data.

Cloud Services layer

Responsible for functions such as: - Authentication and access control - Parsing and optimization - Metadata management - Transaction management - Infrastructure coordination - Query history and related services

Interview sequence for query processing

User submits SQL
↓
Cloud Services parses/resolves/checks access/optimizes
↓
Execution plan sent to virtual warehouse
↓
Worker nodes scan required micro-partitions/columns
↓
Results returned and may be persisted in result cache

---

3. Virtual Warehouses, Scaling, and Concurrency

Scale up

Increase warehouse size, for example:

MEDIUM → LARGE → X-LARGE

Use scale-up for compute-heavy or long-running queries, large scans, and spill-heavy workloads.

Scale out

Add clusters to a multi-cluster warehouse. Use this primarily for **concurrency/queueing**, not to make one individual query inherently use multiple independent warehouses.

Scaling policies

The uploaded material calls out: - Standard - Economy

Interview question

Q: Is storage added when a warehouse scales out?
A: No. Storage is decoupled from compute. Additional clusters add compute resources, not separate copies of Snowflake table storage.

---

4. Table Types and Views

Prepare: - Permanent tables - Transient tables - Temporary tables - Standard views - Secure views - Materialized views - External tables - Dynamic tables - Iceberg tables - Hybrid tables

Know how retention/recovery behavior differs among table types and why transient/temporary objects can reduce data-protection storage overhead.

---

5. Micro-Partitions, Pruning, and Clustering

Snowflake automatically stores table data in micro-partitions. The uploaded notes emphasize approximately **50 MB–500 MB uncompressed** per micro-partition.

Metadata can include: - min/max range information - number of distinct values - other optimization statistics

This metadata helps Snowflake perform **micro-partition pruning**.

Clustering key

Use explicit clustering only when there is a demonstrated benefit, particularly for very large tables and common selective filter patterns.

```
ALTER TABLE sales CLUSTER BY (sale_date, region_id);
```

Check clustering information:

```
SELECT SYSTEM$CLUSTERING_INFORMATION(
  'SALES',
  '(SALE_DATE, REGION_ID)'
);
```

Drop a clustering key:

```
ALTER TABLE sales DROP CLUSTERING KEY;
```

Interview point

A high-cardinality/unique column is not automatically a good clustering key. Clustering can add maintenance cost, so justify it using pruning/query patterns and clustering information.

---

6. Caching

Prepare these cache concepts from the material:

1. **Persisted query/result cache** — Cloud Services layer, commonly retained for up to 24 hours and reusable when eligibility conditions are met.
2. **Virtual warehouse local disk cache** — compute-layer cache associated with warehouse nodes.
3. **Remote storage** — underlying cloud storage when data is not available locally.

Disable persisted result reuse for testing:

```
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
```

Retrieve a previous result explicitly:

```
SELECT *
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

Conditions to know

Result reuse depends on factors such as matching query semantics/text, unchanged relevant data, valid persisted results, and compatible session/configuration conditions.

---

7. Stages and File Formats

Internal stages

- User stage
- Table stage
- Named internal stage

External stages

Point to external cloud storage such as Azure Blob/ADLS, AWS S3, or GCS.

File format example

```
CREATE OR REPLACE FILE FORMAT csv_ff
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
```

```
  FIELD_OPTIONALLY_ENCLOSED_BY = ""
  NULL_IF = ('NULL', 'null', '');
```

If a CSV field itself contains commas, FIELD_OPTIONALLY_ENCLOSED_BY is a key option to prepare.

---

8. Azure Blob / ADLS Gen2 → Snowflake Loading Sequence

Memorize this sequence:

1. Target table
2. Storage integration
3. Azure consent / RBAC permission
4. File format
5. External stage
6. LIST / validate files
7. COPY INTO
8. Verify COPY_HISTORY / target data

Step 1 — target table

```
CREATE OR REPLACE TABLE customer (
  customer_id NUMBER,
  customer_name VARCHAR,
  city VARCHAR,
  state VARCHAR
);
```

Step 2 — storage integration

```
CREATE OR REPLACE STORAGE INTEGRATION azure_int
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = AZURE
  ENABLED = TRUE
  AZURE_TENANT_ID = '<tenant-id>'
  STORAGE_ALLOWED_LOCATIONS = (
    'azure://<account>.blob.core.windows.net/<container>/'
  );
```

Inspect the integration:

```
DESC STORAGE INTEGRATION azure_int;
```

Step 3 — Azure permission

Use the consent/application information returned by Snowflake to grant the required Azure storage access.

Step 4 — file format

```
CREATE OR REPLACE FILE FORMAT customer_csv_ff
  TYPE = CSV
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"';
```

Step 5 — external stage

```
CREATE OR REPLACE STAGE customer_azure_stage
  URL = 'azure://<account>.blob.core.windows.net/<container>/customer/'
  STORAGE_INTEGRATION = azure_int
  FILE_FORMAT = customer_csv_ff;
```

Step 6 — validate connectivity

```
LIST @customer_azure_stage;
```

Query staged data:

```
SELECT $1, $2, $3, $4
FROM @customer_azure_stage;
```

Step 7 — load

```
COPY INTO customer
FROM @customer_azure_stage;
```

Or explicit mapping:

```
COPY INTO customer(customer_id, customer_name, city, state)
FROM (
  SELECT $1, $2, $3, $4
  FROM @customer_azure_stage
);
```

Useful COPY INTO options

```
COPY INTO customer
FROM @customer_azure_stage
ON_ERROR = 'CONTINUE';
```

```
COPY INTO customer
FROM @customer_azure_stage
FORCE = TRUE;
```

```
COPY INTO customer
FROM @customer_azure_stage
PURGE = TRUE;
```

Validate errors:

```
COPY INTO customer
FROM @customer_azure_stage
VALIDATION_MODE = 'RETURN_ERRORS';
```

Capture file metadata during load:

```
SELECT
  METADATA$FILENAME,
  METADATA$FILE_ROW_NUMBER,
  $1, $2
FROM @customer_azure_stage;
```

---

9. Snowpipe

Use Snowpipe for continuous/serverless file ingestion. Prepare: - Pipe object - Auto-ingest/event notification flow - COPY INTO statement inside a pipe - Load history and error troubleshooting - Difference between bulk COPY INTO and continuous Snowpipe

Conceptual flow:

File lands in cloud storage
↓
Cloud event notification
↓
Snowpipe
↓
COPY logic
↓
Target table

---

10. Streams, Tasks, and Continuous Pipelines

Stream

A stream records CDC information between transactional points so downstream logic can consume row-level changes.

Task

A task executes SQL or calls a stored procedure on a schedule or as part of a task graph.

Typical pattern:

Source files → Snowpipe → Staging table
↓

Stream
↓
Task
↓
Target table

Check whether a stream has data:

```
SELECT SYSTEM$STREAM_HAS_DATA('MY_STREAM');
```

Prepare stream types, stale streams, task history, DAG/task graphs, and error-notification concepts from the source appendices.

---

11. Time Travel, Fail-safe, and Zero-Copy Cloning

Time Travel

Used to query or restore historical versions within the configured retention period.

Typical syntax patterns:

```
SELECT *
FROM orders AT (TIMESTAMP => '<timestamp>');
```

```
UNDROP TABLE orders;
```

Zero-Copy Clone

```
CREATE TABLE orders_test CLONE orders;
```

```
CREATE DATABASE dev_db CLONE prod_db;
```

A clone initially shares existing underlying storage references; additional storage is driven by changes over time rather than an immediate full physical copy.

---

12. SQL Topics to Prepare

Prepare hands-on examples for: - INNER / LEFT / RIGHT / FULL joins - CTE / WITH - LEAD / LAG - ROW_NUMBER, RANK, DENSE_RANK - Running totals - QUALIFY - MERGE - deduplication - conditional multi-table insert - date/timestamp conversion - error-handling conversion functions such as TRY_CAST

Example deduplication:

```
SELECT *
FROM employees
QUALIFY ROW_NUMBER() OVER (
```

```
  PARTITION BY employee_id
  ORDER BY updated_at DESC
) = 1;
```

Example MERGE:

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN
  UPDATE SET t.name = s.name
WHEN NOT MATCHED THEN
  INSERT (id, name) VALUES (s.id, s.name);
```

---

13. Semi-Structured Data

Prepare: - VARIANT - OBJECT - ARRAY - JSON path notation - FLATTEN - STRIP_OUTER_ARRAY - JSON loading using COPY INTO

Example:

```
SELECT
  src:id::NUMBER AS id,
  src:name::VARCHAR AS name
FROM json_table;
```

---

14. Security and Governance

Prepare: - Role hierarchy and RBAC - USAGE, SELECT, ownership, future grants - Dynamic data masking - Row access policy - Tags - Secure views - Network/authentication concepts from the source notes

Example grants:

```
GRANT USAGE ON DATABASE sales_db TO ROLE analyst;
GRANT USAGE ON SCHEMA sales_db.reporting TO ROLE analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA sales_db.reporting TO ROLE analyst;
```

Row access policies limit which rows are returned based on policy logic, often using role/user context and a mapping table.

---

15. Monitoring, Troubleshooting, and Cost

Prepare: - Query Profile - QUERY_HISTORY - COPY_HISTORY - TASK_HISTORY - load validation - resource monitors - budgets - warehouse auto-suspend/auto-resume - query timeouts and queueing - clustering/search optimization costs

Example load history:

```
SELECT *
FROM TABLE(
  INFORMATION_SCHEMA.COPY_HISTORY(
    TABLE_NAME => 'CUSTOMER',
    START_TIME => DATEADD('HOUR', -24, CURRENT_TIMESTAMP())
  )
);
```

Part II — Advanced Snowflake Topics

16. External Tables

An external table stores metadata about external files while the actual data remains in external cloud storage. External tables are primarily read-only and can be queried/joined without loading the file data into native Snowflake tables.

Creation sequence highlighted in the uploaded material:

Storage integration → External stage → External table → Metadata refresh

Example:

```
CREATE OR REPLACE EXTERNAL TABLE ext_table
WITH LOCATION = @my_s3_stage
FILE_FORMAT = (TYPE = CSV);
```

Prepare automatic vs manual metadata refresh and user-specified partitions.

---

17. Apache Iceberg Tables

Prepare: - What Apache Iceberg is - Table metadata / manifests / snapshots conceptually - Snowflake-managed catalog vs external catalog - External volume - Read/write behavior by table/catalog type - Copy-on-write concept in the uploaded material - Comparison: native Snowflake vs external table vs Iceberg table

---

18. Hybrid Tables

Prepare why hybrid tables exist, how they differ from standard analytic Snowflake tables, and the kinds of transactional/low-latency use cases they target. The uploaded advanced material notes this topic separately and should be reviewed from the preserved section below.

---

19. Query Acceleration Service

Prepare: - Why QAS exists - Which queries can benefit - How extra serverless compute can accelerate eligible portions of a query - Monitoring and cost considerations

---

20. Search Optimization Service

Prepare when search optimization is preferable for highly selective lookup/search patterns and how it differs from clustering. Also prepare cost-estimation examples in the source material.

---

21. Dynamic Tables

A dynamic table materializes the result of a query and automatically refreshes it based on the configured target lag/refresh behavior.

Conceptual pattern:

Base tables
↓
Transformation query
↓
Dynamic table
↓
Automatically maintained materialized result

The uploaded notes compare dynamic tables with materialized views and discuss restrictions and supported source/query patterns.

---

22. Snowpark

Prepare: - Snowpark DataFrame concepts - Creating DataFrames - Transformations - Writes - Pandas DataFrame interaction - Differences/limitations versus Spark APIs noted in the source document

---

Part III — Data Warehouse & Dimensional Modeling

23. OLTP vs OLAP

Typical conversion emphasized in the uploaded material:

OLTP master tables → Dimension tables
OLTP transaction tables → Fact tables

Prepare Inmon/top-down and Kimball/bottom-up concepts, and understand why dimensional models are optimized for analytical reporting.

---

24. Facts and Dimensions

Fact table

Contains measurements and foreign keys to dimensions at a defined grain.

Dimension table

Contains descriptive business attributes used to filter, group, and label facts.

Grain

The grain defines exactly what one fact row represents. Define it before deciding fact measures and dimensions.

Example:

One row per Product × Store × Day

---

25. Types of Facts / Measures

Prepare: - Fully additive - Semi-additive - Non-additive - Derived facts - Factless fact tables - Textual measures

A classic semi-additive example is **account balance**, which can be aggregated across some dimensions but generally not summed across time snapshots.

---

26. Dimension Types

Prepare: - Conformed dimension - Role-playing dimension - Junk dimension - Degenerate dimension - Slowly changing dimension - Rapidly changing dimension

A conformed dimension is reused consistently across fact tables/subject areas, such as a common Date dimension.

---

27. Surrogate Keys

A surrogate key is a generated warehouse key, typically a compact integer used as a dimension primary key rather than depending directly on the natural/business key.

Why prepare it: - Integrates multiple source systems - Supports SCD history - Stable warehouse joins - Separates source business identifiers from warehouse row identity

---

28. Star vs Snowflake Schema

Star schema

Fact table directly connects to denormalized dimensions. Usually simpler and often faster/easier for BI queries.

Snowflake schema

Dimensions are further normalized into related tables. Can reduce redundancy but introduces more joins.

Galaxy / fact constellation

Multiple fact tables share conformed dimensions.

---

29. Slowly Changing Dimensions

Prepare at minimum: - SCD Type 1 — overwrite current value - SCD Type 2 — create new history row, usually with effective dates/current flag and a new surrogate key - SCD Type 3 — preserve limited prior value in additional columns

Also understand the difference between **SCD** (dimension-history modeling technique) and **CDC** (capturing source data changes).

Part IV — Interview Practice Strategy

30. Rapid-fire questions you should be able to answer

1. Explain Snowflake's three-layer architecture.
2. Why are storage and compute decoupled?
3. What happens when a user submits a query?
4. What is a virtual warehouse?
5. Scale up vs scale out?

6. Standard vs Economy scaling policy?
7. Permanent vs transient vs temporary tables?
8. What is a micro-partition?
9. What is partition pruning?
10. When should you use a clustering key?
11. What is clustering depth?
12. Explain Snowflake caches.
13. Can result cache work when a warehouse is suspended?
14. Internal vs external stage?
15. What is a storage integration?
16. Explain Azure Blob → Snowflake loading sequence.
17. What does COPY INTO do?
18. FORCE=TRUE vs PURGE=TRUE?
19. How do you troubleshoot rejected rows?
20. How do you store source filename/row number while loading?
21. Bulk COPY vs Snowpipe?
22. What is a stream?
23. What is stream staleness?
24. What is a task?
25. How do tasks and streams implement CDC?
26. Time Travel vs Fail-safe?
27. What is zero-copy cloning?
28. Explain QUALIFY.
29. Explain MERGE.
30. How do you remove duplicates?
31. How do you handle JSON?
32. What is dynamic data masking?
33. What is a row access policy?
34. What is an external table?
35. Why must external-table metadata be refreshed?
36. External table vs native table vs Iceberg table?
37. What is a dynamic table?
38. Dynamic table vs materialized view?
39. What is Search Optimization Service?
40. What is Query Acceleration Service?
41. What is Snowpark?
42. Explain database replication.
43. How do you control Snowflake costs?
44. What is a resource monitor?
45. What is a budget?
46. How do you diagnose a slow query?

47. What causes spilling?
48. What causes query queueing?
49. What is a surrogate key?
50. Star vs Snowflake schema?
51. Explain fact table grain.
52. Additive vs semi-additive vs non-additive facts?
53. What is a factless fact table?
54. What is a conformed dimension?
55. Explain SCD Type 1/2/3.
56. SCD vs CDC?
57. Explain a migration challenge from Oracle/Teradata/Hive/Athena to Snowflake.
58. How would you design an incremental pipeline?
59. How would you recover from a failed load?
60. How would you reduce Snowflake cost without hurting SLA?

Part V — Complete Source-Preservation Appendices

The sections below preserve the full Pandoc conversion of all four uploaded documents, including extracted screenshots/diagrams as relative media references. This is intentionally included so detailed notes, code, troubleshooting examples, and interview questions from the source documents remain available even when they are not repeated in the simplified study sections above.

Appendix A — Snowflake Core Material

Source: snowflake.md converted from the corresponding uploaded Word document.

Contents

Snowflake with azure blob and datalake

snowflake to azure datalake Gen2

https://www.youtube.com/watch?v=IEzsWsbDDcA

https://www.youtube.com/watch?v=jTIStJfCbdY

datalake zen2 creation refer below link:

https://www.youtube.com/watch?v=2uSkjBEwwq0&t=976s

how to connect ADL GEN2 from snowflake in lasalle

in lasalle private link is created from snowflake to azure datalake gen2

azure dev ops team will provide private tenant id and azure path

https://docs.snowflake.com/en/sql-reference/sql/create-storage-integration.html

below script is used to create integration object it is similar to blob storage

ex:

create storage integration azure_int

type = external_stage

storage_provider = azure

enabled = true

azure_tenant_id = ‘a123b4c5-1234-123a-a12b-1a23b45678c9’

storage_allowed_locations = (‘*’)

storage_blocked_locations = (‘azure://myaccount.blob.core.windows.net/mycontainer/path3/’, ‘azure://myaccount.blob.core.windows.net/mycontainer/path4/’);

create storage integration s3_int

type = external_stage

storage_provider = s3

storage_aws_role_arn = ‘arn:aws:iam::001234567890:role/myrole’

enabled = true

storage_allowed_locations = (‘s3://mybucket1/path1/’, ‘s3://mybucket2/path2/’);

External Stages using storage integration

Amazon S3

Create an external stage named my_ext_stage using a private/protected S3 bucket named load with a folder path named files. Secure access to the S3 bucket is provided via the myint storage integration:

create stage my_ext_stage

url=‘s3://load/files/’

storage_integration = s3_int;

Microsoft Azure

Create an external stage named my_ext_stage using a private/protected Azure container named load with a folder path named files. Secure access to the container is provided via the myint storage integration:

create stage my_ext_stage

url=‘azure://myaccount.blob.core.windows.net/load/files/’

storage_integration = azure_int;

What is snowflake:

https://www.stitchdata.com/resources/snowflake/#:~:text=The%20Snowflake%20architecture%20allows%20storage,secure%20data%20in%20real%20time.

What is a Snowflake data warehouse?

Snowflake is a **data warehouse** built on top of the Amazon Web Services or Microsoft Azure cloud infrastructure. There's no hardware or software to select, install, configure, or manage, so it's ideal for organizations that don't want to dedicate resources for setup, maintenance, and support of in-house servers.

But what sets Snowflake apart is its architecture and data sharing capabilities. The Snowflake architecture allows storage and compute to scale independently, so customers can use and pay for storage and computation separately. And the sharing functionality makes it easy for organizations to quickly share governed and secure data in real time.

Pay per use

In snowflake compute and storage are decoupled.

Types of staging in SnowFlake

In snow flake if staging is with in snowflake then it called as internal staging.

If staging is outside snowflake means staging in aws/azure then it called as external staging.

Cloud computing :

Why we need could computing;

Reduced IT costs

Moving to cloud computing may reduce the cost of managing and maintaining your IT systems. Rather than purchasing expensive systems and equipment for your business, you can reduce your costs by using the resources of your cloud computing service provider. You may be able to reduce your operating costs because:

- the cost of system upgrades, new hardware and software may be included in your contract
- you no longer need to pay wages for expert staff

- your energy consumption costs may be reduced
- there are fewer time delays.

Scalability

Your business can scale up or scale down your operation and storage needs quickly to suit your situation, allowing flexibility as your needs change. Rather than purchasing and installing expensive upgrades yourself, your cloud computer service provider can handle this for you. Using the cloud frees up your time so you can get on with running your business.

Business continuity

Protecting your data and systems is an important part of business continuity planning. Whether you experience a natural disaster, power failure or other crisis, having your data stored in the cloud ensures it is backed up and protected in a secure and safe location. Being able to access your data again quickly allows you to conduct business as usual, minimising any downtime and loss of productivity.

Collaboration efficiency

Collaboration in a cloud environment gives your business the ability to communicate and share more easily outside of the traditional methods. If you are working on a project across different locations, you could use cloud computing to give employees, contractors and third parties access to the same files. You could also choose a cloud computing model that makes it easy for you to share your records with your advisers (e.g. a quick and secure way to share accounting records with your accountant or financial adviser).

Flexibility of work practices

Cloud computing allows employees to be more flexible in their work practices. For example, you have the ability to access data from home, on holiday, or via the commute to and from work (providing you have an internet connection). If you need access to your data while you are off-site, you can connect to your virtual office, quickly and easily.

Access to automatic updates

Access to automatic updates for your IT requirements may be included in your service fee. Depending on your cloud computing service provider, your system will regularly be updated with the latest technology. This could include up-to-date versions of software, as well as upgrades to servers and computer processing power.

The below image refers normal structure without cloud

Below image with cloud

SnowFlake is Software as Service

Snow flake architecture:

Snowflake architecture from learning journal:

Snowflake is cloud datawarehouse which is built on azure/aws cloud. It is SAAS (SOFTWARE AS SERVICE).

FROM SNOW FLAKE DOCUMENTATION:

Snowflake Architecture

Snowflake's architecture is a hybrid of traditional shared-disk database architectures and shared-nothing database architectures. Similar to shared-disk architectures, Snowflake uses a central data repository for persisted data that is accessible from all compute nodes in the data warehouse. But similar to shared-nothing architectures, Snowflake processes queries using MPP (massively parallel processing) compute clusters where each node in the cluster stores a portion of the entire data set locally. This approach offers the data management simplicity of a shared-disk architecture, but with the performance and scale-out benefits of a shared-nothing architecture.

Snowflake's unique architecture consists of three key layers:

- Database Storage (STORAGE LAYER)
- Query Processing(COMPUTE LAYER)
- Cloud Services(CLOUD LAYER)

Database Storage

When data is loaded into Snowflake, Snowflake reorganizes that data into its internal optimized, compressed, columnar format. Snowflake stores this optimized data in cloud storage.

Snowflake manages all aspects of how this data is stored — the organization, file size, structure, compression, metadata, statistics, and other aspects of data storage are handled by Snowflake. The data objects stored by Snowflake are not directly visible nor accessible by customers; they are only accessible through SQL query operations run using Snowflake.

Query Processing

Query execution is performed in the processing layer. Snowflake processes queries using "virtual warehouses". Each virtual warehouse is an MPP compute cluster composed of multiple compute nodes allocated by Snowflake from a cloud provider.

Each virtual warehouse is an independent compute cluster that does not share compute resources with other virtual warehouses. As a result, each virtual warehouse has no impact on the performance of other virtual warehouses.

For more information, see Virtual Warehouses.

The query processing layer takes advantage of the shared nothing architecture. Query execution takes place inside this layer. Queries are processed using virtual warehouses. Each virtual warehouse is an MPP (Massive Parallel Processing)compute cluster composed of many compute nodes allocated by Snowflake from a cloud provider. These compute clusters are basically AWS EC2 instances if the cloud provider is AWS.

Each virtual warehouse is an independent compute cluster and does not share compute resources with other clusters or virtual warehouses. That is why performance of one virtual warehouse does not impact the performance of other virtual warehouse.

Cloud Services

The cloud services layer is a collection of services that coordinate activities across Snowflake. These services tie together all of the different components of Snowflake in order to process user requests, from login to query dispatch. The cloud services layer also runs on compute instances provisioned by Snowflake from the cloud provider.

Among the services in this layer:

- Authentication
- Infrastructure management
- Metadata management
- Query parsing and optimization
- Access control

Connecting to Snowflake

Snowflake supports multiple ways of connecting to the service:

- A web-based user interface from which all aspects of managing and using Snowflake can be accessed.
- Command line clients (e.g. SnowSQL) which can also access all aspects of managing and using Snowflake.
- ODBC and JDBC drivers that can be used by other applications (e.g. Tableau) to connect to Snowflake.
- Native connectors (e.g. Python) that can be used to develop applications for connecting to Snowflake.
- Third-party connectors that can be used to connect applications such as ETL tools (e.g. Informatica) and BI tools to Snowflake.

Virtual Warehouse & Scalability

SNOW FLAKE HAS shared storage

There is concept called multi clustered ware houses this is available for enterprise customers.

Below is from snowflake documentation

https://docs.snowflake.com/en/user-guide/warehouses-multicluster.html

what is a Multi-cluster Warehouse?

By default, a virtual warehouse consist of a single cluster of servers that determines the total resources available to the warehouse for executing queries. As queries are submitted to a warehouse, the warehouse allocates resources to each query and begins executing the queries. If sufficient resources are not available to execute all the queries submitted to the warehouse, Snowflake queues the additional queries until the necessary resources become available.

With multi-cluster warehouses, Snowflake supports allocating, either statically or dynamically, a larger pool of resources to each warehouse. A multi-cluster warehouse is defined by specifying the following properties:

- Maximum number of server clusters, greater than 1 (up to 10).
- Minimum number of server clusters, equal to or less than the maximum (up to 10).

Additionally, multi-cluster warehouses support all the same properties and actions as single-cluster warehouses, including:

- Specifying a warehouse size.
- Resizing a warehouse at any time.
- Auto-suspending a running warehouse due to inactivity; note that this does not apply to individual clusters, but rather the entire warehouse.
- Auto-resuming a suspended warehouse when new queries are submitted.

Maximized vs. Auto-scale

You can choose to run a multi-cluster warehouse in either of the following modes:

Maximized

This mode is enabled by specifying the ***same*** value for both maximum and minimum clusters (note that the specified value must be larger than 1). In this mode, when the warehouse is started, Snowflake starts all the clusters so that maximum resources are available while the warehouse is running.

This mode is effective for statically controlling the available resources (i.e. servers), particularly if you have large numbers of concurrent user sessions and/or queries and the numbers do not fluctuate significantly.

Auto-scale

This mode is enabled by specifying ***different*** values for maximum and minimum clusters. In this mode, Snowflake starts and stops clusters as needed to dynamically manage the load on the warehouse:

- As the number of concurrent user sessions and/or queries for the warehouse increases, and queries start to queue due to insufficient resources, Snowflake automatically starts additional clusters, up to the maximum number defined for the warehouse.

- Similarly, as the load on the warehouse decreases, Snowflake automatically shuts down clusters to reduce the number of running servers and, correspondingly, the number of credits used by the warehouse.

To help control the usage of credits in Auto-scale mode, Snowflake provides a property, SCALING_POLICY, that determines the scaling policy to use when automatically starting or shutting down additional clusters. For more information, see Setting the Scaling Policy for a Multi-cluster Warehouse (in this topic).

Snowflake pricing

Section 4: Getting data into Snowflake

Ingestion / Loading Methods

DATA LOADING options:

https://docs.snowflake.com/en/user-guide/data-load-overview.html

https://docs.snowflake.com/en/user-guide/data-load-tutorials.html —- imp

BULK LOAD

CONTINOUS LOAD

Bulk vs Continuous Loading

Snowflake provides the following main solutions for data loading. The best solution may depend upon the volume of data to load and the frequency of loading.

Bulk Loading Using the COPY Command

This option enables loading batches of data from files already available in cloud storage, or copying (i.e. *staging*) data files from a local machine to an internal (i.e. Snowflake) cloud storage location before loading the data into tables using the COPY command.

Compute Resources

Bulk loading relies on user-provided virtual warehouses, which are specified in the COPY statement. Users are required to size the warehouse appropriately to accommodate expected loads.

Simple Transformations During a Load

Snowflake supports transforming data while loading it into a table using the COPY command. Options include:

- Column reordering
- Column omission
- Casts
- Truncating text strings that exceed the target column length

There is no requirement for your data files to have the same number and ordering of columns as your target table.

Continuous Loading Using Snowpipe

This option is designed to load small volumes of data (i.e. micro-batches) and incrementally make them available for analysis. Snowpipe loads data within minutes after files are added to a stage and submitted for ingestion. This ensures users have the latest results, as soon as the raw data is available.

Compute Resources

Snowpipe uses compute resources provided by Snowflake (i.e. a serverless compute model). These Snowflake-provided resources are automatically resized and scaled up or down as required, and are charged and itemized using per-second billing. Data ingestion is charged based upon the actual workloads.

Simple Transformations During a Load

The COPY statement in a pipe definition supports the same COPY transformation options as when bulk loading data.

In addition, data pipelines can leverage Snowpipe to continously load micro-batches of data into staging tables for transformation and optimization using automated tasks and the change data capture (CDC) information in streams.

Data Pipelines for Complex Transformations

A data pipeline enables applying complex transformations to loaded data. This workflow generally leverages Snowpipe to load "raw" data into a staging table and then uses a series of table streams and tasks to transform and optimize the new data for analysis.

Loading from Data Files Staged on Other Cloud Platforms

Snowflake supports loading data from files staged in any of the following locations, regardless of the cloud platform for your Snowflake account:

- Internal (i.e. Snowflake) stages
- Amazon S3
- Google Cloud Storage
- Microsoft Azure Blob storage

Alternatives to Loading Data

It is not always necessary to load data into Snowflake before executing queries.

External Tables (Data Lake)

External tables enable querying existing data stored in external cloud storage for analysis without first loading it into Snowflake. The source of truth for the data remains in the external cloud storage. Data sets materialized in Snowflake via materialized views are read-only.

This solution is especially beneficial to accounts that have a large amount of data stored in external cloud storage and only want to query a portion of the data; for example, the most recent data. Users can create materialized views on subsets of this data for improved query performance.

1. **Amazon S3**
2. create or replace external table ext_twitter_feed
3. with location = @mystage/daily/
4. auto_refresh = true
5. file_format = (type = parquet)
6. pattern='.*sales.*[.]parquet';

Google Cloud Storage

create or replace external table ext_twitter_feed

with location = @mystage/daily/

file_format = (type = parquet)

pattern='.*sales.*[.]parquet';

Microsoft Azure

create or replace external table ext_twitter_feed

integration = 'MY_AZURE_INT'

with location = @mystage/daily/

auto_refresh = true

file_format = (type = parquet)

pattern='.*sales.*[.]parquet';

7. Refresh the external table metadata:

alter external table ext_twitter_feed refresh;

https://docs.snowflake.com/en/sql-reference/sql/create-external-table.html

Loading BULK Data from cloud & local storage

Refer :

https://docs.snowflake.com/en/user-guide/data-load-internal-tutorial.html

4 STEPS are there for loading BULK data :

Prepare your files

Stage the data

Execute copy command

Managing regular loads

Prepare your files:

STAGE THE DATA:

In this we will upload our files into amazon S3 bucket

Amazon path for s3 bucket: by udemy

LOADING FROM STAGE TO SNOWFLAKE:

COPY INTO command is the common mechanism for loading data into snowflake in bath mode

If we try to load same file again 2nd time into table then data wont be loaded into table bcz snowflake is maintain metadata. This meta data will be maintained upto 90 days. After 90 days you can load same file again.

To check error while copying data example:

To use validate function to check the records failed during loading we have to use on_erros=skipfile other wise we can't see errors using validate function

select * from customer;

copy into customer

from @bulk_copy_example_stage

pattern='.*.csv'

file_format = (type = csv field_delimiter = ',' skip_header = 1)

on_error=skip_file ;

to see the errors use below query:

select * from table(validate(customer, job_id=>'01955c5c-0059-a582-0000-0000246d54b1'));

Validate Errors

The following process returns errors by query ID and saves the results to a table for future reference.

You can view the query ID for the COPY job on the **History** page of the web interface:

1. Log into the Snowflake web interface.
2. Change to the role you have been using to run the tutorial SQL statements.
3. Click **History** .
4. Click the **Query ID** column link for the COPY INTO command. The **Details** panel opens.
5. Copy the **Query ID** value.

6. In the command line interface (e.g., SnowSQL), execute the following command. Replace *query_id* with the **Query ID** value.
7. create or replace table save_copy_errors as select * from table(validate(mycsvtable, job_id=>'<query_id>'));
8. Query the results table:

select * from save_copy_errors;

Loading JSON Data

To handle json data we have to use VARIANT data type in snowflake.

Sample json file :

```
– create a table in which we will load the raw JSON data
CREATE TABLE organisations_json_raw (
json_data_raw VARIANT
);
– copy the example_json_file.json into the raw table
COPY INTO organisations_json_raw
FROM @json_example_stage/example_json_file.json
file_format = (type = json);
```

please refer full script: in this example json file is present in AWS s3 bucket.


load_json_data.sql

Snowpipe

Snowpipe enables loading data from files as soon as they're available in a stage. This means you can load data from files in micro-batches, making it available to users within minutes, rather than manually executing COPY statements on a schedule to load larger batches.

Snowflake SnowPipe Cheat Sheet

Snowflake SnowPipe Overview

1. SnowPipe enables loading data from files as soon as they're available in a external stage
2. A pipe is a named, first-class Snowflake object that contains a COPY statement used by Snowpipe
3. Pipe wraps copy commands, so all data type is supported (json,avro etc)
4. File arrival detecting mechanism
   - Using cloud notification
   - Calling REST API endpoint
5. SnowPipe copies the files into a queue, from which they are loaded into the target table in a continuous, serverless fashion based on parameters defined in a specified pipe object.
6. File Size Recommendation
   - The number of load operations that run in parallel cannot exceed the number of data files to be loaded
   - data files roughly 10 MB to 100 MB in size compressed
   - Split large files into a greater number of smaller files to distribute the load among the servers
7. Semi-Structured File size
   - Variant data type imposes 16Mb compressed for individual rows.
   - we recommend enabling the STRIP_OUTER_ARRAY file format option for the COPY INTO <table> command to remove the outer array structure and load the records into separate table rows
8. Other Important Note
   - Snowpipe charges 0.06 credits per 1000 files queued
   - There is no guarantee that files are loaded in the same order they are staged
   - SnowPipe latency is hard to estimate as it depends on the file size, file format and complexity of copy statement.

SnowPipe/Continouse Loading VS Copy Command/Bulk Loading

- Authentication
  - Bulk Loading - user session
  - Continouse Loading - JSON Web Token using public/private key

- Load History
  - Bulk Loading - 64 days of metadata history
  - Continouse Loading - 14 days of metadata history
- Transaction
  - Bulk Loading - always single transaction
  - Continouse Loading - combine or split based on number of files
- Compute Resource
  - Bulk Loading - WH is required
  - Continouse Loading - uses SF supplied resources
- Cost
  - Bulk Loading - priced everytime WH is active
  - Continouse Loading - bill as per SF supplied resource is used.

How Is Snowpipe Different from Bulk Data Loading?

Refer: https://docs.snowflake.com/en/user-guide/data-load-snowpipe-intro.html

This section briefly describes the primary differences between Snowpipe and a bulk data load workflow using the COPY command. Additional details are provided throughout the Snowpipe documentation.

Authentication

Bulk data load

Relies on the security options supported by the client for authenticating and initiating a user session.

Snowpipe

When calling the REST endpoints: Requires key pair authentication with JSON Web Token (JWT). JWTs are signed using a public/private key pair with RSA encryption.

Load History

Bulk data load

Stored in the metadata of the target table for 64 days. Available upon completion of the COPY statement as the statement output.

Snowpipe

Stored in the metadata of the pipe for 14 days. Must be requested from Snowflake via a REST endpoint, SQL table function, or ACCOUNT_USAGE view.

Important

To avoid reloading files (and duplicating data), we recommend loading data from a specific set of files using ***either*** bulk data loading or Snowpipe but not both.

Transactions

Bulk data load

Loads are always performed in a single transaction. Data is inserted into table alongside any other SQL statements submitted manually by users.

Snowpipe

Loads are combined or split into a single or multiple transactions based on the number and size of the rows in each data file. Rows of partially loaded files (based on the ON_ERROR copy option setting) can also be combined or split into one or more transactions.

Compute Resources

Bulk data load

Requires a user-specified warehouse to execute COPY statements.

Snowpipe

Uses Snowflake-supplied compute resources.

Cost

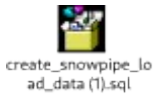
Bulk data load

Billed for the amount of time each virtual warehouse is active.

Snowpipe

Billed according to the compute resources used in the Snowpipe warehouse while loading the files.

LOADING DATA USING SNOW PIPE:



In live while creating in stage table we have to provide aws s3 bucket credentials as shown below image

Snowpipe good example with steps : new

https://calogica.com/sql/snowflake/2019/04/04/snowpipes.html#10-monitor-data-loads

Monitor data Snowpipe Data loads:

We can use the built-in pipe_status command to check on our pipe's status and how many files are current in the queue.

select system$pipe_status('my_pipe');

If files are no longer queued, check to make sure your table has the expected number of records.

select count(*) from src.my_source_table;

Snowflake provides a couple of ways to check on load success and/or errors.

The COPY_HISTORY function provides useful information of load status by file.

select *

from table(information_schema.copy_history(table_name=>'MY_SOURCE_TABLE',

start_time=> dateadd(hours, -24, current_timestamp())));

In my experience this approach often doesn't yield any results. So, if you have ACCOUNTADMIN access, you can also query the equivalent view in the ACCOUNTUSAGE schema directly, and also aggregate it to provide a status overview as shown below.

use role accountadmin;

use snowflake;

select

convert_timezone('America/Los_Angeles', h.last_load_time)::timestamp_ntz::date as load_date,

max(convert_timezone('America/Los_Angeles', h.last_load_time)::timestamp_ntz) as max_load_time,

sum(h.row_count) as rows_loaded,

sum(h.error_count) as errors

from account_usage.copy_history h

where table_name = 'MY_SOURCE_TABLE'

group by 1

order by 1;

This Account Usage view can be used to query Snowflake data loading history for the last 365 days (1 year).

To create AWS notification :

Go to properties→ under advance settings →select events→click on add notifications→

Enter details as shown below

Here main noting point is we have to select SQS queue

In sqs ARN value we have to copy key from snow flake show pipe notification key value.

In show pipes notification_channel column

Sqs queue ARN value has to be taken from snow flake show pipes notification_channel column value

Snowpipe using Azure:

azure

for snowpipe we need to have storage account, container, queue

storage acount has to be created

while creating storage account , account kind should be storage V2 or higher

Fo find erros in snowpipe while loading data use copy_history.

https://docs.snowflake.com/en/sql-reference/functions/copy_history.html

http://cloudsqale.com/2019/05/14/snowflake-monitoring-data-ingestion-using-query_history-and-copy_history-single-large-file-vs-multiple-small-files/

https://docs.snowflake.com/en/sql-reference/functions/validate_pipe_load.html

ex:

select *

from table(information_schema.copy_history(table_name=>'MYTABLE', start_time=> dateadd(hours, -1, current_timestamp())));

select * from table(validate_pipe_load(

pipe_name=>'MY_DB.PUBLIC.MYPIPE',

start_time=>dateadd(hour, -1, current_timestamp())));

This function returns pipe activity within the last 14 days.

MY EXAMPLE:

select * from table(validate_pipe_load(

pipe_name=>'INGEST_DATA.PUBLIC.TRANSACTION_PIPE',

start_time=>dateadd(hour, -1, current_timestamp())));

copy history imp

https://docs.snowflake.com/en/sql-reference/functions/copy_history.html

using copy history we can find data load failure for snowpipe and normal copy into also

This table function can be used to query Snowflake data loading history along various dimensions. The function returns load activity for both COPY INTO <table> statements and continuous data loading using Snowpipe. The table function avoids the 10,000 row limitation of the LOAD_HISTORY View. The results can be filtered using SQL predicates.

To check error while copying data example:

To use COPY HISTORY OR validate function to check the records failed during loading we have to use on_erros=skipfile other wise we can't see errors using validate function

select * from customer;

copy into customer

from @bulk_copy_example_stage

pattern='.*.csv'

file_format = (type = csv field_delimiter = ',' skip_header = 1)

on_error='skip_file' ;

to see the errors use below query:

select * from table(validate(customer, job_id=>'01955c5c-0059-a582-0000-0000246d54b1'));

Note : The validation returns no results for COPY statements that specify ON_ERROR = ABORT_STATEMENT (default value)

Note :This function does not support COPY INTO <table> statements that transform data during a load

select *

from table(information_schema.copy_history(table_name=>'CUSTOMER', start_time=> dateadd(hours, -1, current_timestamp())));

copy with on error options

To check snow pipe status :

Retrieve the status for a pipe with a case-insensitive name:

select SYSTEM$PIPE_STATUS('ingest_data.public."TRANSACTION_PIPE"');

select SYSTEM$PIPE_STATUS('ingest_data.public.TRANSACTION_PIPE')

Loading Data to Snowflake – 4 Best Methods

https://hevodata.com/learn/loading-data-to-snowflake/

refer udemy Pradeep **Section 10: Load data using internal stage.**

4 Methods of Loading Data to Snowflake

Option 1: You can bulk load large amounts of data using SQL commands in SnowSQL using the Snowflake CLI.

Option 2: You can also automate the bulk loading of data using Snowpipe.

Option 3: You can use the web interface to load a limited amount of data.

Option 4: Lastly, you can implement an Official Snowflake ETL Partner, Hevo Data to load data from external sources in real-time.

Depending on the volume of data you intend to load and the frequency of loading, you can prefer one method over the other for loading data to Snowflake.

Option 1: SnowSQL CLI Client

This post details the process of bulk loading data to Snowflake using the SnowSQL client, but we will touch on the other three methods towards the end of this blog post. Using SQL, you can bulk load data from any delimited plain-text file such as Comma-delimited CSV files. You can also bulk load semi-structured data from JSON, AVRO, Parquet, or ORC files. However, this post focuses on loading from CSV files.

Bulk loading is performed in two phases:

Phase 1 – Staging the files

1. First, you upload your data files to a location where Snowflake can access your files. This is referred to as staging your files.

2. Then you load your data from these staged files into your tables.

Snowflake lets you stage files on internal locations called stages. Each table and user has a stage. Snowflake also supports creating named stages for example demo_stage.

Internal stages enable convenient and secure storage of data files without requiring any external resources. However, if your data files are already staged in a supported cloud storage location such GCS or S3 – you can skip Phase 1 and load directly from these external locations – you just need to supply the URLS for the locations as well as access credentials if the location is protected. You can also create named stages that point to your external location.

Phase 2 – Loading the data

Loading data to Snowflake requires a running virtual warehouse. The warehouse extracts the data from each file and inserts it as rows in the table. Warehouse size can impact loading performance. When loading large numbers of files or large files, you may want to choose a larger warehouse.

You will now learn how to use the SnowSQL SQL client to load CSV files from a local machine into a table named Contacts in the demo database demo_db. CSV files are easier to import into database systems like Snowflake because they can represent relational data in a plain-text file.

You will use a named internal stage to store the files before loading.

Below example is named internal staging

For named staging we will use @stagetable name

Step 1 – Use the demo_db database

Last login: Sun Jun 30 15:31:25 on ttys011

Superuser-MacBook-Pro:Documents hevodata$ snowsql -a bulk_data_load

User: johndoe

Password:

* SnowSQL * V1.1.65

Type SQL statements or !help

johndoe#(no warehouse)@(no database).(no schema)>USE DATABASE demo_db;

+————————————————————————-+

| status |

|————————————————————————-|

| Statement executed successfully. |

+————————————————————————-+

1 Row(s) produced. Time Elapsed: 0.219s

Step 2 – Create the contacts table using the following SQL

johndoe#(no warehouse)@(DEMO_DB.PUBLIC)>CREATE OR REPLACE TABLE contacts (id NUMBER (38, 0) first_name STRING, last_name STRING, company STRING, email STRING, workphone STRING, cellphone STRING, streetaddress STRING, city STRING, postalcode NUMBER (38, 0));

+————————————————————————-+

| status |

|————————————————————————-|

| Table CONTACTS successfully created. |

+————————————————————————-+

1 Row(s) produced. Time Elapsed: 0.335s

Step 3 – Populate the table with records

The CONTACTS table should contain records like this;

1, Chris, Harris, BBC Top Gear, harrismonkey@bbctopgearmagazine.com, 606-237-0055, 502-564-8100, PO Box 3320 3 Queensbridge, Northampton, NN4 7BF

2, Julie, Clark, American Aerobatics Inc, julieclark@americanaerobatics.com, 530-677-0634, 530-676-3434, 3114 Boeing Rd, Cameron Park, CA 95682

3, Doug, Danger, MotorCycle Stuntman LA, dougdanger@mcsla.com, 413-239-7198, 508-832-9494, PO Box 131 Brimfield, Massachusetts, 01010

4, John, Edward, Get Psyched, information@johnedward.net, 631-547-6043, 800-860-7581, PO Box 383

Huntington, New York, 11743

5, Bob, Hope, Bob Hope Comedy, bobhope@bobhope.com, 818-841-2020, 310-990-7444, 3808 W Riverside Dr-100, Burbank, CA 91505

Step 4 – Create an internal stage

Next you will create an internal stage called csvfiles.

johndoe#(no warehouse)@(DEMO_DB.PUBLIC)>CREATE STAGE csvfiles;

+————————————————————————-+

| status |

|————————————————————————-|

| Stage area CSVFILES successfully created. |

+————————————————————————-+

1 Row(s) produced. Time Elapsed: 0.311s

Step 5 – Execute a PUT command to stage the records in csvfiles

johndoe#(no warehouse)@(DEMO_DB.PUBLIC)>PUT file:///tmp/load/contacts0*.csv @csvfiles;

contacts01.csv_c.gz(0.00MB): [##########] 100.00% Done (0.417s, 0.00MB/s),

contacts02.csv_c.gz(0.00MB): [##########] 100.00% Done (0.377s, 0.00MB/s),

contacts03.csv_c.gz(0.00MB): [##########] 100.00% Done (0.391s, 0.00MB/s),

contacts04.csv_c.gz(0.00MB): [##########] 100.00% Done (0.396s, 0.00MB/s),

contacts05.csv_c.gz(0.00MB): [##########] 100.00% Done (0.399s, 0.00MB/s),

+————————-+————————-+————————-+————————————+

| source | target | source_size | target_size | status |

|————————————————————————————|

| contacts01.csv | contacts01.csv.gz | 534 | 420 | UPLOADED |

| contacts02.csv | contacts02.csv.gz | 504 | 402 | UPLOADED |

| contacts03.csv | contacts03.csv.gz | 511 | 407 | UPLOADED |

| contacts04.csv | contacts04.csv.gz | 501 | 399 | UPLOADED |

| contacts05.csv | contacts05.csv.gz | 499 | 396 | UPLOADED |

+————————-+————————-+————————-+————————————+

5 Row(s) produced. Time Elapsed: 2.111s

Notice that:

1. This command uses a wildcard contacts0*.csv to load multiple files.
2. The @ symbol specifies where to stage the files – in this case @csvfiles;
3. By default the PUT command will compress data files using GZIP compression.

Step 6 – Confirm that the csv files have been staged

To see if the files are staged you can use the LIST command.

johndoe#(no warehouse)@(DEMO_DB.PUBLIC)>LIST @csvfiles;

Step 7 – Specify a virtual warehouse to use

Now we are ready to load the files from the staged files into the CONTACTS table. First, you will specify a virtual warehouse to use.

johndoe#(no warehouse)@(DEMO_DB.PUBLIC)>USE WAREHOUSE dataload;

+————————————————————————+

| status |

|————————————————————————|

| Statement executed successfully. |

+————————————————————————+

1 Row(s) produced. Time Elapsed: 0.203s

Step 8 – Load the staged files into a Snowflake table

johndoe#(DATALOAD)@(DEMO_DB.PUBLIC)>COPY INTO contacts;

FROM @csvfiles

PATTERN = '.*contacts0[1-4].csv.gz'

ON_ERROR = 'skip_file';

- INTO specifies where the table data will be loaded.
- PATTERN specifies the data files to load. In this case, we are loading files from data files with names that include the numbers 1-4.
- ON_ERROR tells the command what to do when it encounters errors in the files.

Snowflake also provides powerful options for error handling as the data is loading. You can check out the Snowflake documentation to learn more about these options.

If the load was successful, you can now query your table using SQL:

johndoe#(DATALOAD)@(DEMO_DB.PUBLIC)>SELECT * FROM contacts LIMIT 10;

Option 2: Snowpipe

As you saw earlier in this post, you can also use Snowpipe for bulk loading data to Snowflake – particularly from files staged in external locations. Snowpipe uses the COPY command but with additional features that let you automate this process.

Snowpipe also eliminates the need for a virtual warehouse, instead, it uses external compute resources to continuously load data as files are staged and you are charged only for the actual data loaded.

Option 3: Web Interface

The third option for loading data into Snowflake is the data loading wizard in the Snowflake Web Interface.

The Web UI allows you to simply select the table you want to load and by clicking the LOAD button you can easily load a limited amount of data into Snowflake. The wizard simplifies loading by combining the staging and data loading phases into a single operation and it also automatically deletes all the staged files after loading.

The wizard is only intended for loading small numbers of files of limited size (up to 50 MB). This file size limit is intended to ensure better performance because browser performance varies from computer to computer and between different browser versions. Also, the memory

consumption required to encrypt larger files might cause a browser to run out of memory and crash.

The wizard is intended for loading only small numbers of files containing small amounts of data. For large amounts of data, it is best to use one of the other options.

Option 4: Hevo Data – an Official Snowflake Partner

Hevo is third party etl toll

Snowflake timetravel;

Link;

https://docs.snowflake.com/en/user-guide/data-time-travel.html

Operations allowed: queries,DDL,DML etc

Time travel allowed:

Using Time Travel, you can perform the following actions within a defined period of time:

- Query data in the past that has since been updated or deleted.
- Create clones of entire tables, schemas, and databases at or before specific points in the past.
- Restore tables, schemas, and databases that have been dropped.

Once the defined period of time has elapsed, the data is moved into Snowflake Fail-safe and these actions can no longer be performed.

Time Travel SQL Extensions

To support Time Travel, the following SQL extensions have been implemented:

- AT | BEFORE clause which can be specified in SELECT statements and CREATE … CLONE commands (immediately after the object name). The clause uses one of the following parameters to pinpoint the exact historical data you wish to access:
  - TIMESTAMP
  - OFFSET (time difference in seconds from the present time)
  - STATEMENT (identifier for statement, e.g. query ID)
- UNDROP command for tables, schemas, and databases.

Data Retention Period

A key component of Snowflake Time Travel is the data retention period.

When data in a table is modified, including deletion of data or dropping an object containing data, Snowflake preserves the state of the data before the update. The data retention period specifies the number of days for which this historical data is preserved and, therefore, Time Travel operations (SELECT, CREATE … CLONE, UNDROP) can be performed on the data.

The standard retention period is 1 day (24 hours) and is automatically enabled for all Snowflake accounts:

- For Snowflake Standard Edition, the retention period can be set to 0 (or unset back to the default of 1 day) at the account and object level (i.e. databases, schemas, and tables).
- For Snowflake Enterprise Edition (and higher):
  - For transient databases, schemas, and tables, the retention period can be set to 0 (or unset back to the default of 1 day). The same is also true for temporary tables.
  - For permanent databases, schemas, and tables, the retention period can be set to any value from 0 up to 90 days.

Note

A retention period of 0 days for an object effectively disables Time Travel for the object.

Reference:

https://www.youtube.com/watch?v=IK1LwL_Xl_4

Here are the list of commands used in the video

– DATA_RETENTION_TIME_IN_DAYS

```
show parameters;
show parameters in database SALES;
show parameters in warehouse XMALLFORFINANCE;
show parameters in account;
show parameters like '%DATA%';
show parameters like 'DATA%' in account;
–CHANGES DATA RETENTION FOR THE DATABASEshow parameters in database SALES;

alter DATABASE SALES set DATA_RETENTION_TIME_IN_DAYS = 1 ;

–CHANGES DATA RETENTION FOR THE WHOLE ACCOUNTshow parameters like 'DATA%' in account;

alter ACCOUNT set DATA_RETENTION_TIME_IN_DAYS = 1 ;
```

select * from PATIENTS;
INSERT INTO PATIENTS values (5,'TEST PATIENT','AMSTERDAM','Mayo Clinic');

select * from PATIENTS at(offset => -60*5);
delete from PATIENTS where id=5;

drop table sales.snow.PATIENTS ;
–Over a dayselect * from PATIENTS at(offset => -60*500000);

select * from PATIENTS at(timestamp => 'Sat, 16 Mar 2019 16:20:00 -0700'::timestamp);

select * from user_access;
–Before changes make by a specific query

select * from patients before(statement => '025e545d-fc23-4e8d-9ac5-335943a1bec2');
create table restored_table clone user_access at(timestamp => 'Mon, 09 May 2019 01:01:00 +0300'::timestamp);

create table restored_table clone user_access at(offset => -60*10);

select * from restored_table; drop table restored_table; –Schema as existed 1 hour before
create schema restored_schema clone snow at(offset => -3600);

drop schema restored_schema;

create database restored_db clone sales before(statement => '025e545d-fc23-4e8d-9ac5-335943a1bec2'); drop database restored_db; show tables history in sales.snow;show tables history in sales;—Restoring objects
undrop table patients;
undrop schema snow;
undrop database sales;

Note: retention period we can set at database level, schema level, table lavel.

Means in in database and same schema different table may have different retention periods

Changing the retention period for your account or individual objects changes the value for all lower-level objects that do not have a retention period explicitly set. For example:

- If you change the retention period at the account level, all databases, schemas, and tables that do not have an explicit retention period automatically inherit the new retention period.
- If you change the retention period at the schema level, all tables in the schema that do not have an explicit retention period inherit the new retention period.

Keep this in mind when changing the retention period for your account or any objects in your account because the change may have Time Travel consequences that you did not anticipate or intend. In particular, we do ***not*** recommend changing the retention period to 0 at the account level.

FAIL-SAFE in SnowFlake

CLONING IN SNOWFLAKE:

https://community.snowflake.com/s/article/cloning-in-snowflake

Internal (i.e. Snowflake) named stages ***cannot*** be cloned.

Cloning and Stages

Individual external named stages can be cloned. An external stage references a bucket or container in external cloud storage; cloning an external stage has no impact on the referenced cloud storage.

Internal (i.e. Snowflake) named stages ***cannot*** be cloned.

When cloning a database or schema:

- External named stages that were present in the source when the cloning operation started are cloned.
- Tables are cloned, which means the internal stage associated with each table is also cloned. Any data files that were present in a table stage in the source database or schema are not copied to the clone (i.e. the cloned table stages are empty).
- Internal named stages are ***not*** cloned.

Cloning and Streams

Currently, when a database or schema that contains source tables and streams is cloned, any unconsumed records in the streams (in the clone) are inaccessible. This behavior is consistent with Time Travel for tables. If a table is cloned, historical data for the table clone begins at the time/point when the clone was created.

Cloning and Tasks

When a database or schema that contains tasks is cloned, the tasks in the clone are suspended by default. The tasks can be resumed individually (using ALTER TASK ... RESUME).

Cloning and Clustering Keys

A table can have a subset of columns designated as a clustering key to co-locate similar rows in the same micro-partition. When a table with a clustering key is cloned, the new table is created with a clustering key. By default, Automatic Clustering is suspended for the new table. To resume automatic clustering for the new table, run the following command:

ALTER TABLE <*name*> RESUME RECLUSTER

Impact of DDL on Cloning

Cloning is fast, but not instantaneous, particularly for large objects (e.g. tables). As such, if DDL statements are executed on source objects (e.g. renaming tables in a schema) while the cloning operation is in progress, the changes may not be represented in the clone. This is because DDL statements are atomic and not part of multi-statement transactions.

Furthermore, Snowflake does not record which object names were present when the cloning operation started and which names changed. As such, DDL statements that rename (or drop and recreate) source child objects compete with any in-progress cloning operations and can cause name conflicts.

In the following example, the t_sales table is dropped and another table is altered and given the same name as the dropped table while the parent database is being cloned, producing an error:

create or replace database staging_sales clone sales;

drop table sales.public.t_sales;

alter table sales.public.t_sales_20170522 rename to sales.public.t_sales;

002002 (42710): None: SQL compilation error: Object 'T_SALES' already exists.

Tip

To avoid conflicts in name resolution during a cloning operation, we suggest refraining from renaming objects to a name previously used by a dropped object until cloning is completed.

Impact of DML and Data Retention on Cloning

The DATA_RETENTION_TIME_IN_DAYS parameter specifies the number of days for which Snowflake retains historical data for performing Time Travel actions on an object. Because the data retained for Time Travel incurs storage costs at the table-level, some users set this parameter to 0 for some tables, effectively disabling data retention for these tables (i.e. when the value is set to 0, Time Travel data retained for DML transactions is purged, incurring negligible additional storage costs).

Cloning operations require time to complete, particularly for large tables. During this period, DML transactions can alter the data in a source table. Subsequently, Snowflake attempts to clone the table data as it existed when the operation began. However, if data is purged for DML transactions that occur during cloning (because the retention time for the table is 0), the data is unavailable to complete the operation, producing an error similar to the following:

programmingerror occured: “000707 (02000): None: Data is not available.” with query id none

Tip

As a workaround, we recommend ***either*** of the following best practices when cloning an object:

- Refrain, if possible, from executing DML transactions on the source object (or any of its children) until after the cloning operation completes.
- If this isn’t possible, prior to starting cloning, set DATA_RETENTION_TIME_IN_DAYS=1 for all tables in the schema (or database if you are cloning an entire database). Once the operation completes, remember to reset the parameter value back to 0 for those tables in the source, if desired.

  You might also want to set the value to 0 for the cloned tables (if you plan to make DML changes to the cloned tables and do not wish to incur additional storage costs for Time Travel on the tables).

TABLE COLING :

```
CREATE OR REPLACE TABLE demo_db.public.employees_clone
  CLONE employees;
```

- Cloning Databases

– Create a database clone (demo_db_clone) from the original database demo_db

```
CREATE or replace DATABASE demo_db_clone CLONE demo_db;
```

```
– Cloning Schema
– Create a cloned schema from the original public schema
CREATE or replace SCHEMA public_clone CLONE public;
```

EMIRATES SNOW FLAKES DATA FLOW

Skywards data

HELIX to azure

From Azure to snowflake staging table .. schedule oozie scheduler

From snow flake table to to main table table to table -→ ozzie scheduler

Snaplogic → AWS s3 bucket→ autoinject using snow pipe. Power bi will connect to tables of snowflake.

—-pnrdw new data flow with oracle as destination

Tibco to –helix—kafka – snaplogic—oracle

pnrdw new data flow with snowflake as destination

scheduling using zookeper

Tibco to –helix—kafka – snaplogic—aws s3— snowpipe — staging raw table –tasks n stream—final table— create secure views –access to these views to reporting module

Snaplogic Kafka consumer sample . it will take data from kafka topic

---

To find errors in bulk load using copy command:

Validate Errors

The following process returns errors by query ID and saves the results to a table for future reference.

You can view the query ID for the COPY job on the **History** page of the web interface:

9. Log into the Snowflake web interface.
10. Change to the role you have been using to run the tutorial SQL statements.
11. Click **History** .
12. Click the **Query ID** column link for the COPY INTO command. The **Details** panel opens.
13. Copy the **Query ID** value.
14. In the command line interface (e.g., SnowSQL), execute the following command. Replace *query_id* with the **Query ID** value.
15. create or replace table save_copy_errors as select * from table(validate(mycsvtable, job_id=>'<query_id>'));
16. Query the results table:

    select * from save_copy_errors;

Introduction to Data Pipelines

https://docs.snowflake.com/en/user-guide/data-pipelines-intro.html#workflow

https://www.youtube.com/watch?v=AlOgnPCZNf4 → examples to implemt taskts and streams

Data pipelines automate many of the manual steps involved in transforming and optimizing continuous data loads. Frequently, the “raw” data is first loaded temporarily into a staging table used for interim storage and then transformed using a series of SQL statements before it is inserted into the destination reporting tables. The most efficient workflow for this process involves transforming only data that is new or modified.

In this Topic:

In this Topic:

Features Included in Continuous Data Pipelines

Workflow

Features Included in Continuous Data Pipelines

Snowflake provides the following features to enable continuous data pipelines:

Continuous data loading

Options for continuous data loading include the following:

- Snowpipe
- Snowflake Connector for Kafka
- Third-party data integration tools

Change data tracking

A *stream* object records the delta of change data capture (CDC) information for a table (such as a staging table), including inserts and other data manipulation language (DML) changes. A stream allows querying and consuming a set of changes to a table, at the row level, between two transactional points of time.

In a continuous data pipeline, table streams record when staging tables and any downstream tables are populated with data from business applications using continuous data loading and are ready for further processing using SQL statements.

For more information, see Change Tracking Using Table Streams.

Recurring tasks

A *task* object defines a recurring schedule for executing a SQL statement, including statements that call stored procedures. Tasks can be chained together for successive execution to support more complex periodic processing.

Tasks may optionally use table streams to provide a convenient way to continuously process new or changed data. A task can transform new or changed rows that a stream surfaces. Each time a task is scheduled to run, it can verify whether a stream contains change data for a table (using SYSTEM$STREAM_HAS_DATA) and either consume the change data or skip the current run if no change data exists.

Users can define a simple tree-like structure of tasks that executes consecutive SQL statements to process data and move it to various destination tables.

For more information, see Executing SQL Statements on a Schedule Using Tasks.

Workflow

The following diagram illustrates a common continuous data pipeline flow using Snowflake functionality:

1. One of the following Snowflake features or a third-party data integration tool (not shown) loads data continuously into a staging table:

Snowpipe

Snowpipe continuously loads micro-batches of data from an external stage location (Amazon S3, Google Cloud Storage, or Microsoft Azure) into a staging table.

Snowflake Connector for Kafka

The Kafka connector continuously loads records from one or more Apache Kafka topics into an internal (Snowflake) stage and then into a staging table using Snowpipe.

2. One or more table streams capture change data and make it available to query.
3. One or more tasks execute SQL statements (which could call stored procedures) to transform the change data and move the optimized data sets into destination tables for analysis. Each time this transformation process runs, it selects the change data in the stream to perform DML operations on the destination tables and then consumes the change data when the transaction is committed.

Example of stream and tasks :

A stream stores data in the same shape as the source table (i.e. the same column names and ordering) with following additional columns:

Column	Description
METADATA$ACTION	Indicates the DML operation (INSERT, DELETE) recorded
METADATA$ISUPDATE	Indicates whether the operation was part of an UPDATE statement. Updates to rows in the source table are represented as a pair of DELETE and INSERT records in the stream with a metadata column METADATA$ISUPDATE values set to TRUE.
METADATA$ROW_ID	Specifies the unique and immutable ID for the row, which can be used to track changes to specific rows over time

https://www.snowflake.com/blog/building-a-type-2-slowly-changing-dimension-in-snowflake-using-streams-and-tasks-part-1/

https://www.snowflake.com/blog/building-a-type-2-slowly-changing-dimension-in-snowflake-using-streams-and-tasks-part-2/

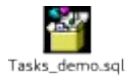
https://saamaanalytics.com/blogs/building-etl-and-scd-with-snowflake-streams-tasks/

https://docs.snowflake.com/en/sql-reference/sql/create-task.html

https://saamaanalytics.com/blogs/using-matillion-for-data-loading-into-snowflake-metadata-driven-approach/

CONTROL LOGGING WITH STREAMS AND TASKS refer:

https://support.snowflake.net/s/article/Control-Logging-with-Streams-and-Tasks



how to find queries executed in a task and query id ,task status

Tak_history

https://docs.snowflake.com/en/sql-reference/functions/task_history.html

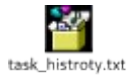
is used to get the info.

This table function can be used to query the history of task usage within a specified date range. The function returns the history of task usage for your entire Snowflake account or a specified task.

This function returns task activity within the last 7 days.

Note

To retrieve only tasks that are completed or still running, filter the query using WHERE query_id IS NOT NULL. Note that this filter is applied ***after*** RESULT_LIMIT already reduces the results returned, so the query could return 9 tasks if 1 task was scheduled but had not started yet.


task_histroty.txt

Email notifications from snowflake

https://docs.snowflake.com/user-guide/notifications/email-stored-procedures

https://select.dev/posts/snowflake-alerts

How to create a notification integration

Tasks new concepts DAG , PUSHING ERROR NOTIFICATIONS TASK GRAPHS, DATA LOAD SCHEMA EVOLUTION

https://docs.snowflake.com/en/user-guide/tasks-intro#label-finalizer-task

https://docs.snowflake.com/en/user-guide/tasks-graphs

https://docs.snowflake.com/en/user-guide/tasks-errors —- pushing task error notifications

https://docs.snowflake.com/en/user-guide/data-load-schema-evolution

https://medium.com/snowflake/how-to-view-snowflake-task-graphs-visualization-in-dag-format-4e08373abacb

https://www.youtube.com/watch?v=_gi_mVlFZgM

Snowflake Task graphs Visualization in DAG format

Snowflake Task graphs depict executions of a root task and child tasks in DAG format. In this video , the way to visualize Task Graph is explained with a practical demo. Prerequisite:

———————————— Snowflake Stored Procedure Parallel Execution using SQL in-depth intuition (Part 1) https://youtu.be/lC52sAIaIyo Snowflake Stored ...

www.youtube.com

https://www.youtube.com/watch?v=kR_RUIfzuWI

below has all latest snowflake topics incluidng snowpark , task dag , stream types

https://www.youtube.com/playlist?list=PLrFeXmwYWoDBFeJH0mTMEr4jujklASATG

SNOWFLAKE TUTORIAL

www.youtube.com

Streams Types and stream staleness

https://thinketl.com/change-data-capture-using-snowflake-streams/#google_vignette

Types of Snowflake Streams

There are three different types of Streams supported in Snowflake

1. Standard
2. Append-only
3. Insert-only

1. Standard Streams

A Standard (i.e. delta) stream tracks all DML changes to the source object, including inserts, updates, and deletes (including table truncates). Supported on standard tables, directory tables and views.

The syntax to create Standard stream is as below

CREATE OR REPLACE STREAM my_stream ON TABLE my_table;

2. Append-only Streams

An Append-only stream tracks row inserts only. Update and delete operations (including table truncates) are not recorded. Supported on standard tables, directory tables and views.

The syntax to create Append-only streams similar to Standard streams except that the APPEND_ONLY parameter value needs to be set to TRUE as below

CREATE OR REPLACE STREAM my_stream ON TABLE my_table

APPEND_ONLY = TRUE;

3. Insert-only Streams

Supported for **External tables** only. An insert-only stream tracks row inserts only. They do not record delete operations that remove rows from an inserted set.

The syntax to create Insert-only stream is as below

CREATE OR REPLACE STREAM my_stream ON EXTERNAL TABLE my_table

INSERT_ONLY = TRUE;

he below select statements on the stream gives the details of records which needs to be inserted, updated and deleted in the target table.

–INSERT

SELECT * FROM MY_STREAM

WHERE metadata$action = 'INSERT'

AND metadata$isupdate = 'FALSE';

–UPDATE

SELECT * FROM MY_STREAM

WHERE metadata$action = 'INSERT'

AND metadata$isupdate = 'TRUE';

–DELETE

SELECT * FROM MY_STREAM

WHERE metadata$action = 'DELETE'

AND metadata$isupdate = 'FALSE';

Finally we can use a **MERGE** statement with the Stream using these filters to perform the insert, update and delete operations on target table as shown below.

MERGE INTO EMPLOYEES a USING MY_STREAM b ON a.ID = b.ID

WHEN MATCHED AND metadata$action = 'DELETE' AND metadata$isupdate = 'FALSE'

THEN DELETE

WHEN MATCHED AND metadata$action = 'INSERT' AND metadata$isupdate = 'TRUE'

THEN UPDATE SET a.NAME = b. NAME, a.SALARY = b.SALARY

WHEN NOT MATCHED AND metadata$action = 'INSERT' AND metadata$isupdate = 'FALSE'

THEN INSERT (ID, NAME, SALARY) VALUES (b.ID, b.NAME, b.SALARY)

;

Stream Staleness

A stream becomes stale when its offset is outside of the data retention period for its source table. When a stream becomes stale, the historical data for the source table is no longer accessible, including any unconsumed change records.

To view the current staleness status of a stream, execute the **DESCRIBE STREAM** or **SHOW STREAMS** command. The **STALE_AFTER** column timestamp indicates when the stream is currently predicted to become stale.

To avoid having a stream become stale, it is strongly recommended to regularly consume the changed data before its ***STALE_AFTER*** *timestamp.*

If the data retention period for a table is **less than 14 days**, and a stream has not been consumed, Snowflake temporarily extends this period to prevent it from going stale. The period is extended to the stream's offset, up to a maximum of 14 days by default, regardless of the Snowflake edition for your account.

Note that this restriction does not apply to streams on **Directory tables** or **External tables**, which have no data retention period.

snowflake sql commands:

The external stages can also be created using SQL statements by passing cloud storage credentials as shown below.

use schema mydb.public;

– Creating an external stage in Snowflake on Azure location

CREATE STAGE my_azure_stage

URL = ‘azure://myazurespace.blob.core.windows.net/snowflake’

CREDENTIALS = (AZURE_SAS_TOKEN = ‘**********’);

– Creating an external stage in Snowflake on AWS S3 location

CREATE STAGE my_s3_stage

URL = ‘s3://mybucket/snowflake/’

CREDENTIALS = (AWS_KEY_ID = ‘**********’ AWS_SECRET_KEY = ‘**********’);

Creating storage integration and external stage

Amazon S3

create storage integration s3_int

type = external_stage

storage_provider = s3

enabled = true

storage_aws_role_arn = ‘<iam_role>’

storage_allowed_locations = (‘s3://bucket/path/’, ‘s3://bucket/path2/’);

AZURE

create storage integration azure_int

type = external_stage

storage_provider = azure

enabled = true

azure_tenant_id = ‘<tenant_id>’

storage_allowed_locations = (‘azure://account.blob.core.windows.net/container/path/’,‘azure://account.blob.core.windows.net/container/path2/’);

desc integration s3_int;

Create an External Stage

Amazon S3

Creating an External Stage on AWS S3

create stage my_s3_stage

storage_integration = s3_int

url = ‘s3://bucket1/path1’

file_format = my_csv_format;

Microsoft Azure

Creating an External Stage on Azure

create stage my_azure_stage

storage_integration = azure_int

url = ‘azure://myaccount.blob.core.windows.net/container1/path1’

file_format = my_csv_format;

Stored procedures must write in java only

Important interview questions:

NTT Data

1) 1.DEFUALT WAREHOUSE SIZE : XSMALL capgemini interview

2) HOW TO FIND TABLE size

Ans: from gui if we mouser over on table then it will show

Or

Show table like '%CUSTOMER% – This query o/p will have column name bytes which give table size

OR

TABLE_STORAGE_METRICS View

Snowflake query error: as Unsupported subquery type cannot be evaluated

RESULT_SCAN

When working with cached results, the following functions are useful:

- RESULT_SCAN, to access the cached result directly: https://docs.snowflake.net/manuals/sql-reference/functions/result_scan.html

Example: Run a complex query, and then to access its results again, execute the following query:

select * from table(result_scan(last_query_id()))

3) Tasks
4) Streams cdc changes
5) Oracle analytical function
6) Joins
7) LEAD, LAG analytical function
8) Scheduling tool used to schedule snowflake scripts as part of batch load
9) Time taken for migration – 4 to 5 months
10) Which tool is used for ajile – https://jiraagile.emirates.com/
11) What is restrospective meeting

Process flow for scrum

Refer below link

https://www.atlassian.com/agile/tutorials/how-to-do-scrum-with-jira-software

AJAIL SCRUM EK PROCESS FLOW.doc

SNOWFLAKE Interview Qs.docx

SELECT empno,

deptno,

sal,

RANK() OVER (PARTITION BY deptno ORDER BY sal) AS myrank

FROM emp;

EMPNO DEPTNO SAL MYRANK

7934 10 1300 1

7782 10 2450 2

7839 10 5000 3

7369 20 800 1

7876 20 1100 2

7566 20 2975 3

7788 20 3000 4

7902 20 3000 4

7900 30 950 1

7654 30 1250 2

7521 30 1250 2

7844 30 1500 4

7499 30 1600 5

7698 30 2850 6

When multiple rows share the same rank the next rank in the sequence is not consecutive.

The DENSE_RANK function acts like the RANK function except that it assigns consecutive ranks, so this is not like olympic medaling.

SELECT empno,

deptno,

sal,

DENSE_RANK() OVER (PARTITION BY deptno ORDER BY sal) AS myrank

FROM emp;

EMPNO DEPTNO SAL MYRANK

7934 10 1300 1

7782 10 2450 2

7839 10 5000 3

7369 20 800 1

7876 20 1100 2

7566 20 2975 3

7788 20 3000 4

7902 20 3000 4

7900 30 950 1

7654 30 1250 2

7521 30 1250 2

7844 30 1500 3

7499 30 1600 4

7698 30 2850 5

12) Table design best practices (refer below links or attached NTT document)

https://hevodata.com/blog/snowflake-etl-best-practices-cloud-data-warehouse/#table_design

https://docs.snowflake.com/en/user-guide/table-considerations.html

1. Data Warehouse Considerations
2. Table Design Considerations
3. Data Storage Considerations
4. Data Loading Considerations
5. Planning a Data Load
6. Data Staging Considerations
7. A Robust and Easy Way to Perform Snowflake ETL

13) **Control M scheduler integration with snowflake to schedule snoqsql quries**

YTD query –r QUESTION

MTD query

What is MTD?

Month to date (MTD) is a period that starts from the beginning of current a calendar month and ends at the current date, but it does not include the date of the present day.Suppose, if the present day is 19th Sep then, your MTD will cover the data from the time period of 1st Sep – 18th Sep.

MTD query will consider upto yesterday MTD query below

Today 25th august so We will sum the values from 01- aug-2020 to till yesterday i.e 24-aug-2020.

YTD: Year to date (YTD) is a period beginning from the first day of the current year (either calendar year or fiscal year) continuing up to the current day. A calendar year is a period from January 1st to 31st December whereas a fiscal year is a time period lasting for a year but beginning from January 1st is not necessary. The information gathered from YTD is quite useful for analyzing the improvement in the business, comparing the performance data, etc

Jadeglobal interview questions

1.Result cache will store data for 24 hours ,if we run the same query at 23 rd houd then same data will be there in results cache. If keep on run the same query at 23rd hour then how any days this result cache will store the data?

Sol :Each time the persisted result for a query is reused, Snowflake resets the 24-hour retention period for the result, up to a maximum of 31 days from the date and time that the query was first executed. After 31 days, the result is purged and the next time the query is submitted, a new result is generated and persisted.

2. HOW TO: RECOVER AN ACCIDENTALLY DROPPED TABLE RECREATED AND POPULATED WITH NEW DATA

Solution 1

In order to recover the dropped table, you will need to perform the following steps:

1. **Rename the recreated table:**

   alter table <table_name> rename to <new_name>;
2. **Execute UNDROP to reclaim the dropped table:**

   undrop table <original_name>;

   solution 2:

   clone the entire schema ..in new cloned schema by using time travel we can recover the accidently deleted data from the table.

3. ERROR: YOUR STATEMENT '<ID>' WAS ABORTED BECAUSE THE NUMBER OF WAITERS EXCEEDS THE 20 STATEMENTS LIMIT

Problem Description

When executing INSERT, COPY, UPDATE, MERGE, or DELETE statements from multiple clients simultaneously, you could encounter the following error:

Your statement '<id>' was aborted because the number of waiters exceeds the 20 statements limit

Cause

Snowflake limits the number of certain types of DML statements that target the same table from multiple clients.

Solution
Several options are available:

- **Issue the DML statements using no more than the statement limit mentioned in the error message. e.g. 20 clients.**
- **Gather all updates happening during a session, and issue a single DML statement that includes all of them.**
- **Create multiple audit/temporary tables over the course of a session, and at the end of the session copy all the records from them to the main table.**

Not able to insert timestamp or date into snowflake as NTZ format

I am trying to insert a timestamp value from an input table in snowflake to an output table in snowflake. Getting error

"*Expression type does not match column data type, expecting TIMESTAMP_NTZ(9) but got TIMESTAMP_LTZ(9) for column ABC*"

Snowflake wont allow **timestamp data** if table column is defined as *TIMESTAMP_NTZ(9.* *Solution* is to change the datatype of column from timestamp_ntz to timestamp_tz .

If column data type is timestamp_tz then we can insert data with time zone and data without time zone also

Below are from SNOWFLAKE MONJI interview good questions

3. **For performance point of view how much we need clustering depth**

   Ans: 0 or 1
4. **How to find which column is used for auto clustering**
5. **To use select * from named stage alias name is mandatory**

6. **If we shared secured view then updated data will be reflected or not to consumer? I guess yes**
7. **I created streams on table. After than few dml operations are performed .. after that table is re crated using create or replace .. then stream will hold data or not?..... stream wont hold data as stream will get data from cache bcz we used create or replace cache is cleared so data wont be there in stream**
8. **Yesterday deleted record I want to insert again into the table.. how we do?**

Ans: insert overwrite into emp select * from emp with time travel offset value

9. **We cant create materialized views by using joins.even it wont allow self join also.**

we can create materilised view as: select * from emp;... single table only

Limiting and controlling Snowflake spend

1. Choosing warehouse sizes

Choose proper lize like small, medium based on requirement

2. Setting warehouse auto-suspend and auto-resume (on by default) and other limits

3.resource monitors

4.budgets

10.ware house resource monitors

In the Cloud computing era with pay-as-you go resources it is necessary to have a billing alerts set to get notified when there are unexpected spend increases. This is an extremely useful tool to keep a close watch on your resource usage and stay on-budget.

Resource Monitors in Snowflake assist in cost management and prevent unforeseen credit usage caused by operating warehouses. They issue alarm alerts and helps in stopping user-managed warehouses when certain limits are reached or approaching.

Resource monitors can only be created by Account Administrators (i.e. users with the ACCOUNTADMIN role).

However, Account Administrators can choose to enable users with other roles to view and modify resource monitors using SQL.

*A Resource Monitor must have **at least one action defined. If no actions have been defined, nothing happens when the used credits reach the threshold.***

Actions

You can define certain defined actions when the credit quota is reaches a certain limit. Following are the actions that resource monitors support.

- One **Suspend and Notify** action.
- One **Suspend Immediately and Notify** action.
- Up to five **Notify** actions.

Suspend: Suspends all assigned warehouses after all statements being executed by the warehouse(s) have completed.

Suspend Immediately: Suspends all assigned warehouses immediately, which cancels any statements being executed by the warehouses at the time.

Monitor Level/monitor type

The Resource Monitors in Snowflake can monitor the credit usage at two different levels.

- **ACCOUNT**: At the Account level i.e. all the warehouses in the account.
- **WAREHOUSE**: At the individual Warehouse or a group of warehouses level.

If you have selected the Monitor level as Warehouse, you need to individually select the Warehouses to monitor.

Creating resource monitor using sql query

The below image shows an example of resource monitor with default schedule.

We can create recource monitors from ui only..go to admin →costmanagemet.

Budgets

https://docs.snowflake.com/en/user-guide/budgets/config

https://medium.com/snowflake/take-control-of-your-snowflake-spend-with-budgets-now-generally-available-00943f9d8ad6

Budgets is the latest feature in Snowflake's cost control portfolio that monitors Snowflake compute credits across warehouses and serverless features (auto-clustering, replication, search optimization, etc.) within a Snowflake account.

You can set up a budget for the entire account which will monitor all the compute resources within the account for the month, against the spending limit set for that account

A budget can monitor the following Snowflake objects:

Object

Monitored costs

Compute pool

Compute pool usage for Snowpark Container Services. For more information, see Compute pool cost.

Databases

When you add a database to a budget, all supported objects the database contains are also automatically added. The budget monitors credit usage for the following objects and serverless features:

Supported schema objects as described above.

Replication for secondary (replica) databases.

Note

Replication costs for secondary databases that are replicated in a replication or failover group can only be monitored by the account budget.

Materialized views

Background maintenance for the materialized view. For more information, see Materialized Views Cost.

Pipes

Resource consumption for loading data using Snowpipe. For more information, see Snowpipe costs.

Tables

Background maintenance operations for automatic clustering and search optimization if they are enabled on the table.

Tasks

Serverless tasks are monitored by a custom budget. To monitor the credit usage for a task that executes using a user-managed warehouse, you must add the warehouse to the budget. For more information, see Task costs.

Schemas

When you add a schema to a budget, all supported objects the schema contains are also automatically added. The budget monitors the credit usage for schema objects as described above.

Warehouses

Compute resources for query execution, web interface, and other features (see Virtual warehouse credit usage), serverless tasks, and cloud services compute.

Budget Types

1. Account Level Budget

2. Custom Budget

The *account budget* monitors spending for all supported objects in the account. To get notifications for the account budget, set up the spending limit and specify email addresses to receive notifications.

You can also create a *custom budget* to monitor the spending limits for a specific group of supported objects. Like the account budget, you must set the spending limit and the notification email addresses in order to receive notification emails.

Account Level Budget:

Setting up this budget works on account level and may not be good use case where Snowflake account is used in big enterprise having multiple enterprise functions using isolated Snowflake resources under one or multiple Snowflake accounts mapped to one organization.

Activate and set up the account budget using SQL

Create a custom budget using Snowsight

Create a custom budget using SQL

Add or remove objects from a custom budget using SQL

13 .**How to handle special charcaters/Chinese characters while data loading using copy command**

When we are loading data from source file to stage/target using copy command if source file has junk characters or Chinese characters .. to avoid to load these junk we can use ... this option will convert non UTF-8 character into **.**

When we are performing history load from Teradata to snowflake .. Teradata has junk characters to avoid that we used **REPLACE_INVALID_CHARACTERS =TRUE**

REPLACE_INVALID_CHARACTERS = TRUE | FALSE

Use

Data loading only

Definition

Boolean that specifies whether to replace invalid UTF-8 characters with the Unicode replacement character (**).

If set to TRUE, Snowflake replaces invalid UTF-8 characters with the Unicode replacement character.

If set to FALSE, the load operation produces an error when invalid UTF-8 character encoding is detected.

Default

FALSE

Copy option PURGE=TRUE

Copy option FORCE=TRUE

14.What is column collation in snowflake ... HUMANA interview question

15.is it possible to drop default clustering key ...

Ans :Snowflake doesnot apply any default clustering key…. If clustering key is existed then we can drop .

ALTER TABLE <name> DROP CLUSTERING KEY

Note: even if we not created cluster key on table and we use drop cluster key command it wont throw error ...looks like snowflake is maintaing clustering key internally.

to summarize:

- **No Default Clustering Key**: Snowflake does not apply a default clustering key to tables.
- **Automatic Optimization**: Snowflake manages data organization internally through micro-partitions.
- **Custom Clustering Key**: For large tables or specific query patterns, you might manually define clustering keys to improve performance.

How to load data files that has column mismatch with table columns

Sometimes we can get few additional columns in source file and sometimes few columns are missing in the csv file

Note: The order of the columns the same in both cases.

There are scenarios where you need to load data files where the column count in the data file does not match with column available in the snowflake table. Then how will you run copy command and what kind of changes need to be done to accommodate this requirement?

how to load data files that has column mismatch issue and still get loaded into target table using copy command.

Sol is : ERROR_ON_COLUMN_COUNT_MISMATCH =FLASE

Default: TRUE

If set to FALSE, an error is not generated and the load continues. If the file is successfully loaded:

- If the input file contains records with more fields than columns in the table, the matching fields are loaded in order of occurrence in the file and the remaining fields are not loaded.
- If the input file contains records with fewer fields than columns in the table, the non-matching columns in the table are
- loaded with NULL values.

Example 1. Copy command

Example 2: creating file format with ERROR_ON_COLUMN_COUNT_MISMATCH

Use above file format in copy command below

Schema detection and schema evolution

https://www.phdata.io/blog/schema-detection-and-evolution-in-snowflake/

In this blog we would discuss about Snowflake's ability to solve problems around evolving schema & detecting them in the right manner. Imagine we have a use case where we continuously get the files in internal stage and there is a need to dynamically identify the schema of the file & not only that create some tables out of it dynamically. Apart from these 2 also think of a case where if the incoming files schema changes like more columns getting added how do we ensure that those data is dynamically getting processed through the same pipeline. This is where within Snowflake we can use couple of features i.e.,

1. Schema detection(INFER SCHEMA)
2. Schema evolution(ENABLE_SCHEMA_EVOLUTION)

INFER_SCHEMA help detects and returns the schema from a staged file. This SQL function supports named stages (internal or external) and user stages only. It does not support table stages.

Schema evolution is a feature where Snowflake attempts to scan incoming data as part of the COPY command to identify any new column to be added and add them to the table before loading the data into the table. To enable schema evolution we have to alter table and enable schema evolution

Note the option PARSE_HEADER=TRUE in the file format option. INFER_SCHEMA uses this to generate the field names. The output of this function will look like the following:

The data type generated will depend on the number of rows parsed from the file. If you use a relatively small sample, then the datatype may be inaccurate.

Verify if the data is loaded by querying the table.

The COPY command succeeds now and loads the data into CUSTOMER_TABLE with 2 additional columns. For the older records, the value will be set to NULL. Validate the data by querying the table.

Below screen shot explains file format option PARSE_HEADER=TRUE

Infer schema issues :

Using INFER_SCHEMA with CSV files throws the error: “empty string in the header is not allowed”

Issue: When using the INFER_SCHEMA function to load staged CSV files, it fails with the following error:

Error with CSV header: empty string in the header is not allowed File ‘<File Name>’ Row 0 starts at line 0.

Cause

This error occurs under the below conditions:

- The first row of the file contains one or more null values
- The file format in use has the option **PARSE_HEADER = true**

If PARSE_HEADER is set to true, the first row in the CSV file will be used to determine column names. Therefore if any of these values are null, the above error will occur.

Solution

To continue using INFER_SCHEMA and avoid the error, one of the following actions should be taken:

- Remove the **PARSE_HEADER = true** option from the file format. The default value FALSE will return column names as **c***, where * is the position of the column.
  - (Note that the SKIP_HEADER option is not supported with PARSE_HEADER = **true**).
- Keep **PARSE_HEADER = true** but ensure the headers in the file contain no null values.

Data governance in snowflake

https://medium.com/snowflake/snowflake-data-governance-row-access-policy-overview-91b50d604a57

https://medium.com/snowflake/snowflake-data-governance-object-tagging-overview-3f9acf431a0

https://medium.com/snowflake/snowflake-data-governance-access-history-overview-8bd7866b1410

Row access policy

How to define two digit century value in snowflake when we use only YY

SNOWFLAKE has below parameters . below parameter has to change from which year we have to consider we use year format as YY

TWO_DIGIT_CENTURY_START

DUPLICATE DELETION IN SNOWFLAKE :

How to remove duplicate record based on KEY field in Snowflake table

In some instances, there are duplicate records based on the KEY column and not full row dupes. In this case, you can safely remove the unwanted record alone using the below method. In this example, we are keeping the latest record for each KEY field and removing the oldest record.

KEY	COLX	COLY	COLZ
1	A	B	2018-10-01
1	A	B	2018-10-02
2	A	B	2018-10-01
2	A	B	2018-10-02

```
DELETE FROM MY_TABLE T
USING (
SELECT KEY,colx,coly,COLZ
,ROW_NUMBER() OVER (PARTITION BY KEY,colx,coly ORDER BY key,COLZ) AS
```

RANK_IN_KEY
FROM MY_TABLE T
) X
WHERE T.KEY = X.KEY
AND T.COLX = X.COLX

AND T.COLY=X.COLY

AND T.COLZ!=X.COLZ

AND RANK_IN_KEY >1;

AFTER EXECUTING ABOVE QUERY table will have

KEY	COLX	COLY	COLZ
1	A	B	2018-10-02
2	A	B	2018-10-02

NOTE: IF ALL COLUMNS IN THE TABLE ARE DUPLICATE VALUE, THEN I ABOVE QUERY WONT WORK. WE HAVE TO USE INSERT OVERWRITE

INSERT OVERWRITE INTO EMP SELECT DISTINCT * FROM EMP;

DELETE FROM tempa using

(

SELECT id,amt, ROW_NUMBER() OVER (PARTITION BY amt ORDER BY id) AS rn

FROM tempa

) dups

WHERE tempa.id=dups.id and dups.rn > 1;—— this wont work

Minus functionality using join condition

Emp table

empno
1
2
3
4

Tmp table

empno
3
4

If we write

select empno from emp

minus

select empno from tmp

then we will get 1 2 as o/p to implement same thing using joins

```
select a.empno
from emp a left join tmp b
on(a.empno=b.empno)
where b.empno is null;
```

explanation: if we write left join means we will get all records from emp table and for non matched rows we will get null value from tmp table

so we to get minus operation then has to add condition as b.empno is null then we will get records only present in emp table but not present in tmp table

SOLUTION: STATEMENT REACHED ITS STATEMENT OR WAREHOUSE TIMEOUT OF XXX SECOND(S) AND WAS CANCELED.

Problem Description:

Sometimes you receive below error on statement or warehouse timeout for a query and when you check the parameter **STATEMENT_TIMEOUT_IN_SECONDS** for the account, you see the value is set above **xxx** seconds and the question arise that why this error has been received even when the parameter is set to a higher value.

Statement reached its statement or warehouse timeout of xxx second(s) and was canceled.

Causes

This happens because you have this parameter STATEMENT_TIMEOUT_IN_SECONDS set at the warehouse level with the xxx seconds.
If the warehouse level number is less for the parameter STATEMENT_TIMEOUT_IN_SECONDS, it takes precedence over account and therefore the query times out.

Solution

You can identify the parameter details and it's value by running the below statements and make the changes to the parameter STATEMENT_TIMEOUT_IN_SECONDS to high value to avoid this issue :

show parameters like '%STATEMENT_TIMEOUT_IN_SECONDS%' for warehouse <warehousename>;

show parameters like '%STATEMENT_TIMEOUT_IN_SECONDS%';

To make the changes, you can alter this parameter for your warehouse or account, whichever value is less:

alter warehouse <warehousename> set STATEMENT_TIMEOUT_IN_SECONDS=172800;

Applies To

This issue can also be seen for such queries which don't even need a warehouse to run like CLONE, CREATE etc. and get's timed out by the value set for the parameter STATEMENT_TIMEOUT_IN_SECONDS at the warehouse level. This happens because you have warehouse in your current session. Even though it doesn't take part in query execution, it's parameter STATEMENT_TIMEOUT_IN_SECONDS will take effect if it is set to a lesser number.

To resolve such issues, you need to make sure, your session doesn't have any warehouse available.

QUALIFY

In a SELECT statement, the QUALIFY clause filters the results of window functions.

QUALIFY does with window functions what HAVING does with aggregate functions and GROUP BY clauses.

In the execution order of a query, QUALIFY is therefore evaluated after window functions are computed. Typically, a SELECT statement's clauses are evaluated in the order shown below:

1. From
2. Where

3. Group by
4. Having
5. Window
6. QUALIFY
7. Distinct
8. Order by
9. Limit

Examples

The QUALIFY clause simplifies queries that require filtering on the result of window functions. Without QUALIFY, filtering requires nesting. The example below uses the ROW_NUMBER() function to return only the first row in each partition.

Create and load a table:

```
create table qt (i integer, p char(1), o integer);

insert into qt (i, p, o) values

(1, 'A', 1),

(2, 'A', 2),

(3, 'B', 1),

(4, 'B', 2);
```

This query uses nesting rather than QUALIFY:

```
select *

from (

select i, p, o,

row_number() over (partition by p order by o) as row_num

from qt

)

where row_num = 1

;
```

```
+—+—+—+———+

| I | P | O | ROW_NUM |
```

|—+—+—+———|

| 1 | A | 1 | 1 |

| 3 | B | 1 | 1 |

+—+—+—+———+

This query uses QUALIFY:

select i, p, o

from qt

qualify row_number() over (partition by p order by o) = 1

;

+—+—+—+

| I | P | O |

|—+—+—|

| 1 | A | 1 |

| 3 | B | 1 |

+—+—+—+

14) **What is CTE (Common Table Expressions) or** or subquery factoring clause, or with clause

https://dwgeek.com/snowflake-with-clause-syntax-usage-and-examples.html/

WITH clause is called as CTE .it is same as oracle with clause

A CTE (common table expression) is a named subquery defined in a WITH clause. You can think of the CTE as a temporary view for use in the statement that defines the CTE. The CTE defines the temporary view's name, an optional list of column names, and a query expression (i.e. a SELECT statement). The result of the query expression is effectively a table. Each column of that table corresponds to a column in the (optional) list of column names.

WITH subQ1(SchoolID) AS (SELECT SchoolID FROM Roster)
SELECT * FROM subQ1;

The WITH clause are commonly referred to as a **common table expression(s)**.

What is Snowflake WITH Clause?

WITH clause is an optional query construct precedes the SELECT statement within your query.

In many data warehouse applications, there will always be a requirement to reuse the results of some query construct across multiple location. For example, you may use piece of the code in

many locations of your query. You may create the *common table expression (CTE)* for that query construct and use CTE across all other locations.

You can also use the WITH clauses to improve the complex sub-queries and improve overall Snowflake performance.

Type of WITH clause in Snowflake?

Snowflake cloud data warehouse supports two type of WITH clauses.

- **Non-Recursive** WITH Clause
  - These are simple WITH clauses
- **Recursive** WITH Clause
  - *It can refer to itself and usually used to resolve hierarchy*

Where you can use Snowflake WITH Clause?

The main advantage of WITH clause is, *you can use it wherever SELECT clause is acceptable in the SQL script or query*.

For instance, you can use it in:

- INSERT INTO … SELECT
- UPDATE – Within a WHERE clause of subquery
- CREATE VIEW
- DELETE
- CTAS (create table as select)
- SELECT

Snowflake WITH Clause Syntax

Below is the syntax for WITH clause:

[<WITH clause>] < <with list element> [{ <comma> <with list element> }…]>

<Select Statement>;

Snowflake WITH Clause Examples

Following example demonstrates the WITH clause usage.

WITH Clause usage along with SELECT statement

```
with s_patient_CTE as (select * from s_patient where ID = 100001)

select * from s_patient_CTE
```

```
order by 1;
```

Now, let us check common table expression usage with few examples.

WITH Clause in an INSERT Statement

Following Snowflake example demonstrate CTE in an INSERT statement.

```
INSERT INTO sample_table1
WITH CTE AS
(SELECT 1, 2 FROM dual)
SELECT * from CTE;
```

WITH Clause in an UPDATE Statement Example

You can use Snowflake CTE in an UPDATE statement WHERE sub query.

For example:

```
UPDATE sample_table1
SET col1 = 3
WHERE col1 = (WITH sample_cte
AS (SELECT 1
FROM dual)
SELECT *
FROM sample_cte);
```

WITH clause in DELETE statement

You can use the Snowflake CTE in DELETE statement WHERE sub query.

For example:

```
DELETE FROM sample_table1
WHERE col1 IN (WITH sample_cte
AS (SELECT 3
FROM dual)
SELECT *
FROM sample_cte);
```

WITH clause in CREATE TABLE AS Statement

You can use WITH clause in CREATE TABLE AS statement.

For example,

```
CREATE TABLE sample_table2

AS

WITH CTE AS

(

SELECT current_date as col1

)

SELECT col1

FROM CTE;
```

Recursive WITH Clause Example

The *recursive WITH clause in Snowflake* is something that refers to itself. These types of recursive queries are used to resolve hierarchical solutions.

```
WITH RECURSIVE rec_cte (X, Y) AS

(

SELECT X, Y FROM table1

UNION ALL

SELECT X, Y

FROM table1 JOIN rec_cte_name ON <join_condition>

)

SELECT … FROM …
```

Snowflake WITH Clause Restrictions

Below are some of WITH clause restrictions:

1. You cannot specify another WITH clause inside a WITH clause sub query.
2. You cannot make forward references to tables defined by WITH clause sub queries
3. CTEs are not currently fully supported in *DDL* operations. *You can use with CREATE TABLE AS …*
4. For recursive CTE, *column list* is mandatory.

5. *Keyword recursive* is used only once.

→Diff B/w oracle and snowflake data base objects

Oracle will subtract one date from another date (for example, SELECT date '2018-12-31' – date '2018-12-01' from dual;). In Snowflake, to do this same type of date comparison, use the DATEDIFF function (for example, SELECT DATEDIFF(day, '2018-12-01', '2018-12-31')).

Oracle can also subtract integers from dates, for example, SELECT hire_date, (hire_date-1) FROM employees. This syntax is not supported in Snowflake. The DATEADD and TIMESTAMP add functions are used to add or subtract units from dates or timestamps in Snowflake

→how to store filename on each record when copying files' data into snowflake?

Ans: user these metadata columns metadata$filename, metadata$file_row_number

FACT LESS FACT TABLE

IF WE CHNAGE THE upper to lower in query whther it will use result cache ?

Ans: No. Because snowflake will check the executed queries in cloud service layer using hash. If you change the case the hash of query will change. Hence snowflake will think this as new query.

IF WE ADD SPACE IN THE QUERY then it will use result cache?

Ans : NO

Bcz result cache will be used when The new query matches the previously-executed query (with an exception for spaces).

TYPES OF TABLES → DIFF B/W TEMPORARY,TRANSIENT, PERMANNET TABLES

Table Definition

Before loading data into Snowflake, you must define a structure. Since Snowflake is ANSI SQL-compliant, you can execute DDL such as CREATE TABLE AS from within the web interface (in

the Worksheet tab) or through the command line interface (snowsql), or you can just use the Database tab in the web interface. There are two key elements to defining a table:

- Table Type
- Table Structure

Table Type

Snowflake supports three table types: *permanent*, *transient*, and *temporary*. The differences between the three are twofold: how long the table persists and how much Time Travel and Fail-safe protection is provided (which contributes to storage cost).

- Temporary tables are just that — they exist for the length of your session. They are typically used as a temporary place to store and act on data while programming. They do not provide any Time Travel or Fail-safe protection so they incur no additional storage footprint beyond the data in the table, which is purged from Snowflake when the session ends.
- Transient tables are often used as staging areas for data that could easily be reloaded if lost or corrupted. Like temporary tables, they provide no Fail-safe protection, but they persist for a day upon deleting, dropping or truncating the table, allowing for Time Travel (**https://docs.snowflake.net/manuals/user-guide/data-time-travel.html**). Transient tables persist data exactly like permanent tables, except for the Fail-safe aspect.
- Permanent tables are the default type created by the CREATE TABLE command. By default, they provide 1 day of Time Travel and have the added benefit of seven days of Fail-safe protection (**https://docs.snowflake.net/manuals/user-guide/data-cdp-storage-costs.html**). And, with our Enterprise offering, permanent tables can be configured for up to 90 days of Time Travel.

Comparison of Table Types

The following table summarizes the differences between the three table types, particularly with regard to their impact on Time Travel and Fail-safe:

Type	Persistence	Cloning (source type => target type)	Time Travel Retention Period (Days)	Fail-safe Period (Days)
Temporary	Remainder of session	Temporary => Temporary Temporary => Transient	0 or 1 (default is 1)	0
Transient	Until explicitly	Transient => Temporary Transient	0 or 1 (default is	0

Type	Persistence	Cloning (source type => target type)	Time Travel Retention Period (Days)	Fail-safe Period (Days)
	dropped	=> Transient	1)	
Permanent (Standard Edition)	Until explicitly dropped	Permanent => Temporary Permanent => Transient Permanent => Permanent	0 or 1 (default is 1)	7
Permanent (Enterprise Edition and higher)	Until explicitly dropped	Permanent => Temporary Permanent => Transient Permanent => Permanent	0 to 90 (default is configurable)	7

Table Structure

The structure you chose for your table depends on the type of data you are loading:

- For structured data (CSV/TSV), we highly recommend pre-defining each column/field in your target table with proper data type definition before import. Snowflake provides a File Format object that can help with date conversion, column and row separators, null value definition, etc.
- For semi-structured data (JSON, Avro, XML), define a table with a single column using VARIANT as the data type.

When you load semi-structured data into the VARIANT column, Snowflake automatically shreds each object into a sub-column optimized for query processing and accessible via SQL using dot notation; e.g.:

SELECT COUNT(fieldname:objectname) FROM table;

if we use where cluase extra whether it will use same result cache

https://community.snowflake.com/s/article/Caching-in-Snowflake-Data-Warehouse –imp

https://visualbi.com/blogs/snowflake/caching-techniques-snowflake/

https://snowflakecommunity.force.com/s/article/Understanding-Result-Caching

https://sonra.io/2018/03/05/deep-dive-on-caching-in-snowflake/

https://blog.ippon.tech/innovative-snowflake-features-caching/

https://www.analytics.today/blog/caching-in-snowflake-data-warehouse

max no clusters in vartual warehouse— more than 10 possible? Max is 10

→when virtual warehouse is spinup to multiple virtual then is RAM also added for these warehouses?

Refer: https://medium.com/@a.kaushik5587/what-makes-snowflake-so-powerful-its-the-hybrid-of-shared-disk-and-shared-nothing-architecture-5b4fa8f039fa

Ans: in snowflake **virtual warehouse uses shared nothing architecture. So it will add RAM also**

Database systems with a shared nothing architecture are made up of cluster of independent machines or nodes connected with each other over a high-speed network. Each machine or node has its own RAM and disk. All the database tables are spread over multiple machines in the cluster using horizontal data partitioning so that the processors can run independently of others. This means that each node stores only a portion of whole data in its disk.

Whenever a shared nothing cluster receives a client SQL request, the query is sent to every node or machine of the cluster and they in turn execute it against the portion of data they store. This parallel execution of query on all machines or nodes of the cluster results in faster data processing.

when virtual warehouse is spinup to multiple virtual then is storage also added for these warehouses **? ans :it wont add storage bcz in snowflake warehouse is for compute purpouse and it is de coupled with storage**

Scaling policy: Standard /Economy

→If source has multiple commas in csv then how to handle that

Ans: we have to create file format with option as Filed optionally enclosed by

15) **Snowchange a Database Change Management Tool**

It is used for devops to create CI/CD pipeline

you can use Github or Gitlap ci/cd pipeline to deploy code to production →pradeep hc

Refer link for medium articles

https://medium.com/hashmapinc/using-dbt-to-execute-elt-pipelines-in-snowflake-dbe76d5beed5 →dbt

https://medium.com/hashmapinc/ci-cd-on-snowflake-using-sqitch-and-jenkins-245c6dc4c205 →jenkin n sqitch

https://dzone.com/articles/learn-how-to-setup-a-cicd-pipeline-from-scratch –jenkin

https://github.com/jamesweakley/snowchange/blob/master/README.md

https://medium.com/@jeremiahhansen

https://medium.com/@jeremiahhansen/a-new-approach-to-database-change-management-with-snowflake-8e3f0fee281

https://www.youtube.com/watch?v=vGqRyMlvYjo —- snowchange demo

FOR bayer (CTS) project will use liquibase for to migrate code dev to stage to prod

FLYWAY ALSO USED it is very easy

16) STAR SCHEMA n snowflake schema difference

17) **Diff b/w tuples n list in python**

LIST vs TUPLES

LIST

TUPLES

Lists are mutable i.e they can be edited.

Tuples are immutable (tuples are lists which can't be edited).

Lists are slower than tuples.

Tuples are faster than list.

Syntax: list_1 = [10, 'Chelsea', 20]

Syntax: tup_1 = (10, 'Chelsea' , 20)

The builtin list type should not be used as a dictionary key.

Note that since tuples are immutable, they do not run into the troubles of lists - they can be hashed by their contents without worries about modification. Thus, in Python, they provide a valid __hash__ method, and are thus usable as dictionary keys.

18) **SLOWLY CHANGING Dimesnios (SCD) types**

Slowly Changing Dimensions

What is a Slowly Changing Dimension?

A Slowly Changing Dimension (SCD) is a dimension that stores and manages both current and historical data over time in a data warehouse. It is considered and implemented as one of the most critical ETL tasks in tracking the history of dimension records.

There are three types of SCDs and you can use Warehouse Builder to define, deploy, and load all three types of SCDs.

What are the three types of SCDs?

The three types of SCDs are:

Type 1 SCDs - Overwriting

In a Type 1 SCD the new data overwrites the existing data. Thus the existing data is lost as it is not stored anywhere else. This is the default type of dimension you create. You do not need to specify any additional information to create a Type 1 SCD.

Type 2 SCDs - Creating another dimension record

A Type 2 SCD retains the full history of values. When the value of a chosen attribute changes, the current record is closed. A new record is created with the changed data values and this new record becomes the current record. Each record contains the effective time and expiration time to identify the time period between which the record was active.

Type 3 SCDs - Creating a current value field

A Type 3 SCD stores two versions of values for certain selected level attributes. Each record stores the previous value and the current value of the selected attribute. When the value of any of the selected attributes changes, the current value is stored as the old value and the new value becomes the current value.

Difference Between Slowly Changing Dimensions and Change Data Capture

https://streamsets.com/blog/slowly-changing-dimensions-vs-change-data-capture/

While some might observe that the difference between slowly changing dimensions (SCD) And Change Data Capture (CDC) might be subtle, there is in fact a technical difference between the two processes.

Both processes detect changes in a source database and deliver the changed data to a target database. The difference between the two is almost entirely about what happens in the target database to the data.

What Are Slowly Changing Dimensions (SCD)?

There are actually six types of SCD with the most common being Type 1, Type 2 and Type 3. SCD types 4, 5, and 6 are inefficient and overly complicated for maintaining a history of all changes or overwriting old data, which are the two essential purposes of Slowly Changing Dimensions.

In Type 1, any new data that is ingested overwrites existing data. In Type 2, new data are inserted as new records and the data that would have been overwritten are flagged as inactive or closed with effective time and expiration time assigned to the change to maintain a history. In Type 3, one column is designated for storing previous data (i.e. the data that would've been overwritten in Type 1).

In short, Type 1 stores no historical data, Type 2 stores all historical data, and Type 3 stores limited historical data.

For a modern data integration tool to be considered truly modern support for SCD is key. StreamSets supports both type 1 and type 2 Slowly Changing Dimensions. Check out a few SCD patterns to see examples of how they can help you manage customer records.

What Is Change Data Capture?

CDC is a method of detecting and extracting new or updated records in a source and loading just this new information into your destination. Very often, the alternative to CDC is a full load from one table to another resulting in a very costly and time consuming operation. By sipping into your target database just the delta or changed data you get a much more streamlined process.

There are actually three different ways of performing CDC: log-based, query-based, and trigger based. Differences that are explored elsewhere in our blogs in detail. Essentially, however, log-based CDC updates a log for every INSERT, UPDATE or DELETE and reads that information when it is time to insert into the target database, while trigger CDC kicks off a trigger every operation with the same result. Log-based CDC is considered to be more efficient than a trigger CDC method. Query-based CDC involves using queries to find differences between datasets and can be untenable with larger datasets as it can require much more resources to perform this comparison.

CDC looks the most like Type 1 Slowly Changing Dimensions as overwriting new data as it appears. It is most useful to use when you're not worried about maintaining a history of all the changes to your database. Like most other modern data integration systems, StreamSets supports log-based CDC.

Choosing SCD over CDC or vice versa should be based on business process, not technical limitations. SCD is ideal for organizations that must maintain a record of all changes to the data flowing through their systems. And CDC is ideal if your business process requires only that the changed data arrive in your target.

When, Why, and How to Use Change Data Capture (CDC)

https://streamsets.com/blog/change-data-capture-uses/

Change data capture (CDC) is a software design pattern that identifies and tracks data changes in a source system. Outside of full replication, CDC is the only way to ensure database environments, including data warehouses, are synced across hybrid environments.

With the help of CDC, subsequent stages of the data pipeline are only sourcing, transforming, and publishing data that has been altered rather than performing resource-intensive operations on the entire source system. This leads to lower latency, more efficient throughput, and increased data durability.

Change Data Capture Systems and Mechanisms

Today, there are three primary ways to implement change data capture: **Log-based, Query-based,** and **Trigger-based**.

1. **Log-based** – Log-based CDC is one of the most efficient CDC strategies. Every new database transaction is recorded in a log file in this approach. Moving forward, the polling system can source information from the log file without incurring and resource hit on the original database.

2. **Query-based** – The database is queried to pick up changes with query-based CDC. This strategy incurs more resource toll than log-based CDC as it polls the source database and requires the database to be configured to preserve metadata like a timestamp for querying.

3. **Trigger-based** – With Trigger-based CDC, the source database system is configured to trigger a notification when data is written to or altered within the source database. This process relies on auditing metadata within the database, such as a timestamp or other indicators that a data entry has changed. With that said, keep in mind that database triggers do incur a more substantial performance impact on the data source. It requires multiple writes each time a database change is identified, and an accompanying trigger is initiated.

Aside from Log-based, Query-based, and Trigger-based CDC, there are also two primary ways that data is extracted from the source target, either via a **Push** operation or a **Pull** operation.

Push-based systems push changes to a target, whereas pull systems poll the source and pull changed data to the next stage of the data pipeline.

Each system has benefits and setbacks that an organization should consider.

- **Push** – The push system runs into challenges when the next stage of the data pipeline is offline or not listening. Here, pushes may be missed, leading to lost data and inefficient data pipelines.

- **Pull** – Pull-based systems are known to be more straightforward in their setup. However, the pulling system has to update the source with extracted data leading to operations overhead.

Push versus pull[edit]

- **Push**: the source process creates a snapshot of changes within its own process and delivers rows downstream. The downstream process uses the snapshot, creates its own subset and delivers them to the next process.
- **Pull**: the target that is immediately downstream from the source, prepares a request for data from the source. The downstream target delivers the snapshot to the next target, as in the push model.

.17) tmap and tjoin differences in Talend

tMap is frequently used component for joins and lookup purpose, it is also use for verity of operations and transformations, whereas tJoin is used for join and lookups only.

tMap

tJoin

It accepts more than one input one is main and rests of the lookups.

It accepts only two inputs and only one is main and other one is lookup.

We can create more than one output

It has two default outputs one is “Main” and another one is ” Inner join reject”

tMap has “inner join” and ” left outer join” joining model

tJoin offers only “inner join</mark>”</td> </tr> <tr class="even"> <td> <mark>tMap offers three match model</mark> <ol type="1"> <mark>Unique Match</mark>   <mark>First Match</mark>   <mark>All Matches</mark> </td> <td>tJoin defaulted with Unique match</td> </tr> <tr class="odd"> <td>tMap allows to store data on file option for lookup data processing</td> <td>tJoin doesnt offer this feature

In tMap you can filter data using filter expression

tJoin doesnt offer this feature</mark></td> </tr> <tr class="odd"> <td><mark>You can write transformation using expression builder at each column level</mark></td> <td>tJoin doesnt offer this feature

18 What is the difference between Built-In and Repository in talend

- **Built-in**: all information is stored locally in the Job. You can enter and edit all information manually.
- **Repository**: all information is stored in the repository.

You can import read-only information into the Job from the repository. If you want to modify the information, you must take one of the following actions:

- Convert the information from **Repository** to **Built-in** and then edit the built-in information.
- Modify the information in the **Repository**. Once you have made the changes, you are prompted to update the changes into the Job.

In which case to use **Built-In** and **Repository**:

- Use **Built-In** for information that you only use once or very rarely.
- Use **Repository** for information that you want to use repeatedly in multiple components or Jobs, such as a database connection.

19) is it possible to create multiple not null constraints on single table

A Snowflake table can have multiple NOT NULL columns.

Snowflake supports defining and maintaining constraints, but does not enforce them, except for NOT NULL constraints, which are always enforced.

similar to most of the MPP databases, Snowflake cloud database allows you to define constraints. The Snowflake database does not enforce constraints like primary key, foreign key and unique key. But, it does enforce the NOT NULL constraint. In this article, we will check **Snowflake NOT NULL constraint**, its syntax and usage.

Snowflake NOT NULL Constraint

Constraints other than NOT NULL are created as disabled. Snowflake enforces only NOT NULL. *You can create NOT NULL constraint while creating tables* in the cloud *database.*

A Snowflake table can have multiple NOT NULL columns.

Snowflake NOT NULL Constraint Syntax

There are many methods that you can use to add NOT NULL on Snowflake table.

- **Column level NOT NULL** – Add NOT NULL constraint during table creation.
- **Alter Table to Add NOT NULL** – Use Alter table command to add NOT NULL constraint.

*Column level **NOT NULL***

You can add the NOT NULL to the Snowflake table DDL along with column the data type.

For example, consider below table DDL with a column *ID* defined as NOT NULL.

```
CREATE TABLE nn_demo_table
(
id INT NOT NULL,
```

NAME VARCHAR(10),

address VARCHAR(100)

);

Now, the Snowflake database will allow only non-null values in the *ID* column. You will end up getting an error if the value is NULL.

Alter Table to Add NOT NULL ***Constraint***

You can also add the NOT NULL constraint to the existing table. The table must be empty in order to SET NOT NULL constraint.

For example, consider following ALTER statement to add NOT NULL with default value.

ALTER TABLE nn_demo_table MODIFY COLUMN ID SET NOT NULL;

+————————————————-+

| status |

|————————————————-|

| Statement executed successfully. |

+————————————————-+

Test Add NOT NULL **Constraint**

Now, let us try to insert the NULL values into the not null column and check the error.

>INSERT INTO nn_demo_table values (1,'a','abc');

+————————————-+

| number of rows inserted |

|————————————-|

| 1 |

+————————————-+

>INSERT INTO nn_demo_table values (null,'a','abc');

100072 (22000): NULL result in a non-nullable column

As you can see in the above example, you can only insert non-null values. Therefore, *Snowflake cloud data warehouse enforces the NOT-NULL constraint.*

20) handling json data with STRIP_OUTER_ARRAY

COPY INTO DEMO_DB.DEMO_SCHEMA.DEMO_INPUT_TABLE

FROM @DEMO_DB.DEMO_SCHEMA.DEMO_STAGE

FILE_FORMAT = (

TYPE = 'JSON'

STRIP_OUTER_ARRAY = TRUE

)

;

We have used an option called **STRIP_OUTER_ARRAY** for this load. This removes the outer set of square brackets **[ ]** when loading the data, separating the initial array into multiple lines. If we did not strip the outer array, our entire dataset would be loaded into a single row in the destination table. With our low data volumes, this is not a problem; however, it would be an issue in larger datasets. As we have stripped the outer array, the data is loaded into two rows in Snowflake. The file format and **STRIP_OUTER_ARRAY** option are explained in more detail in the previous blog post.

Data Size Limitations for Semi-Structured Data

The VARIANT data type has a 16 MB (compressed) size limit on the individual rows.

Often JSON and Avro are the most commonly used data formats. Both JSON and Avro are a concatenation of many documents. The source software that provides JSON or Avro output will provide the output in the form of a single huge array having multiple records. Both line breaks and commas are supported for document separation.

For efficiency enhancement, while executing the COPY INTO <table> command it is recommended to enable the STRIP_OUTER_ARRAY file format

option. This will load the records into separate table rows by removing the outer array structure. Below is an example:

COPY INTO <table_name>

FROM @~/<file_name>.json

file_format = (type = 'JSON' strip_outer_array = true);

BcFoward

What is column security and DATA MASKING

Snowflake achieve column level security using data masking

We can achive column level sucrity by using secure view also

https://www.snowflake.com/blog/column-level-security-in-snowflake/

https://support.snowflake.net/s/article/How-to-Secure-PII-Data-with-Data-Masking

https://blog.satoricyber.com/snowflake-data-masking-static-vs-dynamic

Snowflake Dynamic Data Masking

For starters, let's approach this with a relatively new way to mask data in Snowflake, which is the Dynamic Data Masking feature (available for the Enterprise plan). Dynamic Data Masking allows you to set data masking policies, and apply them on certain columns.

When setting dynamic data masking in Snowflake, you are defining masking policies, which may give different results for columns generally based on the user's role (in most cases). For example, a policy can be:

CREATE OR REPLACE MASKING POLICY phone_masking **AS** (val string)

RETURNS string ->

CASE

WHEN CURRENT_ROLE() **IN** ('ADMIN_TEAM', 'ACCOUNTING_TEAM') **THEN** val

ELSE '[REDACTED]'

END;

From here, you can apply the dynamic masking policies on any column:

ALTER TABLE customers **MODIFY COLUMN** home_phone **SET** MASKING POLICY phone_masking;

ALTER TABLE customers **MODIFY COLUMN** work_phone **SET** MASKING POLICY phone_masking;

For some more support, think of dynamic masking as an abstraction of Snowflake Secure Views, which creates a more reusable way to apply policies. This is all an effort that makes them easier to manage and scale.

Snowflake Data Masking Using Views

Now, let's say that you don't have an enterprise Snowflake account for some reason or another. If this is the case for you, or maybe you have other logic you want to include in the same abstraction layer, you still have options here. For instance, you can write your own custom dynamic data masking logic within views. As an example, let's say that we have the following table:

CREATE TABLE customers (**id int**, name text);

INSERT INTO customers **VALUES** (1, 'Ben'), (2, 'Karl');

If you want to apply dynamic data masking that will give your ACCOUNTING team full read access, your ANALYST team hashed data for statistical purposes, and others redacted data, you can create a view abstract, such as:

CREATE SECURE **VIEW** v_customers **AS**

SELECT id, (

CASE

WHEN CURRENT_ROLE() **IN** ('ACCOUNTING') **THEN** name

WHEN CURRENT_ROLE() **IN** ('ANALYST') **THEN** sha2(name)

ELSE '[REDACTED]'

END

) **AS** name **FROM** customers;

By revoking access from the underlying asset (customers) and granting access to the view (v_customers), users will now have data masking per their roles and can only retrieve data based on the commands in place.

USE ROLE ACCOUNTING;

SELECT * **FROM** v_customers;

// returns Ben, Karl

USE ROLE ANALYST;

SELECT * **FROM** v_customers;

// returns hashed values

USE ROLE OTHER;

SELECT * **FROM** v_customers;

// returns redacted values

Types of views in snowflake:

When to create snowflake views

Materialized views are particularly useful when:

- Query results contain a small number of rows and/or columns relative to the base table (the table on which the view is defined).
- Query results contain results that require significant processing, including:
  - Analysis of semi-structured data.
  - Aggregates that take a long time to calculate.
- The query is on an external table (i.e. data sets stored in files in an external stage), which might have slower performance compared to querying native database tables.
- The view's base table does not change frequently.

Deciding When to Create a Materialized View or a Regular View

In general, when deciding whether to create a materialized view or a regular view, use the following criteria:

- Create a materialized view when ***all*** of the following are true:
  - The query results from the view don't change often. This almost always means that the underlying/base table for the view doesn't change often, or at least that the subset of base table rows used in the materialized view don't change often.
  - The results of the view are used often (typically significantly more often than the query results change).
  - The query consumes a lot of resources. Typically, this means that the query consumes a lot of processing time or credits, but it could also mean that the query consumes a lot of storage space for intermediate results.
- Create a regular view when ***any*** of the following are true:
  - The results of the view change often.
  - The results are not used often (relative to the rate at which the results change).
  - The query is not resource intensive so it is not costly to re-run it.

Advantages of Materialized Views

Snowflake's implementation of materialized views provides a number of unique characteristics:

- Materialized views can improve the performance of queries that use the same subquery results repeatedly.
- Materialized views are automatically and transparently maintained by Snowflake. A background service updates the materialized view after changes are made to the base table. This is more efficient and less error-prone than manually maintaining the equivalent of a materialized view at the application level.
- Data accessed through materialized views is always current, regardless of the amount of DML that has been performed on the base table. If a query is run before the materialized view is up-to-date, Snowflake either updates the materialized view or uses the up-to-date portions of the materialized view and retrieves any required newer data from the base table.

Important

The automatic maintenance of materialized views consumes credits. For more details, see Maintenance Costs for Materialized Views (in this topic).

LIMITATIONS OF MATERIALIZED VIEWS

The following table shows key similarities and differences between tables, regular views, cached query results, and materialized views:

	Performance Benefits	Security Benefits	Simplifies Query Logic	Supports Clustering	Uses Storage	Uses Credits for Maintenance	Notes
Regular table				✔	✔		
Regular view		✔	✔				
Cached	✔						Used only if data has not changed and if query only uses deterministic functions (e.g. not CURRENT_DATE).

	Performance Benefits	Security Benefits	Simplifies Query Logic	Supports Clustering	Uses Storage	Uses Credits for Maintenance	Notes
query result							
Materialized view	✔	✔	✔	✔	✔	✔	Storage and maintenance requirements typically result in increased costs.
External table							Data is maintained outside Snowflake and, therefore, does not incur any storage charges within Snowflake.

NIFI etl tool

Access and grants to reports module

How we do migration of existing db

Difference b/w oracle db and snowflake

Security and roles in snowflake

TCS

Sonata

- **Retention period questions →CTS**

Note: retention period we can set at database level, schema level, table lavel.

Means in SAME database and same schema different table may have different retention periods

Changing the retention period for your account or individual objects changes the value for all lower-level objects that do not have a retention period explicitly set. For example:

- If you change the retention period at the account level, all databases, schemas, and tables that do not have an explicit retention period automatically inherit the new retention period.
- If you change the retention period at the schema level, all tables in the schema that do not have an explicit retention period inherit the new retention period.

  Keep this in mind when changing the retention period for your account or any objects in your account because the change may have Time Travel consequences that you did not anticipate or intend. In particular, we do ***not*** recommend changing the retention period to 0 at the account level

- **Query to get rows into comma separted values**

  Ans: list aggregate function

SELECT deptno, LISTAGG(ename, ‘,’) WITHIN GROUP (ORDER BY ename) AS employees

FROM emp

GROUP BY deptno;

DEPTNO EMPLOYEES

---

10 CLARK,KING,MILLER

20 ADAMS,FORD,JONES,SCOTT,SMITH

30 ALLEN,BLAKE,JAMES,MARTIN,TURNER,WARD

- Query to get running total

  SELECT month,val,

  SUM(val) OVER (ORDER BY val

  RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total

  from month_data;

- Adding the ORDER BY clause allows us to display a running total salary within a partition. In the example below, the default windowing clause is used, as well as being specified explicitly.
- SELECT empno,
- ename,
- deptno,
- sal,
- SUM(sal) OVER (PARTITION BY deptno ORDER BY sal) AS running_tot_sal_by_dept_1,
- SUM(sal) OVER (PARTITION BY deptno ORDER BY sal
- **RANGE BETWEEN** UNBOUNDED PRECEDING AND CURRENT ROW) AS running_tot_sal_by_dept_2
- FROM emp;
- EMPNO ENAME DEPTNO SAL RUNNING_TOT_SAL_BY_DEPT_1 RUNNING_TOT_SAL_BY_DEPT_2

- 7934 MILLER 10 1300 1300 1300
- 7782 CLARK 10 2450 3750 3750
- 7839 KING 10 5000 8750 8750
- 7369 SMITH 20 800 800 800
- 7876 ADAMS 20 1100 1900 1900
- 7566 JONES 20 2975 4875 4875
- 7788 SCOTT 20 3000 10875 10875
- 7902 FORD 20 3000 10875 10875
- 7900 JAMES 30 950 950 950
- 7654 MARTIN 30 1250 3450 3450
- 7521 WARD 30 1250 3450 3450
- 7844 TURNER 30 1500 4950 4950
- 7499 ALLEN 30 1600 6550 6550
- 7698 BLAKE 30 2850 9400 9400

- SQL>
- The RANGE BETWEEN windowing clause is a reporting range, so all rows of the same value are included, which makes the running totals look wrong, if that's not what you were expecting. If we switch to the ROWS BETWEEN windowing clause, you might get the result you were expecting. look at the comparison between the results of the first call using the default windowing clause, and the explicit windowing clause using ROWS BETWEEN below.
- SELECT empno,
- ename,
- deptno,
- sal,
- SUM(sal) OVER (PARTITION BY deptno ORDER BY sal) AS running_tot_sal_by_dept_1,
- SUM(sal) OVER (PARTITION BY deptno ORDER BY sal
- **ROWS BETWEEN** UNBOUNDED PRECEDING AND CURRENT ROW) AS row_running_tot_sal_by_dept_2
- FROM emp;
- EMPNO ENAME DEPTNO SAL RUNNING_TOT_SAL_BY_DEPT_1 ROW_RUNNING_TOT_SAL_BY_DEPT_2

- 7934 MILLER 10 1300 1300 1300
- 7782 CLARK 10 2450 3750 3750
- 7839 KING 10 5000 8750 8750
- 7369 SMITH 20 800 800 800
- 7876 ADAMS 20 1100 1900 1900
- 7566 JONES 20 2975 4875 4875
- 7788 SCOTT 20 3000 10875 7875
- 7902 FORD 20 3000 10875 10875
- 7900 JAMES 30 950 950 950
- 7654 MARTIN 30 1250 3450 2200
- 7521 WARD 30 1250 3450 3450

- 7844 TURNER 30 1500 4950 4950
- 7499 ALLEN 30 1600 6550 6550
- 7698 BLAKE 30 2850 9400 9400

Getting Cumulative Sum (Running Total) Using Analytical Functions

```
SELECT
   DEPTNO,
   ENAME,
   SAL,
   SUM(SAL) OVER (PARTITION BY DEPTNO ORDER BY SAL,ENAME) CUMDEPTTOT,
   SUM(SAL) OVER (PARTITION BY DEPTNO) DEPTTOTAL,
   SUM(SAL) OVER (ORDER BY DEPTNO, SAL) CUMTOT,
   SUM(SAL) OVER () TOTSAL
FROM
   SCOTT.EMP
ORDER BY
   DEPTNO,
   SAL
```

-→ techm

CITY table has below data. In below table org to dest and dest to org combinations are there .. need to display those records which has pair of org- dest and dest-org are exists and distance is same means find the the sample records highlighted in yellow and green

And need to display only one record for that comibation

ORG	DEST	DIST
MAA	DXB	300
DXB	MAA	300
DXB	DEL	200
LON	MAA	2000
MAA	FRK	3000
DEL	DXB	200

Query:

```
SELECT * FROM(
SELECT A.ORG,A.DEST,A.DIST,ROW_NUMBER() OVER (PARTITION BY DIST ORDER BY ORG) RN
FROM CITY A
WHERE EXISTS( SELECT 1 FROM
```

```
CITY B
WHERE A.ORG=B.DEST
AND A.DEST=B.ORG
)
) WHERE RN=1;
```

o/p:

** **

ORG	DEST	DIST	RN
DEL	DXB	200	1
DXB	MAA	300	1

Results from query 8

Scale up vs scale out → CTS

Scaling up is all about increasing the compute power of the existing warehouse node. This should assist long-running queries, queries that require a lot of bytes scanned and queries with storage spillage. This would be done by increasing the size property of the warehouse.

Scaling Out is the process of adding more clusters to an existing warehouse. This will assist when there are a large number of concurrent queries being executed in the same warehouse. Scaling Out will allow for those queued queries to be executed on the new provisioned cluster.

Scaling up →ware house size increasing like from Large to extra large. scaling up is used when we are running complex queries

Scaling out→ adding more clusters i.e multi cluster . multi clustering is use full when queries large no of queries are queued in same warehouse

Scaling up means vertical scaling

Scaling out means horizontal scaling

→Grants and roles for tables, schema, database → CTS

https://medium.com/hashmapinc/heres-your-day-1-and-2-checklist-for-snowflake-adoption-e0e7ff8f105a

https://copycoding.com/d/grant-access-to-database-objects-in-a-schema-to-a-role-in-snowflake

You can grant the USAGE access to Warehouse / Database / Schema

Grant usage on the database:

GRANT USAGE ON DATABASE <database> TO ROLE <role>;

Grant usage on the schema:

GRANT USAGE ON SCHEMA <database>.<schema> TO ROLE <role>;

Grant the ability to query an existing table:

GRANT SELECT ON TABLE <database>.<schema>.<table> TO ROLE <role>;

→how to split large file into multiple small files

If your source database does not allow you to export data files in smaller chunks, you can use a third-party utility to split large CSV files.

Linux or macOS

The split utility enables you to split a CSV file into multiple smaller files.

Syntax:

split [-a suffix_length] [-b byte_count[k|m]] [-l line_count] [-p pattern] [file [name]]

For more information, type man split in a terminal window.

Example:

split -l 100000 pagecounts-20151201.csv pages

This example splits a file named pagecounts-20151201.csv by line length. Suppose the large single file is 8 GB in size and contains 10 million lines. Split by 100,000, each of the 100 smaller files is 80 MB in size (10 million / 100,000 = 100). The split files are named pages<suffix>.

Windows

Windows does not include a native file split utility; however, Windows supports many third-party tools and scripts that can split large data files.

→what is file size suggested for Bulk load … between 100 to 250 MB

→what is file suggested for snow pipe

→Is it possible to use function in copy command like concat →yes

https://docs.snowflake.com/en/user-guide/data-load-transform.html

→Diff b/w economy and standard scale out methods

→what is metadata in snowflake

Imp points : snowflake uses Foundationdb for metadata.

In a data warehouse, metadata defines warehouse objects and functions as a directory to help locate data warehouse content. It is usually divided into three distinct types or sets: operational, technical, and business data.

THE SNOWFLAKE META STORE

THE SNOWFLAKE META STORE

Snowflake has used open source FoundationDB as its meta store since 2014 and has helped develop the open source, distributed, and transactional store ever since. With FoundationDB, Snowflake has an extremely reliable store as a key part of its architecture, allowing the cloud data warehouse to handle multiple versions of objects concurrently.

https://www.snowflake.com/blog/how-foundationdb-powers-snowflake-metadata-forward/

https://community.snowflake.com/s/article/Metadata-Archiving-with-Snowflake

While selecting a metadata store, **we prefer key-value** stores for the simplicity and flexibility they bring to schema evolution. Also, our cloud services expect the underlying store to be ACID compliant. FoundationDB fits these requirements perfectly. It supports triple replication of data for availability and has a change-in-value notification feature called “watch”.

Snowflake’s cloud services layer is composed of a collection of stateless services that manage virtual warehouses, query optimization, transactions and others, as shown in Fig. 1. To perform their tasks, these services rely on rich metadata stored in FoundationDB. For high availability, we not only triple replicate the metadata but also store it across multiple cloud availability zones. Sensitive metadata is encrypted, using our key management infrastucture.

To make it easy to add new metadata objects, we built an object-mapping layer on top of key-values. Schema definition, evolution and metadata versioning are done by this layer as well. User-visible objects, such as catalog definitions, users, sessions, access control, copy history and others all have metadata backing them. Every statement executed has a metadata entry, along with statistics of its execution. Transaction state and lock queues are also persisted in FoundationDB. In fact, lock queues are implemented using the watch feature mentioned earlier. A data manipulation statement is enqueued on a resource’s lock queue, and a FoundationDB watch notifies the statement when the statement reached the front of the resource’s queue. There is also user-invisible metadata such as data distribution information, servers and encryption keys.

For instance, **metadata enables the zero-copy clone feature,** which allows cloning tables, schemas and databases without having to replicate the data. Metadata for each table keeps track of the set of micro-partitions that belong to the table at each version.

A powerful feature such as Snowflake Data Sharing is also achieved by a metadata-only operation. The data share object is created containing references to the source and destination catalog objects and access control objects. Data sharing does not copy data from the provider, it exposes the data to the consumer via the data share object.

Cluster key CTS interview question

- **For one million unique records data if we add cluster key is performance will be improved. ANS: NO**
- **If above table is joined with any other table on clustering key then it will improve performace? Yes**
- If all records are unique you should not be adding any clustering key. As it will not add any benefit.
- If you cluster the table based on the joining key(assuming it's having less cardinality) then you can see performance improvement.

→ COPY COMMAND – CTS QUESTION

COPY command usage and limitations

Transforming Data During a Load COPY COMMAND

Related Topics

- Querying Data in Staged Files
- Script: Loading JSON Data into a Relational Table
- Script: Loading and Unloading Parquet Data

Snowflake supports transforming data while loading it into a table using the COPY INTO <table> command, dramatically simplifying your ETL pipeline for basic transformations. This feature helps you avoid the use of temporary tables to store pre-transformed data when reordering columns during a data load.

The COPY command supports:

- Column reordering, column omission, and casts using a SELECT statement. There is no requirement for your data files to have the same number and ordering of columns as your target table.
- The ENFORCE_LENGTH | TRUNCATECOLUMNS option, which can truncate text strings that exceed the target column length.

Note that COPY transformations do ***not*** support the FLATTEN function, or JOIN or GROUP BY (aggregate) syntax.

Filtering the results of a FROM clause using a WHERE clause is not supported.

The DISTINCT keyword in SELECT statements is not fully supported. Specifying the keyword can lead to inconsistent or unexpected ON_ERROR copy option behavior.

The VALIDATION_MODE parameter does not support COPY statements that transform data during a load.

Convert staged data into other data types during a data load. All conversion functions are supported.

For example, convert strings as binary values, decimals, or timestamps using the TO_BINARY, TO_DECIMAL , TO_NUMBER , TO_NUMERIC, and TO_TIMESTAMP / TO_TIMESTAMP_* functions, respectively.

- Copy command supports column reordering, column omission when issuing a select statement within a copy command.
- It can include sequence columns, timestamp() and other supported functions

  https://docs.snowflake.net/manuals/user-guide/data-load-transform.html
- Not supported: Joins, filters, and aggregations
- VALIDATION_MODE parameter not supported in transformations

Currently, these copy options support CSV data only.

>>Only named stages (internal or external) and user stages are supported for COPY transformations.

Snowflake currently supports the following subset of functions for COPY transformations:

- ARRAY_CONSTRUCT
- ARRAY_SIZE
- ASCII
- CASE
- CAST , ::
- CEIL
- CHECK_JSON
- CHECK_XML
- CHR , CHAR
- CONCAT , ||
- CONVERT_TIMEZONE
- ENDSWITH
- EQUAL_NULL

- FLOOR
- GET
- GET_PATH , :
- HEX_DECODE_STRING
- HEX_ENCODE
- IFF
- IFNULL
- ILIKE
- [NOT] IN
- IS_ARRAY
- IS_BOOLEAN
- IS_DECIMAL
- IS_INTEGER
- IS_NULL_VALUE
- IS_OBJECT
- IS_TIME
- IS_TIMESTAMP_*
- LENGTH, LEN
- LIKE
- LPAD
- LTRIM
- MD5 , MD5_HEX
- NULLIF
- NVL
- NVL2
- OBJECT_CONSTRUCT
- PARSE_IP

- PARSE_JSON
- PARSE_URL
- PARSE_XML
- RANDOM
- REGEXP_REPLACE
- REGEXP_SUBSTR
- REPLACE
- REVERSE
- RPAD
- RTRIM
- SPLIT
- SPLIT_PART
- STARTSWITH
- SUBSTR , SUBSTRING
- TO_ARRAY
- TO_BINARY
- TO_BOOLEAN
- TO_CHAR , TO_VARCHAR
- TO_DATE , DATE
- TO_DECIMAL , TO_NUMBER , TO_NUMERIC
- TO_DOUBLE
- TO_OBJECT
- TO_TIME , TIME
- TO_TIMESTAMP / TO_TIMESTAMP_*
- TO_VARIANT
- TRY_CAST
- TRY_HEX_DECODE_STRING

- TRY_TO_BINARY
- TRY_TO_BOOLEAN
- TRY_TO_DATE
- TRY_TO_DECIMAL, TRY_TO_NUMBER, TRY_TO_NUMERIC
- TRY_TO_DOUBLE
- TRY_TO_TIME
- TRY_TO_TIMESTAMP / TRY_TO_TIMESTAMP_*
- UNICODE
- UUID_STRING
- XMLGET

Note in particular the lack of support for the VALIDATE function.

Note that COPY transformations do ***not*** support the FLATTEN function, or JOIN or GROUP BY (aggregate) syntax.

The list of supported functions might expand over time.

The following categories of functions are also supported:

- Scalar SQL UDFs.

END COPY COMMAND##

→snowflake with aws lambda

https://medium.com/hashmapinc/quick-tips-for-using-snowflake-with-aws-lambda-9e88cbbba61e

https://snowflakecommunity.force.com/s/article/How-to-Use-Snowflake-with-AWS-Lambda

https://medium.com/hashmapinc/quick-tips-for-using-snowflake-with-aws-lambda-9e88cbbba61e

https://blog.redpillanalytics.com/snowflake-data-warehouse-and-aws-lambda-8f6d8345f326 –good

→ STITCHDATA

Data loading from aws s3 to snowflake using stitch data

Source –aws s3

Destination –snowflake

https://www.youtube.com/watch?v=P0nLOpUCec8&t=123s

→**what is use of information schema in snowflake**

https://www.snowflake.com/blog/using-snowflake-information-schema/

→

TRUNCATE TABLE

Removes all rows from a table but leaves the table intact (including all privileges and constraints on the table). Also deletes the load metadata for the table, which allows the same files to be loaded into the table again after the command completes.

Note that this is ***different*** from DROP TABLE, which removes the table from the system but retains a version of the table (along with its load history) so that they can be recovered.

The Snowflake Information Schema (aka "Data Dictionary") consists of a set of system-defined views and table functions that provide extensive metadata information about the objects created in your account. The Snowflake Information Schema is based on the SQL-92 ANSI Information Schema, but with the addition of views and functions that are specific to Snowflake.

Each database created in your account automatically includes a built-in, read-only schema named INFORMATION_SCHEMA. The schema contains the following objects:

- Views for all the objects contained in the database, as well as views for account-level objects (i.e. non-database objects such as roles, warehouses, and databases)
- Table functions for historical and usage data across your account.

There are 18 views in the Information Schema that you can query directly. You can see the full list in the documentation here.

It is important to note that, for every database in Snowflake, there is a separate Information Schema so that queries only return data about your current database. Additionally, when writing the SQL, the view names in the Info Schema must be fully-qualified, particularly with 'information_schema' as you will see in the examples.

How to use information schema

A very simple place to start is to list the tables and views in one of your database schemas:

```
SELECT table_name, table_type

FROM kent_db.information_schema.tables
```

WHERE table_schema = 'PUBLIC'

ORDER BY 1;

Note that this SQL example (and all the examples in this post) specifies a particular schema (i.e., PUBLIC). If you have multiple schemas in your database, it would be wise to include a schema specification in the predicate whenever possible (unless you really do want to see everything in the database). Also keep in mind that the values in the information schema views are usually strings and are case sensitive so be sure to enclose them in single quotes when referencing in the predicate clause.

Dynamically Generate SQL using information schema

Dynamic SQL is a method for using data from the information schema to generate SQL statements. For example, suppose you need to clean up a database and drop most of the tables so you can regression test the CREATE script. There are many ways you can do this. If you want to use a SQL script to do it, you could write the script by hand, which is fine if you only have a few tables, but it would be better to generate the script.

Using the Information Schema in Snowflake, you can do something like this:

SELECT 'drop table'||table_name||' cascade;'

FROM kent_db.information_schema.tables tables

WHERE table_schema = 'PUBLIC'

ORDER BY 1;

The output should be a set of SQL commands that you can then execute. And as the schema evolves and more tables are added, this script will pick up the new tables the next time you run it so you don't even have to remember to edit it (hence the "dynamic" part of "dynamic SQL").

→SNOWSQL

In user prifle we will have

C:\snowsql/config file here we can give credentials and connection name.

In below screen connection name is trainingdb

while connecting command window we have to mention connection name

OR WE CAN GIVE as below

C:>snowsql -a lv74318.ap-south-1 -u VIYAAN -d demo_db -s public

SnowSQL is the next-generation command line client for connecting to Snowflake to execute SQL queries and perform all DDL and DML operations, including loading data into and unloading data out of database tables.

SnowSQL is an example of an application developed using the Snowflake Connector for Python;

Using SnowSQL

There are a few capabilities we would like to highlight in this blog.

SnowSQL Commands

First, SnowSQL offers a wide range of commands a user can make use of. As a general rule, all SnowSQL commands start with a bang character '!'. For a complete list of currently supported commands, please see our documentation here.

Auto-Complete and Syntax Highlighting

Secondly, with our context-sensitive auto-complete feature, SnowSQL users are released of cumbersome and error-prone typing of long object names Instead they can complete SQL keywords and functions once typing the first three letters. By leveraging auto-complete, SnowSQL users can become increasingly more productive and quickly explore data in Snowflake. Furthermore, SQL statements are highlighted in different colors resulting in a better readability for SnowSQL users when interacting with a terminal

Auto-Upgrade

Thinking as a service, the auto-update framework enables users to always stay up-to-date with both – Snowflake's and SnowSQL's latest features to streamline end user experience. No more additional downloads or tedious re-installation are needed. The upgrade is transparent for the end user and takes place as a background process when you start SnowSQL. Next time a user runs SnowSQL, the new version will be automatically picked up while their workflows remain unaffected and will not be interrupted during the upgrade.

Secure Connection and Encryption

Finally, security is core in SnowSQL's design. SnowSQL secures connections to Snowflake using TLS (Transport Layer Security) with OCSP (Online Certificate Status Protocol – OCSP) checks. The auto-upgrade binaries are always validated by using RSA signature. In addition to the secured connection, SnowSQL provides end-to-end security of data moving in and out of Snowflake by using AES (Advanced Encryption Standard) for Snowflake's PUT and GET commands.

In the next few weeks, we will provide a deeper dive into some of the unique capabilities of SnowSQL. Please stay tuned as we gather and incorporate feedback from our customers. We would like to acknowledge our main software engineers Shige Takeda (@smtakeda) and Baptiste Vauthey (@thabaptiser) for their main contributions.

→ by default DML statements are auto commit in snowflake

Autocommit

By default, a DML statement executed without explicitly starting a transaction is automatically committed on success or rolled back on failure at the end of the statement. This behavior is called autocommit. This behavior is controlled with the AUTOCOMMIT parameter.

→How to find error occurred in snow flake queries.

Ans: using try_cast

https://docs.snowflake.com/en/sql-reference/functions-conversion.html#label-try-conversion-functions

Error-handling Conversion Functions

Conversion functions with a TRY_ prefix are special versions of their respective conversion functions. These functions return a NULL value instead of raising an error when the conversion can not be performed:

- TRY_CAST
- TRY_TO_BINARY
- TRY_TO_BOOLEAN
- TRY_TO_DATE
- TRY_TO_DECIMAL, TRY_TO_NUMBER, TRY_TO_NUMERIC
- TRY_TO_DOUBLE
- TRY_TO_GEOGRAPHY
- TRY_TO_TIME
- TRY_TO_TIMESTAMP / TRY_TO_TIMESTAMP_*

Script to insert failed rows into a table while inserting into table

Refer below file for full script


bad rows logging
ogic using try_to .txt

To load only matched/unmatched use merge

https://docs.snowflake.com/en/sql-reference/sql/merge.html

(1)When not matched then insert (2) When matched then update

2) clustred index and non clustered index(oracle)

3) while loading data error occurred few records are error out. Then how to load error records

Ans:

Fix Errors and Load Again

In regular use, you would fix the problematic records manually and write them to a new data file. Alternatively, you could regenerate a new data file from the data source containing only the records that did not load.

You would then stage the fixed data files to the S3 bucket and attempt to reload the data from the files.

4) how to log error query text and query id which is failed .

```
select H.*
from table(information_schema.query_history()) H , (select last_query_id() AS QUERY_ID) Q
WHERE H.QUERY_ID=Q.QUERY_ID
AND H.ERROR_MESSAGE IS NOT NULL;
```

>>>>

last_query_id

Specifies the query to return, based on the position of the query (within the session).

If no query parameter is specified, the most recently-executed query is returned

Default: -1

Instead of last_query_id() w ecan use simply '_last'

Ex: below both commands are same

```
select * from table(validate(${P_TABLE_NAME}, job_id = '_last'))
select * from table(validate(${P_TABLE_NAME}, job_id = last_query_id()))
```

>>>>>>

5) WHY DOES SNOWFLAKE LAST_QUERY_ID() FUNCTION RETURNS NULL WHEN I TRY TO GET THE QUERY ID WHICH IS EXECUTED IN STORED PROCEDURE?

Refer :

https://community.snowflake.com/s/article/Why-does-Snowflake-LAST-QUERY-ID-function-returns-NULL-when-I-try-to-get-the-query-ID-which-is-executed-in-stored-procedure

Snowflake LAST_QUERY_ID() function returns NULL if you try to get the query ID which is executed in stored procedure. This is expected behaviour. To overcome this issue, it's possible to create the stored procedure with "EXECUTE AS CALLER" clause.

Example:

There is a security limitation that placed on purpose to prevent accessing query IDs in stored procedure unless the procedure is created with "EXECUTE AS CALLER" clause.

Let's create a simple test case and observe the result of LAST_QUERY_ID() function:

```
CREATE OR REPLACE PROCEDURE last_query_id_test()

RETURNS STRING NOT NULL

LANGUAGE JAVASCRIPT

AS

$$

snowflake.execute( {sqlText: "select CURRENT_TIMESTAMP"} )

return 'OK';

$$

;

CALL last_query_id_test();

SELECT LAST_QUERY_ID(), LAST_QUERY_ID(-2);
```

You will see that LAST_QUERY_ID() will return the query ID of calling our function, but LAST_QUERY_ID(-2) will return NULL instead of the ID of the query executed within the stored procedure. Please note that LAST_QUERY_ID() and LAST_QUERY_ID(-1) are same because the default value of the query number parameter is -1.

If the stored procedure is created with “EXECUTE AS CALLER” clause, it’s possible to see the query IDs executed in the stored procedure.

```
CREATE OR REPLACE PROCEDURE last_query_id_test()

RETURNS STRING NOT NULL

LANGUAGE JAVASCRIPT

EXECUTE AS CALLER

AS

$$

snowflake.execute( {sqlText: “select CURRENT_TIMESTAMP”} )

return ‘OK’;

$$

;

CALL last_query_id_test();

SELECT LAST_QUERY_ID(), LAST_QUERY_ID(-2);
```

This time, LAST_QUERY_ID(-2) will return the ID of the query executed executed within the stored procedure.

5.1) **execute multiple sql statemnts in stored procedure**

Note: if we use multiple sql in procedure if first one has error/exception then it wont goes to second one. For this we have to write java try catch exception

Below is the example to run multiple sql with out try catch

multiple sql without try catch.sql

Below code is example for simple single sql statement with try catch

o/p is : Success

if we use var my_sql_command = “select count(*) from emp”; in above query it will throw error bcz emp table not exist then o/p is

Failed: Code: 100183 State: P0000 Message: SQL compilation error: Object 'EMP' does not exist or not authorized. Stack Trace: At Statement.execute, line 6 position 21

5.2) procedure with multiple sql statemnts with option report errors and continue or report first error and stop.

This updated stored procedure handles generated SQL statements that may encounter an error. You have two options to handle errors — report errors and continue or report first error and stop.

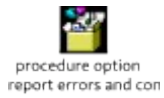
The stored procedure is overloaded, meaning that you can call it with or without the second parameter "continueOnError". If you do not supply the second parameter, it will default to false and stop after the first error.

The output of the stored procedure is as follows:

<SQL statement to run> –Succeeded

<SQL statement to run> –Failed: <reason for failure>

By indicating the success or failure status as a SQL comment, you can modify and re-run the line manually or do some troubleshooting.



https://support.snowflake.net/s/article/Executing-Multiple-SQL-Statements-in-a-Stored-Procedure-Part-Deux

https://dwgeek.com/snowflake-stored-proceduressyntax-limitations-and-examples.html/

5.3 Snowflake Stored Procedure Limitations

Following are the some of Snowflake stored procedure limitations.

- Currently, Snowflake stored procedure does not support transaction control commands such as BEGIN, COMMIT and ROLLBACK. Stored procedure runs entirely within a single transaction.
- JavaScript cannot refer the **third-party** libraries within stored procedures.
- Currently, you can only nest **up to 8** stored procedures.
- Sometimes, calling too many stored procedures at the same time can cause a **deadlock**.

ALTER PROCEDURE

Modifies the properties for an existing stored procedure. Currently, the only supported operations are renaming a stored procedure or adding/overwriting/removing a comment for a

stored procedure. If you need to make any other changes to a stored procedure, use DROP PROCEDURE instead and then recreate the stored procedure.

To see the code of a procedure

Return the DDL to create a stored procedure named stproc_1 that has one parameter of type FLOAT:

select get_ddl('procedure', 'stproc_1(float)');

5.4 USING IF ELSE AND CALLING UDTFS IN SNOWFLAKE STORED PROCEDURES

https://support.snowflake.net/s/article/Using-IF-ELSE-and-Calling-UDTFs-in-Snowflake-Stored-Procedures

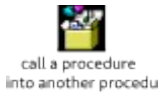
EXECUTE AS CALLER VS OWNER IN STORED PROCEDURE

- Do not allow non-owners to view information about the procedure from the PROCEDURES view.
- Do not allow non-owners to view information about the procedure from the PROCEDURES view.
- Do not allow non-owners to view information about the procedure from the PROCEDURES view.

5.5 How do I call a procedure into another procedure

https://community.snowflake.com/s/question/0D50Z00009VvipRSAR/how-do-i-call-a-procedure-into-another-procedure

in below procedure TEST_CALL_SP we are calling another procedure TEST_CALLED_SP



1. CREATE OR REPLACE PROCEDURE TEST_CALLED_SP(PARAM1 TEXT, PARAM2 float)
2. RETURNS VARIANT
3. LANGUAGE JAVASCRIPT

EXECUTE AS CALLER

```
4.  AS
5.  $$
6.  return [PARAM1, PARAM2];
7.  $$
8.  ;
9.  CREATE OR REPLACE PROCEDURE TEST_CALL_SP(PARAM1 TEXT)
10. RETURNS VARIANT
11. LANGUAGE JAVASCRIPT
12. EXECUTE AS CALLER
13. AS
14. $$
15.
16. var param2 = 12345;
17. var return_rows = [];
18. var stmt = snowflake.createStatement({
19. sqlText: 'CALL TEST_CALLED_SP(:1, :2)',
20. binds: [PARAM1, param2]
21. });
22. var result = stmt.execute();
23. result.next();
24. return_rows.push(result.getColumnValue(1))
25. return return_rows;
26. $$
27. ;
28. CALL TEST_CALL_SP('Testing');
```

This should return a JSON:

[[“Testing”, 12345]]

5.6 diff b/w stored procedures and UDF

5.7 calling Snowflake procedure from python

We have to use conn.cursor().execute this statement is same for sql queries and procedure also

Snowflake procedure

In python after creating snowflake connation

conn.cursor().execute(“call read_person_proc()”)

PYTHON CONNECTOR

Using cursor to Fetch Values in python

https://docs.snowflake.com/en/user-guide/python-connector-example.html

Fetch values from a table using the cursor object iterator method.

For example, to fetch columns named “col1” and “col2” from the table named testtable, which was created earlier (in Creating Tables and Inserting Data), use code similar to the following:

```
cur = conn.cursor()
try:
cur.execute(“SELECT col1, col2 FROM test_table ORDER BY col1”)
for (col1, col2) in cur:
print(‘{0}, {1}’.format(col1, col2))
finally:
cur.close()
```

Alternatively, the Snowflake Connector for Python provides a convenient shortcut:

```
for (col1, col2) in con.cursor().execute(“SELECT col1, col2 FROM testtable”):
print(‘{0}, {1}’.format(col1, col2))
```

If you need to get a single result (i.e. a single row), use the fetchone method:

col1, col2 = con.cursor().execute("SELECT col1, col2 FROM testtable").fetchone()

print('{0}, {1}'.format(col1, col2))

If you need to get the specified number of rows at a time, use the fetchmany method with the number of rows:

cur = con.cursor().execute("SELECT col1, col2 FROM testtable")

ret = cur.fetchmany(3)

print(ret)

while len(ret) > 0:

ret = cur.fetchmany(3)

print(ret)

Note

Use fetchone or fetchmany if the result set is too large to fit into memory.

If you need to get all results at once:

results = con.cursor().execute("SELECT col1, col2 FROM testtable").fetchall()

for rec in results:

print('%s, %s' % (rec[0], rec[1]))

To set a timeout for a query, execute a "begin" command and include a timeout parameter on the query. If the query exceeds the length of the parameter value, an error is produced and a rollback occurs.

In the following code, error 604 means the query was canceled. The timeout parameter starts Timer() and cancels if the query does not finish within the specified time.

conn.cursor().execute("create or replace table testtbl(a int, b string)")

conn.cursor().execute("begin")

try:

conn.cursor().execute("insert into testtbl(a,b) values(3, 'test3'), (4,'test4')", timeout=10) # long query

except ProgrammingError as e:

if e.errno == 604:

print("timeout")

conn.cursor().execute("rollback")

else:

raise e

else:

conn.cursor().execute("commit")

##

How to use variable filename(file name will changing) in snowflake copy command

Suppose file name in s3 bucket will in the format tkt_master_2020_jun_10 (i.e year month n time)

In copy command we can't change file name we can give file name as expression. This can be achived by snowflake java procedure

.we will use variable to get the date and time and we will concat that variable while creating snowsql query in java.

Refer:

https://stackoverflow.com/questions/61597017/how-can-i-add-datetime-stamp-to-zip-file-when-unload-data-from-snowflake-to-s3

https://bigdatadave.com/2020/03/21/snowflake-dynamic-unload-path-copy-into-location/

5.8 HOW TO COPY A DATABASE FROM ONE ACCOUNT TO ANOTHER ACCOUNT using replication

https://community.snowflake.com/s/article/How-to-Copy-a-Database-from-One-account-to-Another

This article is to list out the steps to be performed to copy a database from one account to another using Replication Feature.

Apr 10, 2020 • How To

This article is to list out the steps to be performed to copy a database from one account to another using Replication Feature.

Prerequisites:

1. Both the accounts are under the same Organization.
2. Replication is enabled at the account level (both the source and Target Account)
3. The Source Database should not have been created from a share.
4. Source Database should not include External Tables.

When using the replication feature to create a secondary database, the secondary database is maintained in Read-Only (R/O) mode which can refresh data from the Primary DB based on the required intervals. A new database can be created from the Secondary Database using the clone command option and that database will be in Read-Write (R/W) mode.

Example:

1. Promote the Source Database as a Primary Database.

alter database rep_pr enable replication to account deployment_region.account_name;

e.g if the deployment region is AWS_AP_SOUTHEAST_2 and account name is ABC123 then the command will be:

alter database rep_pr enable replication to account AWS_AP_SOUTHEAST_2.ABC123;

2. Create a replica of the Source Database in the Secondary Account.

create database rep_sec as replica of AWS_AP_SOUTHEAST_2.ABC123.rep_pr;

3. Refresh the Secondary DB.

alter database rep_sec refresh;

4. Clone the Target DB.

create database third_db clone rep_sec;

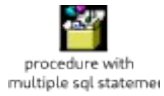
5. Drop the Secondary Database [Optional Step]

Drop database rep_sec;

For further information refer to the Product Documentation on Replication Feature.

https://docs.snowflake.com/en/user-guide/database-replication-failover.html#database-replication-and-failover-failback

6) how to run multiple sql statements from snowflake procedure using dynamic sql



7) horizontal and vertical clustering in snowflake(value labs)

https://support.snowflake.net/s/article/understanding-micro-partitions-and-data-clustering#:~:text=Snowflake%20is%20columnar%2Dbased%20and,in%20the%20same%20micro%2Dpartition.&text=These%20columns%20are%20called%20clustering,you%20to%20recluster%20on%20command.

— snowsql connection

snowsql Command Usage

You can use following command to connect to Snowflake using Snowsql.

snowsql -a accountName -u userName -d databaseName -s schemaName

For example, consider following command to connect Snowflake from Windows command line.

C:>snowsql -a xta99637.us-east-1 -u vithaljs -d demo_db -s public

Password:

* SnowSQL * v1.2.1

Type SQL statements or !help

snowuser#COMPUTE_1@DEMO_DB.PUBLIC>

8)

Running multiple statements Python API throws the following error will throw error

Running multiple statements in Python API throws the following error:

“ProgrammingError: 000006 (0A000): <queryID>: Multiple SQL statements in a single API call are not supported; use one API call per statement instead”

Solution

As of now, Snowflake’s functionality does not support multi-statement transaction and therefore we get this error.
Some of the workarounds are provided below.

- Execute multiple SQL files using SnowSql (command line utility) as described below:

snowsql -c cc -f file1.sql -f file2.sql -f file3.sql

- Once we have downloaded and installed the snowsql tool, we can wrap up all our SQL queries in a single .sql file and call that file using bash.

For example, suppose that we have written all the queries which we would like to run around in a file named “abc.sql” stored in /tmp.
We can then run the following command:

snowsql -a <enter accountname> -u <enter your user name> -f /tmp/abc.sql

7 .Unloading data from Snowflake tables to local system

https://copycoding.com/d/unloading-data-from-snowflake-tables

Similar to data loading, Snowflake supports bulk export (i.e. unload) of data from a database table into flat, delimited text files.

First, Set the Context:

USE WAREHOUSE TRAINING_WH;
USE DATABASE SALES_NAVEEN_DB;
USE SCHEMA SALES_DATA;

For the purpose of this tutorial let us create a temporary sales table, from where we can unload the data

CREATE TABLE SALES_NAVEEN_DB.SALES_DATA.SALES AS select * from snowflake_sample_data.TPCDS_SF100TCL.STORE_SALES LIMIT 1000;

Create a named stage:

create stage my_unload_stage;

Unload the table into a file in the named stage:

copy into @my_unload_stage
from (select * from SALES_NAVEEN_DB.SALES_DATA.SALES)
file_format = (type = csv field_optionally_enclosed_by=‘“’);

List the files to make sure the export was successful

list @my_unload_stage;

Finally, download the files to our local system:

get @my_unload_stage file://C:;

8) BEST PRACTICES FOR DATA UNLOADING

https://community.snowflake.com/s/article/Best-Practices-for-Data-Unloading

Data Unloading Considerations:

A. Defining a File Format:

File format defines the type of data to be unloaded into the stage or S3. It is best practice to define an individual file format when regularly used to unload a certain type of data based on the characteristics of the file needed.
Example :

```
create or replace file format unloading_format
  type = 'csv'
  field_delimiter = '|';
```

B. General File Sizing Recommendations

Unload using parameter SINGLE=FALSE to maximize parallel processing.
The number of files generated is impacted by the warehouse-size:

- Smaller Virtual Warehouse = fewer files with a bigger file size
- Bigger Virtual Warehouse = more files with a smaller file size

Example:

Small WH = 2 nodes * 8 servers = 16 servers which will unload 16 files in parallel at any time
Large WH = 8 nodes * 8 servers = 64 servers which will unload 64 files in parallel at any time

The default file size to be unloaded is 16 MB. 'MAX_FILE_SIZE' is an option that can be used when unloading data. It doesn't determine the size of the output file rather it simply sets an upper limit or caps the file size.

Example:

MAX_FILE_SIZE of 5 GB for a single file (for AWS S3) given would output a file of approx 4.7 GB from a table.

Writing to a single file works only for small tables.

Scenario

For a small table 1GB, using a Large WH (8 cores) would result in 64MB file size. so in order to avoid small files here, you may want to use a smaller warehouse.

To unload a single output file of approx 5GB, you can specify SINGLE = TRUE option
Example :

copy into @mystage/myfile.csv.gz from mytable

file_format = (type=csv compression='gzip')

single=true

max_file_size=4900000000;

C. Unloading considerations for Semi-Structured Data(Json and Parquet)

A relational table can be unloaded into a file using OBJECT_CONSTRUCT function in conjunction with the copy command.
Example:

copy into @mystage

from (select object_construct("col1","col2","col3","col4") from mytable)

file_format = (type = json);

Options For Data Unload:

Transformations:

- Copy command supports column reordering, column omission when issuing a select statement within a copy command.
- It can include sequence columns, timestamp() and other supported functions

  https://docs.snowflake.net/manuals/user-guide/data-load-transform.html
- Not supported: Joins, filters, and aggregations
- VALIDATION_MODE parameter not supported in transformations

Error_handling:

- ON_ERROR option determines the flow of data load.

Validation:

- Run Copy command in validation mode

Copy into mytable

from @mystage/file1.csv.gz

validation_mode = return_all_errors;

- Validate Function after load

  Returns all the errors during load of last copy command.

select * from mytable(validate(table,job_id => '_last'));

Examples:

Unloading to the named internal stage:

create or replace stage my_unload_stage

file_format = my_csv_unload_format;

copy into @mystage/unloading/ from table1;

Unloading to the external stage(S3 etc):

create or replace stage my_ext_unload_stage url='s3://unload/files/'

storage_integration = s3_int

file_format = my_csv_unload_format;

copy into @my_ext_unload_stage/d1 from mytable;

Unloading directly int S3 bucket:

copy into s3://mybucket/unload/ from mytable storage_integration = s3_int;

9) how to insert data insto multi tables :

INSERT (multi-table)

https://docs.snowflake.com/en/sql-reference/sql/insert-multi-table.html

https://roboquery.com/app/syntax-insert-multi-table-command-snowflake

Insert – ALL

When the *ALL* option is specified, all of the associated tables are loaded with the specified data.

Insert – OVERWRITE ALL

When the *OVERWRITE ALL* option is specified, the insert behavior is the same as that for the *ALL* option.

Conditional Multi-table Insert

The conditional insert includes a *WHEN* clause that allows the user to include logic to be applied to the process of inserting data into the target tables.

There are two forms of the conditional insert statement, distinguished by the use of the clause *ALL* or *FIRST*.

Insert – ALL

When the *ALL* option is specified, all 'when' clauses are evaluated and, where found to be true, the associated tables are loaded with the specified data.

Insert – OVERWRITE ALL

When the *OVERWRITE ALL* option is specified, the insert behavior is the same as that for the *ALL* option.

Insert – FIRST

When the *FIRST* option is specified, each WHEN clause is evaluated in order. The load actions associated with the first WHEN clause to return true are executed and the remaining WHEN clauses are ignored.

Insert – OVERWRITE FIRST

To overwrite data in the target tables the script will use the *overwrite first* clause:

```
insert overwrite first
 when n1 > 100 then
  into mti_target1
 when n1 > 10 then
  into mti_target1
  into mti_target2
 else
```

into mti_target2

select n1, n2, n3

From mti_source;

Refer attached doc which has queries from snowflake documentation

multi table insert.docx

- **writing data from pandas to snow flake**

Writing Data from a Pandas DataFrame to a Snowflake Database

https://docs.snowflake.com/en/user-guide/python-connector-api.html#label-python-connector-api-write-pandas

To write data from a Pandas DataFrame to a Snowflake database, do ***one*** of the following:

- Call the write_pandas() function.-→ prefer this
- Call the pandas.DataFrame.to_sql()method, and specify pd_writer as the method to use to insert the data into the database.
  - ex for write_pandas

- The following example writes the data from a Pandas DataFrame to the table named 'customers'.
- import pandas
- from snowflake.connector.pandas_tools import write_pandas
- # Create the connection to the Snowflake database.
- cnx = snowflake.connector.connect(...)
- # Create a DataFrame containing data about customers
- df = pandas.DataFrame([('Mark', 10), ('Luke', 20)], columns=['name', 'balance'])
- # Write the data from the DataFrame to the table named "customers".
- success, nchunks, nrows, _ = write_pandas(cnx, df, 'customers')

->Python script to connect snow flake rela time using mysql imp by ashish

DataLoading.py

→Python to oracle connection

https://www.geeksforgeeks.org/oracle-database-connection-in-python/

Read data base records into dataframe and write it back to db

Refer : https://github.com/codebasics/py/blob/master/pandas/21_sql/pandas_sql.ipynb

https://docs.sqlalchemy.org/en/13/core/engines.html

for oracle sqlalchamy

The Oracle dialect uses cx_oracle as the default DBAPI:

```
import pandas as pd
import cx_Oracle
import sqlalchemy
### creating datbase conenction
engine = sqlalchemy.create_engine('oracle+cx_oracle://talend1:password@xe')
```

Read entire table in a dataframe using read_sql_table

```
#### to read entire table we will use read_sql_table
df = pd.read_sql_table('emp',engine)
##Read only selected columns
df_column= pd.read_sql_table('emp', engine, columns=["empno","ename","sal"])
print(df_column)
```

Join two tables and read them in a dataframe using read_sql_query

```
###read_sql_query is used to run queries using select
df_simpleqry = pd.read_sql_query("select empno,ename from emp",engine)
print('********simple query using read_sql_query')
print(df_simpleqry)
```

*#### query with joins using read_sql_query

```
*query_join = **'''
select e.empno, e.ename,e.job,d.deptno,d.dname
from emp e ,dept d
```

where e.deptno =d.deptno
'''

**df_query_join = pd.read_sql_query(query_join,engine)

print('*********query joins using read_sql_query********')
print(df_query_join)

read_sql is a wrapper around read_sql_query and read_sql_table

####read_sql is a wrapper around read_sql_query and read_sql_table. read_sql is used for both full table and query
print('*********query joins using read_sql********')
df_read_sql_only =pd.read_sql(query_join,engine) *### sql query as passed to read_sql*
print('*********table using read_sql********')
df_read_sql_fulltable =pd.read_sql('emp',engine) *## table name emp passed to read_sql*

Write to oracle database using to_sql

example of to_sql .here df_read_sql_fulltable has data frame which we got from above statemnet
to_sql is auto commit no need of separate commit
df_read_sql_fulltable.to_sql(
name='emp_pandas_test', *# database table name*
con=engine,
if_exists='append',
index=False
)

Using Textual SQL exceuitng raw sql

SQLAlchemy lets you just use strings, for those cases when the SQL is already known and there isn't a strong need for the statement to support dynamic features. The text() construct is used to compose a textual statement that is passed to the database mostly unchanged.

*##########sqlalchemy to run raw sql **select using text** ################3

***from** sqlalchemy.sql **import** text*
s = text("select empno,ename,deptno from emp e where e.empno between :x and :y")

##a =engine.execute(s, x = 7600, y = 7800).fetchall()
a =engine.execute(s, x = 7600, y = 7800) ### it will give o/p as class so we can convert that into dataframe
*print(type(a))
print('**********text op')
print(a)
b= pd.DataFrame(a,columns=["empno","ename","deptno"])

print('**********text to dataframe op')
print(type(b))
print(b)
*#########sqlalchemy to run raw sql **update** ###############3*
u = text("**update emp_pandas_test e set deptno=:v_deptno where e.empno between :x and :y**")
engine.execute(u, v_deptno=77,x = 7600, y = 7800) *### it will update dept as 90 and auto commits the data

*print('** **update using text module****')

→ cx_Oracle

sometimes as the part of programming, we required to work with the databases because we want to store huge amount of information so we use databases, such as Oracle, MySQL etc. So In this article, we will discuss the connectivity of Oracle database with Python. This can be done through the module name **cx_Oracle**.

Oracle Database
For communicating any database with our Python program, then we required some connector which is nothing but the *cx_Oracle* module.

For installing cx_Oracle :

pip install cx_Oracle

By this command, you can install cx_Oracle package but it is required to install Oracle database first in your PC.

How to use this module for connection

- **Import database specific module**
  Ex. import cx_Oracle
- **connect():** Now Establish a connection between Python program and Oracle database by using connect() function.

con = cx_Oracle.connect('username/password@localhost')

- **cursor():** To execute sql query and to provide result some special object required is nothing but cursor() object

cursor = cx_Oracle.cursor()

- **execute method :**

cursor.execute(sqlquery) – – – -> to execute single query.
cursor.execute(sqlqueries) – – – -> to execute a group of multiple sqlquery seperated by ";"

- **commit():** For DML(Data Manuplate Language) query in this query you have (update, insert, delete) operation we need to commit() then only the result reflecte in database.
- **Fetch():** This retrieves the next row of a query result set and returns a single sequence, or None if no more rows are available.
- **close():** After all done mendentory to close all operation
- cursor.close()

con.close()

→Snowflake store procedure with exception handling in java:

https://community.snowflake.com/s/article/How-to-Migrate-Existing-Stored-Procedures-to-Snowflake-Cloud-Data-Platform-Using-JavaScript

Stored procedures are commonly used to encapsulate logic for data transformation, data validation, and business-specific logic. By combining multiple SQL steps into a stored procedure, you can reduce round trips between your applications and the database.

A stored procedure may contain one or many statements and even call additional stored procedures, passing parameters through as needed.

Snowflake supports JavaScript-based stored procedures. In a previous blog, I explained how to convert the existing stored procedure using python.

Pre-requisites:

- Snowflake account and access to create objects in Snowflake.

Sample SP

```
CREATE or replace PROCEDURE employee_temp (cond_param IN int,
tmp_table_name INOUT varchar(256)) as $$
DECLARE
```

```
 row record;
BEGIN
 EXECUTE 'drop table if exists' || tmp_table_name;
 EXECUTE 'create temp table' || tmp_table_name || ' as select * from employee where
salary >= ' || cond_param;
END;
$$ LANGUAGE plpgsql;
```

The above stored procedure is used to create a temp table with condition after dropping the existing table.

This stored procedure is accepting two parameters:

- COND_PARAM with Integer data type
- TMP_TABLE_NAME with varchar data type

Similar to the above stored procedure, Snowflake supports lightweight javascript-based stored procedures and has very good documentation.

Here is the Snowflake equivalent stored procedure:

```
CREATE OR REPLACE PROCEDURE SAMPLE_PROC(tmp_table_name
string,cond_param float)
 returns string
 language javascript
 execute as caller
AS
$$
   try{
   var drop_sql  = "DROP TABLE IF EXISTS" + TMP_TABLE_NAME;
   var drop_stmt = snowflake.createStatement({sqlText : drop_sql});
   var drop_res  = drop_stmt.execute();
   while(drop_res.next()){
   try{
   var create_sql  = 'CREATE TEMP TABLE' + TMP_TABLE_NAME + ' AS SELECT *
FROM EMPLOYEE WHERE SALARY >=' + COND_PARAM;
   var create_stmt = snowflake.createStatement({sqlText : create_sql});
   create_stmt.execute();
   }
   catch(err){
```

```
        return “Error while creating TEMP TABLE :” + err;
        }
        return “Temp table creation:”+ TMP_TABLE_NAME +’ success!’
        }
          }
        catch(err){
          return “Error while dropping the table :” + err;
        }
    $$;
```

The following command will execute your SP:

call SAMPLE_PROC(‘EMPLOYEE_STG’,1000);

- First, it will try to drop the temp table. If it is successful, it will create the table using the filter condition
- At the end of successful execution, you will get a console result: “Temp table creation : EMPLOYEE_STG success!”

If there are any exceptions during any SQL execution it will be, catch and returns at end of SP execution.

PRADEEP CH MATERIEAL STARTS

Shared vs shared nothing reference articles:

https://medium.com/@a.kaushik5587/what-makes-snowflake-so-powerful-its-the-hybrid-of-shared-disk-and-shared-nothing-architecture-5b4fa8f039fa

http://www.benstopford.com/2009/11/24/understanding-the-shared-nothing-architecture/

Query processing in snow flake

Shared n shared nothing vs snowflake

Shared nothing architecture

Shared nothing drawbacks

Main drawback of shared nothing is compute resources are tightly coupled each other

Shared nothing hardware problems

Snow flake hybrid model:

Cloud service layer provides , Authentication, Infrastructure management , Metadata management, Query parsing and optimization and Access control

Snowflake process queries using virtual ware house layer/compute layer

In snowflake data will be stored internal optimized,compressed,columnar format

The data objects stored by Snowflake are not directly visible nor accessible by customers; they are only accessible through SQL query operations run using Snowflake

Snowflake architecture topic ends here

Section 4: Caching in snowflake data warehouse

What is Snowflake Caching ?

Refer : https://metriccamp.com/snowflake/caching.html

Imagine executing a query that takes 10 minutes to complete. Now if you re-run the same query later in the day while the underlying data hasn't changed, you are essentially doing again the same work and wasting resources

Instead Snowflake caches the results of every query you ran and when a new query is submitted, it checks previously executed queries and if a matching query exists and the results are still cached, it uses the cached result set instead of executing the query. This can greatly reduce query times because Snowflake retrieves the result directly from the cache.

Snowflake Cache results are global and can be used across users.

eg. If User A executes a query and the results are cached,

When User B executes the same query the results will be served from cache

Type of Caching Layers in Snowflake ?

Below image is from snowflake level up

https://articles.analytics.today/snowflake-cache-how-it-works-and-why-it-matters

1. Query Results Caching:

The Results cache holds the results of every query executed in the past 24 hours. These are available across virtual warehouses, so query results returned to one user is available to any other user on the system who executes the same query, provided the underlying data has not changed.

2. Virtual Warehouse Local Disk Caching

Whenever data is needed for a given query it's retrieved from the Remote Disk storage, and cached in SSD and memory of the Virtual Warehouse. This data will remain until the virtual warehouse is active. When there is a subsequent query fired an if it requires the same data files as previous query, the virtual warhouse might choose to reuse the datafile instead of pulling it again from the Remote disk

3. Metadata Cache

This is not really a Cache. Instead, It is a service offered by Snowflake. Snowflake automatically collects and manages metadata about tables and micro-partitions

- Row Count
- Table Size in Bytes
- File references and table versions

For Micro-Partitions, Snowflake stores:

- The range of values (MIN/MAX values)
- Number of distinct values
- NULL Count

For Clustering, Snowflake Stores:

- The total number of Micro-Partitions
- Number of Micro-Partitions containing values overlapping with each together
- The depth of overlapping Micro-Partitions
    - This is an indication of how well-clustered a table is since as this value decreases, the number of pruned columns can increase.

All DML operations take advantage of micro-partition metadata for table maintenance. Some operations are metadata alone and require no compute resources to complete, like the query below

```
SELECT MIN(L_SHIP_DATE), MAX(L_SHIP_DATE) FROM LINE_ITEM;
```

Micro-partition metadata also allows for the precise pruning of columns in micro-partitions. When pruning, Snowflake does the following:

1. Snowflake's pruning algorithm first identifies the micro-partitions required to answer a query.
2. Snowflake will only scan the portion of those micro-partitions that contain the required columns.
3. Snowflake then uses columnar scanning of partitions so an entire micro-partition is not scanned if the submitted query filters by a single column.

Benefits of Snowflake Query Caching ?

- Results Cache is Automatic and enabled by default. You do not have to do anything special to avail this functionality
- All the Results are Cached for 24 hours
- There is no space restictions. Snowflake Cache has infinite space (aws/gcp/azure)
- Cache is global and available across all WH and across users
- Faster Results in your BI dashboards as a result of caching
- Reduced compute cost as a result of caching

What happens to Cache results when the underlying data changes ?

Snowflake Cache results are invalidated when the data in the underlying micro-partition changes. Although more information is available in the Snowflake Documentation, a series of tests demonstrated the result cache will be reused unless the underlying data (or SQL query) has changed. As a series of additional tests demonstrated inserts, updates and deletes which don't affect the underlying data are ignored, and the result cache is used, provided data in the micro-partitions remains unchanged

Finally, results are normally retained for 24 hours, although the clock is reset every time the query is re-executed, up to a limit of 30 days, after which results query the remote disk

How to disable Snowflake Query Results Caching?

To disable the Snowflake Results cache, run the below query. It should disable the query for the entire session duration

alter session set use_cached_result =false;

How to run a query without cache usage?

While you cannot disable the data cache or tell Snowflake, "Don't use the data cache", you can temporarily clear the cache, which is automatically maintained in the virtual warehouse using the following SQL.

alter warehouse prod_reporting_vwh suspend;

The next SQL statement will automatically resume the warehouse, starting with a clean cache, and data will be read from remote storage (which will be slower than when it's read from local storage.

Be aware, however, if you immediately restart the virtual warehouse, Snowflake will try to recover the same database servers and restore the cache, although this is not guaranteed.

Because suspending the virtual warehouse clears the data cache, it is good practice to set an automatic suspension of around 5-10 minutes for warehouses used for online queries. However, warehouses used for batch processing can be suspended within 60 seconds.

How to Disable the Snowflake Results Cache

While it is impossible to disable the virtual warehouse cache, the option exists to disable the results cache, although this only makes sense when benchmarking query performance. Use the following SQL statement:

alter session set use_cached_result = false;

Setting the parameter USE_CACHED_RESULT = FALSE is often used to temporarily suspend the Snowflake results cache purely for performance testing benchmarks. Executing the following command resets this parameter:

alter session set use_cached_result = TRUE;

Different States of Snowflake Virtual Warehouse ?

1. **Run from Cold Virtual Warehouse:** When you all your Virtual warehouses are suspended (nothing active), and if you run a query, it will start a new instance of Virtual Warehouse (Cold). Which meant starting a **NEW** virtual warehouse (with no local disk caching), and executing the query.

2. **Run from Warm Virtual Warehouse:** Your virtual warehouse has been active and running for a while and has processed few queries, Then its called WARM Virtual warehouse. Now, If you disable the result caching, and repeat the query.It will make use of the local disk caching which it pulled in the past, which is termed as Warm Caching

3. **Run from Hot Virtual Warehouse:** Which means you repeated the query execute, and the result caching is switched on. The results are fully served from the cache and this is the most efficient operation among all the three types

Lets go through a small example to notice the performace between the three states of the virtual warehouse. In this example we have a 60GB table and we are running the same SQL query but in different Warehouse states

	Cold Ware house	**Warm Ware house**	**Hot Ware house**	
Run	20	1.2	2	The lower the Query Run time, the better

	Cold Ware house	Warm Ware house	Hot Ware house	
Time	seconds	seconds	milliseconds	
Remote Disk (Source)	12.5 GB	0	0	This is the data that is being pulled from Snowflake Micro partition files (Disk)
Local Disk Cache	0	12.5 GB	0	This is the files that are stored in the Virtual Warehouse disk and SSD Memory. The more the local disk is used the better
Results Cache	0	0	100%	The results cache is the fastest way to fullfill a query

Cashing clarifications

Refer :

https://visualbi.com/blogs/snowflake/caching-techniques-snowflake/

Let's have a deep understanding of how Caching is done in Snowflake with a few examples. We will use pre-built benchmark tables that come along with Snowflake Datawarehouse. The dataset contains details about *Parts* and its *Attributes*. It contains 200 million rows that constitute to 5GB of data.

The *Virtual Warehouse* of size *XS* with *Single Cluster* is used. The Execution time might vary depending on the size of the virtual Warehouse and the number of clusters.

Once the Virtual Warehouse is ON, the Part Manager and Part Brand is selected along with the total Retail Price

Since the Virtual Warehouse is just turned on the Local disk cache is empty. The data will be fetched from Remote disk S3 storage, processed and then cached in Local disk SSD.

The query execution plan is as follows

Total execution time is 4.2s, 0% of data is scanned from Cache and 87% of the time is taken in Remote Disk IO

The same Query is executed with an additional column *p_name*. The expectation is to select the needed columns from the Local disk Cache and the new column from the Remote Disk S3.

The Query Execution plan is as follows,

This time 21.38% of data is scanned from local disk cache and only 4% of the time is spent in Remote Disk IO. For better understanding, *the same query or the subset of the query* can be rerun with result cache disabled in order to fetch the whole data from Local disk cache. By default, the result cache will be enabled. This can be disabled using the following command -and the same query will be rerun again.

100% of the result set is scanned from the Local Disk cache. The cache is Volatile and it is dropped once the Virtual Warehouse is suspended, this might result in slower initial performance once the warehouse is resumed. Better Performance can be achieved by segmenting the Query workload by grouping similar Users in the same Virtual Warehouse thus the query used by one user can be used by the other user. The size of the local disk cache depends on *the number of servers* in the warehouse. Larger the Warehouse, increased number of servers hence larger the size of Cache. Hence the awareness of losing the performance benefits of cache should be there before reducing the size or suspending the Warehouse.

The same query is now executed with the result cache enabled. The Query Execution plan is as follows.

The *Result Cache* is independent of the Virtual Warehouses hence any query executed by any user in the account is available in the Result cache provided the SQL query is the same. Many dashboard applications involve the re-execution of the same SQL across the various screens where data can be fetched from result cache for fast retrieval. Each query executed is retained for 24 hours and the time is reset to 31 days if the query is re-executed. Since the cache occurs in the service layer there is *no cost* associated with it (*no storage or computation cost*). Retrieval optimization and Post-processing query results can be achieved using Result cache for optimized performance.

The Result cache of the query will automatically be used only if it meets few conditions. The conditions can be found in this
blog https://community.snowflake.com/s/article/Understanding-Result-Caching.

Result cache can be explicitly used while building a complex query using the following command

As discussed above the Unique features present in Snowflake caching at different levels is as follows,

- No Configuration for setting up the cache is involved
- Cache available from all the warehouse
- Result cache persists for 24 hours
- Infinite space for storage Cache(S3)
- Faster retrieval

Hence correct utilization of the cache depending upon the scenario will make the system Cost-efficient in terms of computing and storage with Optimized performance

HOW TO: UNDERSTAND RESULT CACHING

Refer: https://community.snowflake.com/s/article/Understanding-Result-Caching

Description
Snowflake caches and persists the query results for every executed query. This can be used to great effect to dramatically reduce the time it takes to get an answer.

Typically, query results are reused if ***all*** of the following conditions are met:

- The user executing the query has the necessary access privileges for all the tables used in the query.
- The new query syntactically matches the previously-executed query.
- The table data contributing to the query result has not changed.
- The persisted result for the previous query is still available.
- Any configuration options that affect how the result was produced have not changed.
- The query does not include functions that must be evaluated at execution (e.g. CURRENT_TIMESTAMP()).
- The table's micro-partitions have not changed (e.g. been re-clustered or consolidated) due to changes to other data in the table.

To verify whether a query made use of the result cache, check the query profile in the Snowflake UI. It will show a node like the following:

When working with cached results, the following functions are useful:

- RESULT_SCAN, to access the cached result directly: https://docs.snowflake.net/manuals/sql-reference/functions/result_scan.html

Example: Run a complex query, and then to access its results again, execute the following query:

select * from table(result_scan(last_query_id()))

This allows you to retrieve the results again even if the conditions above not satisfied. However in this case, it's not happening automatically, but rather we need to explicitly state that we want the results of the last query.

- DESC RESULT, to get the columns available in a cached result: https://docs.snowflake.net/manuals/sql-reference/sql/desc-result.html

##################33

Query result will be cached upto 24 hours of query execution

Snowflake wont charge to store this result in cache

Imp questions on cache:

(1) Which of these are snowflake cache layers? → result cache,local disk cache, remote disk

(2) **Which cache layer holds query result for 24 hours? →result cache**

(3) Result cache belongs to ? → service layer

(4) Local disk cache belongs to-→ Compute layer

(5) Remote disk is nothing but, → blob storage area like aws s3

(6) Cold , Hot , Warm `match these cache layers with below sequence → remote disk,result cache,local disk cache

Cold→ remote disk cache

Warm→Local disk cache

Hot→ result cache

(7) alter session set use_cached_result=flase — this command will disable Result Cache

(8) You can not disable remote disk cache, Local disk cache

(9) When underlying data for table changes, we use → remote disk cache

(10) I can use Result cache even if i suspend my warehouse → True

Section 5: Clustering in snowflake.

What are Micro-partitions?

https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions.html

All data in Snowflake tables is automatically divided into micro-partitions, which are contiguous units of storage. Each micro-partition contains between 50 MB and 500 MB of uncompressed data (note that the actual size in Snowflake is smaller because data is always stored compressed). Groups of rows in tables are mapped into individual micro-partitions, organized in a columnar fashion. This size and structure allows for extremely granular pruning of very large tables, which can be comprised of millions, or even hundreds of millions, of micro-partitions.

Snowflake stores metadata about all rows stored in a micro-partition, including:

(1)The range of values for each of the columns in the micro-partition.

(2)The number of distinct values.

(3)Additional properties used for both optimization and efficient query processing.

Benefits of Micro-partitioning:

The benefits of Snowflake's approach to partitioning table data include:

- In contrast to traditional static partitioning, Snowflake micro-partitions are derived automatically; they don't need to be explicitly defined up-front or maintained by users.
- As the name suggests, micro-partitions are small in size (50 to 500 MB, before compression), which enables extremely efficient DML and fine-grained pruning for faster queries.
- Micro-partitions can overlap in their range of values, which, combined with their uniformly small size, helps prevent skew.
- Columns are stored independently within micro-partitions, often referred to as *columnar storage*. This enables efficient scanning of individual columns; only the columns referenced by a query are scanned.

- Columns are also compressed individually within micro-partitions. Snowflake automatically determines the most efficient compression algorithm for the columns in each micro-partition

Micropartition depth:

Refer :

https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions.html

http://cloudsqale.com/2019/12/02/snowflake-micro-partitions-and-clustering-depth/ –it is neatly explained

https://docs.snowflake.com/en/user-guide/tables-clustering-keys.html

https://www.analytics.today/blog/snowflake-clustering-best-practice

Above imageis from link : https://docs.snowflake.com/en/user-guide/tables-clustering-keys.html

The process of grouping records with in micro partition is called clustering

Clustering Depth

The clustering depth for a populated table measures the average depth (1 or greater) of the overlapping micro-partitions for specified columns in a table. The smaller the average depth, the better clustered the table is with regards to the specified columns.

Clustering depth can be used for a variety of purposes, including:

- Monitoring the clustering "health" of a large table, particularly over time as DML is performed on the table.
- Determining whether a large table would benefit from explicitly defining a clustering key.

A table with no micro-partitions (i.e. an unpopulated/empty table) has a clustering depth of 0.

Cluster key creation syntax:

Cluster key CTS interview question

- **For one million unique records data if we add cluster key is performance will be improved. ANS: NO**

- **If above table is joined with any other table on clustering key then it will improve performace? Yes**

- If all records are unique you should not be adding any clustering key. As it will not add any benefit.

- If you cluster the table based on the joining key(assuming it's having less cardinality) then you can see performance improvement.

A clustering key can be defined when a table is created by appending a CLUSTER BY clause to CREATE TABLE:

CREATE TABLE <*name*> ... CLUSTER BY (<*expr1*> [, <*expr2*> ...])

Where each clustering key consists of one or more table columns/expressions, **which can be of any data type, *except* VARIANT, OBJECT, or ARRAY**. A clustering key can contain any of the following:

- Base columns.

- Expressions on base columns.

- Expressions on paths in VARIANT columns.

Important Usage Notes

- If you define two or more columns/expressions as the clustering key for a table, the order has an impact on how the data is clustered in micro-partitions.

  For more details, see Strategies for Selecting Clustering Keys (in this topic).

- An existing clustering key is copied when a table is created using CREATE TABLE ... CLONE.

- An existing Clustering key is ***not*** propagated when a table is created using CREATE TABLE ... LIKE.

- An existing clustering key is ***not*** supported when a table is created using CREATE TABLE ... AS SELECT; however, you can define a clustering key after the table is created.

- Defining a clustering key directly on top of VARIANT columns is not supported; however, you can specify a VARIANT column in a clustering key if you provide an expression consisting of the path and the target type.

Changing the Clustering Key for a Table

At any time, you can add a clustering key to an existing table or change the existing clustering key for a table using ALTER TABLE:

ALTER TABLE <*name*> CLUSTER BY (<*expr1*> [, <*expr2*> ...])

Dropping the Clustering Keys for a Table

At any time, you can drop the clustering key for a table using ALTER TABLE:

ALTER TABLE <*name*> DROP CLUSTERING KEY

Manually Reclustering a Table

Use ALTER TABLE with a RECLUSTER clause to manually recluster a table for **which a clustering key have been defined**. You can use a WHERE clause to specify a condition or range on which to recluster data in the table.

For example:

alter table t1 recluster;

alter table t2 recluster where create_date between ('2016-01-01') and ('2016-01-07');

These examples use the current warehouse (for the session) to recluster the table. The amount of resources allocated to manual reclustering is based on the size of the warehouse. The larger the warehouse, the more resources are allocated to the recluster command, which results in more effective reclustering.

Note

Manual reclustering can only be performed on clustered tables (i.e. tables that have a clustering key defined).

Clustering precautions:

Imp points on clustering: mcq:

(1)When user submits query, snowflake submits it to→ Cloud service layer

(2)Which of these are part of query life cycle in cloud service layer?–> parsing,object resolution,access control and plan optimization

(3) Query execution plan will be submitted to ? → worker nodes in virtual warehouse

(4) All query information and statistics are stored for audits and performance analysis. → in cloud service layer

(5) Once execution plan is submitted to virtual warehouse layer, what will happen?–> Nodes will download HEADER FILE remote disk and based on it,scans meta data

(6) select name , department from employee. This query will download ?→ only name and department columns from remote disk based on HEADER FILE information

(7) Micro-partitioning is performed automatically on all snowflake tables

(8) Tables are partitioned based on the ordering of data as it is inserted or loaded → TRUE

(9) Micro partitions size when uncompressed → 50-500 MB

(10) In micro-partition each column is stored independently → TRUE

(11) Degree of overlap of micro-partitions in snowflake is called → Micro partition depth

(12) Constant micro-partition will have a depth of → 1

Section 6: Clustering — Deep dive.

Checking clustering information:

15.is it possible to drop default clustering key …

Ans :Snowflake doesnot apply any default clustering key…. If clustering key is existed then we can drop .

ALTER TABLE <name> DROP CLUSTERING KEY

Note: even if we not created cluster key on table and we use drop cluster key command it wont throw error …looks like snowflake is maintaining clustering key internally.

to summarize:

- **No Default Clustering Key**: Snowflake does not apply a default clustering key to tables.
- **Automatic Optimization**: Snowflake manages data organization internally through micro-partitions.
- **Custom Clustering Key**: For large tables or specific query patterns, you might manually define clustering keys to improve performance.

https://docs.snowflake.com/en/sql-reference/functions/system_clustering_information.html

SYSTEM$CLUSTERING_INFORMATION

Syntax

SYSTEM$CLUSTERING_INFORMATION('<table_name>' , '(<col1> [, <col2> ...])')

You can use this argument to return clustering information for any columns in the table, regardless of whether a clustering key is defined for the table.

In other words, you can use this to help you decide what clustering to use in the future

Important point for clustering (Pradeep ch udemy topic 27 how clustering works)

When we are loading data into a table, snowflake will not organize micro partitions while loading data. After data load completes Snowflake will reorganize the partitions (it will be done in back ground).

Please remember that snowflake will charge for that reorganizing partitions

Topic 28. Improve performance without applying clustering

As Snowflake will charge to arrange partitions to avoid this billing , if we are sure that on which key clustering is required then while loading data from source table itself we can use **order by that column** so that partitions will be organized without creating cluster key on that column which will save cost

In above example we are loading data by using order by C_MKSEGMENT column so that with out creating clustering key snowflake arrange partitions.

We can check clustering info by using system_clustering_information

Chapter 29. Manual Re-clustering

Using alter we can add cluster key . after adding cluster key using alter, to arrange partitions manually we will use RECLUSTER command with alter.

Note

Manual reclustering can only be performed on clustered tables (i.e. tables that have a clustering key defined).

30. How to choose clustering keys.

Clustering deep dive quiz

(1)It's good idea to apply clustering on the table, which frequently changes.–> False bcz Every time data loads in to table. Micro partitions will re group based on the clustering key. If data size is huge, re clustering cost will increase.

(2) **I will order data by columns, on which i more frequently filter data; before i load data in to table.**

Ans: Good idea bcz Micro partitions in back-end will remain well grouped. Remember it is not similar to applying clustering key. Clustering will re group micro partitions based on recently loaded data. But doing an order by, will not re group old micro partitions but it will only ensure better grouping while loading data to table.

(3) **Clustering keys, cardinality order should be Low cardinality to high cardinality**

Means clustering key columns order ,first column should have low cardinality ,then second column should have high cardinality

(4) **I can use `substring` function on clustering keys to get better cardinality value over a column → TRUE**

(5) Can i check clustering information even if i don't have clustering applied on the table? Yes you can use but information will not be accurate.

(6) SELECT SYSTEM$CLUSTERING_INFORMATION('CUSTOMER_NOCLUSTER','(C_MKTSEGMENT,C_CUSTKEY)');

(7)

33. Introduction to virtual warehouse.

One person can be assigned with multiple vartual warehouses. Also many users can use one virtual warehouse. So both scenarios are possible.

If many users are assigned with one virtual warehouse then user queries will start queuing up to solve this problem We can spin up the virtual warehouse to a larger size say like a medium size but when all your queries get processed we should be manually spinning down this to a small vartual house.If you are not spinning it down then this medium cluster will add to the cost.

There is another way to solve this problem.Instead of spinning up a larger warehouse we can ask when query start queuing up, You can ask snowflake to spin up and add another extra small warehouse .Once all the queries got processed then you can ask snowflake to automatically spin it down in this way we can solve the problem of queuing up off queries and still will be maintaining extra small warehouse in this way.We don't have to care about spinning down the larger ware house so this solution is called multi cluster.

Auto scale mode:

Maximize mode:

35. virtual warehouse Scaling policy

1. **So the first question was how many queries does snowflake queues before it spins up additional cluster?**
2.

It dependes on scaling policy. If we are using **standard** policy then what will happen is the queries will **immediately** spin up the new cluster when either a query is queued or the system detects that there is a one or more query than the currently running clusters can execute.

But if you choose the **economy** then answer for that question is only if the system estimates. There is enough query load to keep the cluster busy for at least six minutes.

(2) What is the trigger to suspend a cluster if no load is present?

It has 2 answers it depends on policy which you choose if it is **standard** policy then after two to three consecutive successful checks which determine whether the load on the least loaded cluster would be redistributed to the other clusters without spinning up the cluster again very thoughtful.

If it is **economy** policy **after 5 to 6 consecutive** successful checks(performed at 1 minute intervals) which determine whether the load on the least-loaded cluster cloud be redistributed to the clusters without spinning up the cluster again.

Virtual ware house questions:

Questions 2

Questions 3

Question 4:

Question5:

Question 6:

Question 7:

Question 8:

To help control the usage of credits in Auto-scale mode, Snowflake provides a property, SCALING_POLICY, that determines the scaling policy to use when automatically starting or shutting down additional clusters

Question 9:

Question 10:

Question 11:

Question 12:

44. Types of internal stage

Data Transformation in Snowflake –by Ashish

CONSUMING DATA FROM SNOWfLAKE- Ashish

PRADEEP CH MATERIEAL END

————————————–snwflake rela time environment setup for stage n production
————————

How to setup your Snowflake environment when moving on-premise databases to the cloud

https://medium.com/@bnatarajan/how-to-setup-your-snowflake-environment-when-moving-on-premise-databases-to-the-cloud-8505cf25b5fd

In on-premise database environments like Oracle and SQL Server there will usually be multiple physical servers and in each of them there will be multiple databases. For example, in a typical on-premise Oracle/SQL Server data warehouse environment, companies will have 3 separate sets of physical servers — one each for development, test and production.

In each of these physical servers, multiple databases will be created. Each of these databases will built be for a specific purpose for example one could be a financial systems warehouse (FIN_DW) with data pulled from ERP system and another could be a HR warehouse (HR_DW) with data pulled from HR systems and a third could be data from CRM systems like Salesforce which could be called CRM_DW. In each of the databases there can be multiple schema and in each schema, there can be multiple tables, views and other objects.

So, in total you could have 3 databases per server looking like this in your on-premise environment –

DEV/TEST/PROD Physical Server

In an on-premise environment the following picture depicts a typical hierarchy of objects

How Snowflake works

When a company signs up with Snowflake they are given an URL like

https://companyname.snowflakecomputing.com/

In Snowflake, a database is the highest level and inside a database there can be multiple schemas and inside a schema there can be multiple tables and views. So, in other words Snowflake does not have a server concept like Dev, Test or Production physical servers.

In Snowflake, the following picture depicts a typical hierarchy of objects

How to organize your on-premise databases in Snowflake

Given, that Snowflake environment is "one level lower" in terms of objects (there is no concept of dev, test and production physical servers) how do you organize the Snowflake system to match your on-premise setup. There are 2 ways to go about this –

1. Keep your top-level object as an individual database when you migrate

In this method, you will be creating as many databases in Snowflake as there are number of on-premise physical servers x number of databases in each of them.

In the above example you will create 9 databases in Snowflake –

CRM_DW_DEV

HR_DW_DEV

FIN_DW_DEV

CRM_DW_TEST

HR_DW_TEST

FIN_DW_TEST

CRM_DW_PROD

HR_DW_PROD

FIN_DW_PROD

This method will work fine if you have a small number of on-premise servers with a small number of databases in each of them. But your company could have 4–5 physical servers (Sandbox, Dev, Test, Production, etc.) with 10–20 databases in each of them. You can imagine how the number of databases can proliferate in Snowflake. In this example your looking at anywhere between 40 to 100 databases.

You will have to maintain all these databases within Snowflake and assign security and roles to each of them. In addition, in my opinion you will have a very confusing and cluttered environment to maintain many databases for the longer term.

One of the big issue I see is that normally production servers have a high degree of security and access control than dev or test servers. In the on-premise world the server and the databases in the production environment are audited and are under SOX control. In Snowflake if you end up having 10–20 production databases without an umbrella physical server it will become difficult to report out the internal controls to the audit team.

2. Create as many "dummy databases" as there are on-premise physical servers

In this method you create 3 databases in Snowflake at the top level –

1. Development

2. Test

3. Production

This will represent the 3 physical servers in your on-premise environments. Then you can create the 3 on-premise databases (CRM_DW, HR_DW, FIN_DW) as schemas inside these 3 databases. If a database has multiple schemas you can create multiple schemas inside these databases. For example, if CRM_DW has 2 schemas called Marketing_Schema and Sales_Schema, you can create these as 2 separate schemas as CRM_DW_Marketing_Schema and CRM_DW_Sales_Schema under the Development, Test and Production databases. The respective tables and views can then be created under each of these schemas.

The advantage I see in this method is that you have a more structured way of looking at your Snowflake environment. You will have a Development, Test and Production database and then all schemas and tables that belong to each will be sitting inside these databases. You can put a greater level of security control to the Production database and will be able to prove to your auditors that you have similar controls to the production on-premise server.

The only downside I see to this approach is the case where you a lot of schemas under a database in your on-premise environment. In this case you will just have to rename your schemas with the database name in the front for example to distinguish them.

Summary

Before moving your on-premise data warehouses to Snowflake, it is necessary to put some thought into how you want to organize your Snowflake environment. Since you don't have a concept of a physical development, test or production servers you can try to mimic it by using option 2 above. Option 2 will work well if you have a lot of databases inside each physical server and you have less number of schemas in each database. If you have a lot of schemas in each database and less number of databases in each physical server then option 1 might be better suited for your case.

Create development environment using sampling technique

Refer Pradeep hc material and below link

https://www.snowflake.com/blog/the-dream-data-warehouse-development-environment/

Earlier this month, Snowflake's Customer Enablement Team was assigned an email from one of our customers. The customer stated that he was not happy about the idea of cloning full copies of production databases for development purposes. "Do we really want to develop and maintain a system to copy a fraction of the production DB to dev?", citing the reason for his message that, by just copying the entire production database, the dev team would have access to too much data. Being a veteran of Snowflake, I initially dismissed his concern because of Snowflake's zero-copy clone capability, as outlined in this article. From my perspective, the zero-copy clone

would not incur any additional cost for development purposes, so why not give the dev team all of the data?

Use row sampling and block sampling

The approach outlined by the customer was rooted in complimenting Snowflake's zero-copy clone with the additional technique of using *Block Sampling*

let's take a look at SAMPLE / TABLESAMPLE and see how we can do this. The syntax is quite simple:

DEVOPS n change management in snowflake (CI/CD)

This material is from snowflake official videos

https://www.youtube.com/watch?v=vGqRyMlvYjo&t=133s

There are 2 different approaches for database change management (schema migration or schema change management)

The video author worked on snowchnage.he likes snowchange and flyway.

Roles and grants in real time

https://medium.com/hashmapinc/heres-your-day-1-and-2-checklist-for-snowflake-adoption-e0e7ff8f105a#:~:text=Creating%20Roles%20and%20Role%20Hierarchy&text=in%20Snowflake%2C%20the%20user%20should,on%20the%20object%20of%20choice.

Grant access to database objects in a schema to a Role in Snowflake

Snowflake uses Roles to manage user access provisioning. You create a role with a set of accesses on a particular Table / Schema / Database. Then you assign that ROLE to a USER.

You can grant the USAGE access to Warehouse / Database / Schema.

Grant usage on the database:

GRANT USAGE ON DATABASE <database> TO ROLE <role>;

Grant usage on the schema:

GRANT USAGE ON SCHEMA <database>.<schema> TO ROLE <role>;

Grant the ability to query an existing table:

GRANT SELECT ON TABLE <database>.<schema>.<table> TO ROLE <role>;

The following table privileges are supported:

Privilege

Usage

SELECT

Execute a SELECT statement on the table

INSERT

Execute an INSERT command on the table

UPDATE

Execute an UPDATE command on the table

TRUNCATE

Execute a TRUNCATE command on the table

DELETE

Execute a DELETE command on the table

REFERENCES

Reference the table as the unique/primary key table for a foreign key constraint

ALL [PRIVILEGES]

Grant all privileges, except OWNERSHIP, on the table

OWNERSHIP

Grant full control over a table.

NOTE: You will need to provide the Schema / Database level grants again whenever you create a new table. (It is not auto-refreshed)

SHOW GRANTS on a Table / Role / User in Snowflake

Snowflake uses ROLES to provision access rules. The SHOW GRANTS Command lists all access control privileges that have been granted to roles, users, and shares.

Table level grants:

SHOW GRANTS ON TABLE schema.table;

Database level grants:

show grants on database sales;

+————————————————+———————+————————+————————+————————+————————+————————+

| created_on | privilege | granted_on | name | granted_to | grantee_name | grant_option | granted_by |

|————————————————+———————+————————+————————+————————+————————+————————|

| Thu, 07 Jul 2016 05:22:29 -0700 | OWNERSHIP | DATABASE | REALESTATE | ROLE | ACCOUNTADMIN | true | ACCOUNTADMIN |

| Thu, 07 Jul 2016 12:14:12 -0700 | USAGE | DATABASE | REALESTATE | ROLE | PUBLIC | false | ACCOUNTADMIN |

+————————————————+———————+————————+————————+————————+————————+————————+

Role level grants:

show grants to role analyst;

```
+———————————————+———————————+———————+———————+———————+———————+
———————+
| created_on | privilege | granted_on | name | granted_to | grant_option | granted_by |
|———————————————+———————————+———————+———————+———————+———————+
———————+
| Wed, 17 Dec 2014 18:19:37 -0800 | CREATE WAREHOUSE | ACCOUNT | DEMOENV | ANALYST |
false | SYSADMIN |
+———————————————+———————————+———————+———————+———————+———————+
———————+
```

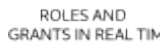
User level grants:

show grants to user demo;

To see all the list of users belonging to a role:

show grants of role analyst;

```
+———————————————+———+———————+———————+———————+
| created_on | role | granted_to | grantee_name | granted_by |
|———————————————+———+———————+———————+———————|
| Tue, 05 Jul 2016 16:16:34 -0700 | ANALYST | ROLE | ANALYST_US | SECURITYADMIN |
| Tue, 05 Jul 2016 16:16:34 -0700 | ANALYST | ROLE | DBA | SECURITYADMIN |
| Fri, 08 Jul 2016 10:21:30 -0700 | ANALYST | USER | JOESM | SECURITYADMIN |
+———————————————+———+———————+———————+———————+
```



HOW TO SCHEDULE A SNOWSQL JOB IN CRONTAB

https://snowflakecommunity.force.com/s/article/How-to-schedule-a-Snowsql-job-in-crontab

https://dwgeek.com/how-to-execute-snowflake-commands-from-shell-script-example.html/

https://dwgeek.com/access-snowflake-using-snowsql-without-password-prompt-snowsql-environment-variables.html/

Jun 22, 2020 • How To

Problem Description

We will demonstrate how to schedule a SnowSQL job through Crontab.

Solution

Through the following commands we will create a script named **cronjob.sh** (for example), and put it in Crontab file.

snowsql -c my_connection <<EOF >> logs.lst

–your set of queries

SELECT current_timestamp();

!exit

EOF

OR

snowsql -c my_connection -f filename.txt >> logs.lst

The output of the queries can be seen in the **logs.lst** file, which will be created after the execution of the script.

Note: In order to configure the connection parameter as mentioned above i.e **my_connection** for Snowsql, go to the location where Snowsql is installed. The default location is **<home folder>/.snowsql**. Open **config** file in the edit mode and add the username, password and the account information in the connection section.

Snowflakedev ops

SNOWFLAKE KEY ENCRYPTION

https://metriccamp.com/snowflake/encryption.html

https://metriccamp.com/snowflake/encryption-key-management.html

Snowflake Cloud Datawarehouse Data Encryption and Security

Snowflake is highly secure and has granular permission levels. In this tutorial, you will learn,

Data Security Features in Snowflake

- Automatic data encryption by Snowflake using Snowflake-managed keys.
- All communication and data transfer between clients and the server protected through TLS
- You can Choose the geographical location where your data is stored, based on your cloud region
- Support for PHI data (in compliance with HIPAA regulations) — requires Business Critical Edition.
- Deployment inside a cloud platform VPC (AWS) or VNet (Azure).
- Isolation of data (for loading and unloading) using:
  - Amazon S3 policy controls
  - Azure SAS tokens
  - Google Cloud Storage access permissions

User Access control Features in Snowflake

- User authentication through standard user/password credentials.
- Enhanced User authentication:
  - Multi-factor authentication (MFA)
  - Federated authentication and single sign-on (SSO)
  - OAuth
- Object-level access control

Data Protection Features in Snowflake

- Snowflake Time Travel (1 day standard for all accounts; additional days, up to 90, allowed with Snowflake Enterprise)

- Snowflake Fail-safe (7 days standard for all accounts) for disaster recovery of historical data

Data Encryption

Snowflake encrypts all customer data by default, using the latest security standards, at no additional cost. Snowflake provides best-in-class key management, which is entirely transparent to customers

End-to-End Encryption

End-to-end encryption (E2EE) means the data is encrypted as it leaves the user till it gets loaded and vice versa. In Snowflake, this means that only a customer and the runtime components can read the data. No third parties, including Snowflake's cloud computing platform or any ISP, can see data in the clear

In the above example:

1. A user uploads one or more data files to a stage. If the stage is a customer-managed container in a cloud storage service (option A), the user may optionally encrypt the data files using client-side encryption. regardless, Snowflake immediately encrypts the data when it is loaded into a table. If the stage is an internal (i.e. Snowflake: option B), data files are automatically encrypted by the client on the local machine prior to being transmitted to the internal stage
2. The user loads the data from the stage into a table. The data is transformed into Snowflake's proprietary file format and stored in a cloud storage container ("data at rest"). In Snowflake, all data at rest is always encrypted
3. Query results can be unloaded into a stage. Results are optionally encrypted using client-side encryption when unloaded into a customer-managed stage, and are automatically encrypted when unloaded to a Snowflake-provided stage
4. When the user downloads data files from the stage and decrypts the data on the client side

E2EE minimizes the attack surface. In the event of a security breach of the cloud platform, the data is protected because it is always encrypted, regardless of whether the breach exposes access credentials indirectly or data files directly, whether by an internal or external attacker

Client-Side Encryption

Client-side encryption provides a secure system for managing data in cloud storage. Client-side encryption means that a user encrypts stored data before loading it into Snowflake. The cloud storage service only stores the encrypted version of the data and never includes data in the clear

1. The Snowflake customer creates a secret master key, which remains with the customer.

2. The client (provided by the cloud storage service) generates a random encryption key and encrypts the file before uploading it into cloud storage. The random encryption key, in turn, is encrypted with the customer's master key.

3. Both the encrypted file and the encrypted random key are uploaded to the cloud storage service. The encrypted random key is stored with the file's metadata.

If you are using client side encryption, Snowflake will need the keys to read the files. The following SQL snippet creates an example Amazon S3 stage object in Snowflake that supports client-side encryption

– create encrypted stage

create stage encrypted_customer_stage

url='s3://customer-bucket/data/'

credentials=(aws _key_id='ABCDEFGH' aws_secret_key='12345678')

encryption=(master_key='eSxX0jzYfIamtnBKOEOxq80Au6NbSgPH5r4BDDwOaO8=');

Encryption Key Management in Snowflake (KMS)

Security is quintessential when it comes moving all your data to the cloud. At Snowflake, all data in your cloud data warehouse is encrypted by default, using latest security standards and best practices, at no additional cost

There are three important concepts with Key Management Systems (KMS)

1. Hierarchical Key Model
2. Key Rotation
3. Rekeying

Hierarchical Key Model

A hierarchical key model is the highlight of Snowflake's encryption key management. A key hierarchy has several layers of keys where each layer of keys (the parent keys) encrypts the layer below (the child keys). When a key encrypts another key, security experts refer to it as "wrapping". In other words, a parent key in a key hierarchy wraps all of its child keys

Snowflake's hierarchical key model consists of four levels of keys:

- The root key
- Account master keys
- Table master keys
- File keys

As the name implies, Each account master key corresponds to one customer account in Snowflake. Each table master key corresponds to one database table in a database. That means that every account and every table is encrypted with a separate key. Similarly, every single data file is encrypted with a separate key.

Hierarchical key models are super unique, as each layer of keys reduces the scope of their applicability. For example, table master keys reduce the scope of their applicability to single tables; file keys further reduce the scope of applicability to single files. Thus, a hierarchical key model is essential to constrain the amount of data each key protects, and the duration of time during which it is usable

Encryption Key Rotation

Account and table master keys are automatically rotated by Snowflake when they are more than 30 days old. Active keys are retired, and new keys are created. When Snowflake determines the retired key is no longer needed, the key is automatically destroyed. When active, a key is used to encrypt data and is available for usage by the originator. When retired, the key is used solely to decrypt data and is only available for usage by the recipient. When wrapping child keys in the key hierarchy, or when inserting data into a table, only the current, active key is used to encrypt data. When a key is destroyed, it is not used for either encryption or decryption. Regular key rotation limits the lifecycle for the keys to a limited period of time

- Version 1 of the TMK is active in April. Data inserted into this table in April is protected with TMK v1.
- In May, this TMK is rotated: TMK v1 is retired and a new, completely random key, TMK v2, is created. TMK v1 is now used only to decrypt data from April. New data inserted into the table is encrypted using TMK v2.
- In June, the TMK is rotated again: TMK v2 is retired and a new TMK, v3, is created. TMK v1 is used to decrypt data from April, TMK v2 is used to decrypt data from May, and TMK v3 is used to encrypt and decrypt new data inserted into the table in June.

Encryption Keys - Rekeying

Rekeying is the process of re-encrypting data with new keys. After a specific time interval, data that has been encrypted with an old key gets re-encrypted with a new key

Key rotation = “new data gets fresh keys”, Rekeying = “old data gets fresh keys”

The TMK in this figure is rotated every month, as was explained in the previous section. In addition, the TMK in Figure 3 is rekeyed after one year. That is, in April 2015, TMK v1 is rekeyed. A new generation 2 of TMK v1 is created, a fully new random key. The data files protected by TMK v1, generation 1 are decrypted and encrypted with TMK v1, generation 2.

Because all data files are now protected with a new TMK, the old TMK v1, generation 1 can be destroyed; it is not used anymore. In this example, the life cycle of a key is limited to a total duration of one year. The benefit of rekeying is that it constraints the total duration during which a key is used for recipient usage

Data that is being rekeyed is always available to the customer. No downtime of the service is necessary to rekey data and no performance impact is observed on the customer workload. You will be charged with additional storage for Fail-safe protection of 7 days of data files storage that were rekeyed

Additional Security measures:

- **Hardware Security Module** : Snowflake relies on your cloud-vendors hardware security module (HSM) services as a tamper-proof, highly secure way to generate, store, and use the root keys of the key hierarchy
- **Tri-Secret Secure and Customer-Managed Keys:** With Tri-Secret Secure enabled for your account, Snowflake combines your key with a Snowflake-maintained key to create a composite master key. This composite master key is then used to encrypt all data in your account. If either key in the composite master key is revoked, your data cannot be decrypted, providing a level of security and control above Snowflake's standard encryption. This dual-key encryption model, together with Snowflake's built-in user authentication, enables the three levels of data protection offered by Tri-Secret Secure

LOCKS in snowflake

https://docs.snowflake.com/en/sql-reference/sql/show-locks.html

https://community.snowflake.com/s/article/how-to-resolve-blocked-queries

http://gcdatagroup.com/2019/12/22/show-locks-in-snowflake-datawarehouse/

https://docs.snowflake.com/en/sql-reference/functions/system_abort_transaction.html

SHOW LOCKS

Lists all running transactions that have locks on resources. The command can be used to show locks for the current user in all the user's sessions or all users in the account.

For information about transactions and resource locking, see Transactions.

See also:

SHOW TRANSACTIONS

Syntax

SHOW LOCKS [IN ACCOUNT]

Parameters

IN ACCOUNT

Returns all locks across all users in the account. This parameter only applies when executed by users using the ACCOUNTADMIN role (i.e. account administrators).

For all other roles, the function only shows locks across all sessions for the current user.

Usage Notes

- The command does not require a running warehouse to execute.
- The command returns a ***maximum*** of 10K records for the specified object type, as dictated by the access privileges for the role used to execute the command; any records above the 10K limit are not returned, even with a filter applied.

  To view results for which more than 10K records exist, query the corresponding view (if one exists) in the Information Schema.
- To post-process the output of this command, you can use the RESULT_SCAN function, which treats the output as a table that can be queried.
- The command output includes the IDs for all running transactions that have locks on resources. **These IDs can be used as input for SYSTEM$ABORT_TRANSACTION to abort a specified transaction**.

HOW TO: RESOLVE BLOCKED QUERIES

Solution

On the **History** page in the Snowflake web interface, you could notice that one of your queries has a BLOCKED status. The status indicates that the query is attempting to acquire a lock on a table or partition that is already locked by another transaction.

Account administrators (ACCOUNTADMIN role) can view all locks, transactions, and session with:

SHOW LOCKS IN ACCOUNT;

This command displays all locked objects, as well as all queries waiting for locks. You should see your blocked queries have a status of WAITING, along with the table that it is attempting to lock. Look for the transaction that has a HOLDING status, with a lock on the target table. Note the session and transaction IDs for that lock.

You can view the query history of that session on the **History** page by filtering on the session ID.

If the session is still available, you can execute a COMMIT or ROLLBACK statement to end the transaction and release the locks. If the session is no longer available, you can release your unintended lock by using the transaction ID from the SHOW LOCKS command to execute:

SELECT SYSTEM$ABORT_TRANSACTION(<transaction_id>);

Account administrators can execute this statement on any user's transactions.

SHOW LOCKS in Snowflake Datawarehouse

http://gcdatagroup.com/2019/12/22/show-locks-in-snowflake-datawarehouse/

There is no table available with the lock information in Snowflake, so you have to get a little bit creative to programmatically get information about the various locks in the system.

The basic command to get locking information is SHOW LOCKS. This information can be used to then manually track down queries, but as a lazy SQL programmer, I prefer to figure out a way to query this once and then never have to step through that process manually again.

This article will outline a query that can help short-cut the amount of time to resolve blocked queries.

SHOW LOCKS and RESULT_SCAN

First, we do have to run the SHOW LOCKS command once to get a query result to use in the result_scan

SHOW LOCKS IN ACCOUNT;

We will need to either record the query_id for that statement or rely on RESULT_SCAN using the previously run query_id as the default.

Once the SHOW LOCKS query has been run, the following query can give us some information about the blocking query, as well as generating the statement to abort the transaction if needed.

with locks(query_id, resource, lock_status, blocking_session, transaction,
txn_length,lock_held_length) as(

SELECT "query_id" , "resource", "status", "session", "transaction",

DATEDIFF('seconds', "transaction_started_on", CURRENT_TIMESTAMP) txn_length,

DATEDIFF('seconds', "acquired_on", CURRENT_TIMESTAMP) lock_held_length

FROM TABLE(RESULT_SCAN('$show_locks_query_id'))

), blocking_query as (

```
SELECT l.query_id, blocking_session

FROM locks l

WHERE lock_status='HOLDING'

and resource in (

SELECT resource

FROM locks

WHERE query_id = '$blocked_query_id'

))

SELECT bl.*, ql.query_text, 'SELECT SYSTEM$ABORT_TRANSACTION(' || bl.transaction || ');' as kill_stmt

FROM blocking_query bl

INNER JOIN

(SELECT query_text, query_id FROM TABLE(information_schema.query_history()) WHERE total_elapsed_time < 0) ql

on ql.query_id=bl.query_id
```

If all you want is a query to run then there it is. If you want more detail, I'll breakdown each section of this query for those that may want to adapt this concept to other uses.

Getting the lock info, enriching with some more useful info, and adjusting the column names for better usability. We access the results of the SHOW LOCKS by either explicitly supplying a query_id or be supplying nothing, which returns the previously run query results of the current session.

```
with locks(query_id, resource, lock_status, blocking_session, transaction, txn_length,lock_held_length ) as(

SELECT "query_id" , "resource", "status", "session", "transaction",

DATEDIFF('seconds', "transaction_started_on", CURRENT_TIMESTAMP) txn_length,

DATEDIFF('seconds', "acquired_on", CURRENT_TIMESTAMP) lock_held_length

FROM TABLE(RESULT_SCAN('$show_locks_query_id'))

)
```

Get the query_id and session for the blocking query, so that we can join to get the query text for inspection.

```
blocking_query as (
```

SELECT l.query_id, blocking_session

FROM locks l

WHERE lock_status='HOLDING'

and resource in (

SELECT resource

FROM locks

WHERE query_id = '$blocked_query_id'

))

Join the previous two common table expressions to the query_history where total_elapsed_time is less than 0. This will get all currently running queries. We then project the blocking information, the query text and generate an abort transaction statement to run if it is determined the blocking query should be killed.

SELECT bl.*, ql.query_text, 'SELECT SYSTEM$ABORT_TRANSACTION(' || bl.transaction || ');' as kill_stmt

FROM blocking_query bl

INNER JOIN

(SELECT query_text, query_id FROM TABLE(information_schema.query_history()) WHERE total_elapsed_time < 0) ql

on ql.query_id=bl.query_id</pre></code>

A Definitive Guide to Python Stored Procedures in the Snowflake UI

https://interworks.com/blog/2022/08/16/a-definitive-guide-to-python-stored-procedures-in-the-snowflake-ui/

https://interworks.com/blog/2022/08/09/definitive-guide-python-udfs-snowflake-ui/

Why Use a Python Stored Procedure Instead of SQL Scripting?

Most Snowflake users are far more comfortable with SQL than they are with Python or other advanced programming languages, simply because SQL is the core language required to leverage the Snowflake platform. With that in mind, it is easy to consider using SQL-based stored procedures instead of other languages; such as Python. However, there are several strong reasons to leverage a non-SQL language instead, especially when compared to Python:

- Anything you can achieve with a SQL stored procedure can also be achieved with a Python stored procedure.
- A SQL stored procedure would have a harder time defining and executing functions within itself, especially when this is paired with looping. SQL is fantastic, but I would not argue for it to be a first class programming language like JavaScript and Python.
- Python is a high-level language capable of far more than standard SQL, including the ability to import and leverage functionality from a wide number of modules.

To summarise, Python is simply more versatile than SQL and unlocks a wider range of functionality. Here are a few examples of powerful functionality that is possible with Python and not with SQL:

- Construct and execute SQL queries with greater ease and flexibility than with SQL scripting
- Access supporting files in cloud storage and read their contents to contribute to the overall Python script
- Perform powerful data transformations built out of multiple components using Pandas
- Leverage the map() function or list comprehension to apply a function to each value of a list; a simple way to support looping and iteration
- Apply machine learning models to generate new forecasts using libraries such as PyTorch or scikit-learn
- Generate authentication key pairs and apply them to users

A Warning Regarding Staged Files and Libraries

It is important to note at this time that stored procedures do not have access to the "outside world." This is a security restriction put in place by Snowflake intentionally. If you wish to create a stored procedure that accesses the outside world, for example to hit an API using the requests library, then it is recommended to use external functions instead.

How to Create Your Own Python Stored Procedure from a Snowflake Worksheet

Snowflake have now integrated the ability to create Python stored procedures directly into the standard commands that can be executed from within a Snowflake worksheet. Whether you are

using the classic UI or the new Snowsight UI, you can simply open a worksheet and use this code template to get started.

This template includes line regarding optional packages and imports that will be discussed after our simple examples.

```
CREATE OR REPLACE PROCEDURE <stored procedure name> (<arguments>)
returns <data type> << optional: not null >>
language python
runtime_version = '3.8'
packages=('snowflake-snowpark-python', <optional list of additional packages>)
<< optional: imports=(<list of files and directories to import from defined stages>) >>
handler = '<name of main Python function>'
as
$$
def <name of main Python function>(snowpark_session, <arguments>):
# Python code to determine the main
# functionality of the stored procedure.
# This ends with a return clause:
return <function output>
$$
;
```

The main elements to understand here are:

- Everything within the set of $$s on lines 9 and 15 must be Python code and forms the stored procedure itself. Meanwhile, everything outside of those $$s is Snowflake's flavour of SQL and contains metadata for the stored procedure.
- On row 7, we define what is known as a handler. This is the name of the main function within our Python code that the Snowflake stored procedure will execute. This handler must match a function within the Python code or the stored procedure will fail.
- On row 2, we define the data type that the stored procedure will return. I intend to explain this through the simple examples below.

- On row 5, we import the snowflake-snowpark-python library. This is the main library when the phrase “Snowpark for Python” is discussed and it allows Python to interact with Snowflake.
- An additional argument called “snowpark_session” is also included on row 10 before the other arguments. This is the active Snowflake session being used by snowflake-snowpark-python and is passed into our function so that we can directly interact with Snowflake objects within our stored procedure; for example to execute SQL queries.
- On rows 5 and 6 are lines regarding optional additional packages and imports for the stored procedure. This will be discussed after our simple examples.
- The arguments passed to the stored procedure on row 1 are the same arguments that are passed to the handler function in Python on row 10; however. data types may need to be changed from Snowflake’s flavour of SQL into Python equivalents. This is discussed more in the note below.

A Note on Stored Procedures and Function Arguments

An important thing to keep in mind when creating Python stored procedures is the data types that are permitted.

When defining the metadata of your stored procedure, all data types are viewed from the perspective of Snowflake’s flavour of SQL. Specifically, I am referring to the arguments on line 1 and the returned data type on line 2.

When defining variables within your Python code, any input arguments in the handler function (line 10) or returned values (row 14) must be one of the data types specified in the Python Data Type column of Snowflake’s SQL-Python Type Mappings table.

It is also important to note that the number of arguments passed to the Stored Procedure on row 1 is the same number of arguments passed to the handler Python function on row 10 and must be in the same intended order, with the exception of the snowpark_session variable. Though I would recommend naming the variables similarly for ease of understanding, the names of these variables do not need strictly need to align as long as they are in the same order, leverage similar data types and come after the snowpark_session.

Simple Examples

Let’s break this down and provide a few simple examples to explain what is happening more clearly. Each of these three examples don’t really have any business being a stored procedure instead of a UDF; however, I want to clearly explain how these procedures work in the simplest terms before we move on to the useful examples.

Please note that these examples are deliberately minimalistic in their design, and thus do not include any error handling or similar concepts. Best practices would be to prepare functions with error handling capabilities to prevent unnecessary execution and provide useful error outputs to the end user.

Multiply Input Integer by Three

Our first example is a very simple stored procedure that takes an integer and multiples it by three. First, let's see the code:

```
CREATE OR REPLACE PROCEDURE multiply_integer_by_three(INPUT_INT int)

returns int not null

language python

runtime_version = '3.8'

packages = ('snowflake-snowpark-python')

handler = 'multiply_by_three_py'

as

$$

def multiply_by_three_py(snowpark_session, input_int_py: int):

return input_int_py*3

$$

;
```

There are a few things to break down here to confirm our understanding:

- On rows 9 and 10, we have defined a very simple function in Python which multiples an input number by three.
- The name of our handler function on row 6 matches that of our Python function defined on row 9.
- On row 2, we can see that our function will return an integer and that the integer will not be NULL.
- The INPUT_INT argument passed to the stored procedure on row 1 will be the integer that is passed to the input_int_py argument when executing the handler function in Python on row 9.

If we execute this code in a Snowflake worksheet, we can then call the stored procedure, in the same way that we would any other stored procedure, to see the result.

Multiply Two Input Integers Together

Our second example is another simple function, this time taking two input integers and multiplying them together. Let's see the code:

```
CREATE OR REPLACE PROCEDURE multiply_two_integers_together(
INPUT_INT_1 int
, INPUT_INT_2 int
)
returns int not null
language python
runtime_version = '3.8'
packages = ('snowflake-snowpark-python')
handler = 'multiply_together_py'
as
$$
def multiply_together_py(
snowpark_session
, input_int_py_1: int
, input_int_py_2: int
):
return input_int_py_1*input_int_py_2
$$
;
```

There are a lot of similarities between this and our previous function; however, this time we have multiple inputs.

Again, if we execute this code in a Snowflake worksheet, we can then call the stored procedure, in the same way that we would any other stored procedure, to see the result.

Multiply All Integers in an Input Array by Another Integer

The last of our simple examples takes things one step further, so that we can introduce an additional Python function in our script and demonstrate the power of list comprehension. This stored procedure will accept an array of integers as an input, along with a second input of a single integer. The stored procedure will multiply all members of the array by that second integer.

Let’s see the code:

```
CREATE OR REPLACE PROCEDURE multiply_all_integers_in_array(
INPUT_ARRAY array
, INPUT_INT int
)
returns array not null
language python
runtime_version = ‘3.8’
packages = (‘snowflake-snowpark-python’)
handler = ‘multiply_integers_in_array_py’
as
$$
# First define a function which multiplies two integers together
def multiply_together_py(
a: int
, b: int
):
return a*b
# Define main function which maps multiplication function
# to all members of the input array
def multiply_integers_in_array_py(
snowpark_session
, input_list_py: list
, input_int_py: int
):
# Use list comprehension to apply the function multiply_together_py
# to each member of the input list
return [multiply_together_py(i, input_int_py) for i in input_list_py]
```

$$

;

There are several things to note here:

- On row 5, we can see that we are now expecting an array as an output instead of an integer.
- On rows 13-18, we are defining a second Python function within our script. This function is called multiply_together_py. This is not strictly needed as we could use a lambda function; however, I have included it to demonstrate the functionality.
- On row 29, we use the concept of list comprehension to apply our multiply_together_py function to every member of our input array.

Again, if we execute this code in a Snowflake worksheet, we can then call the stored procedure in the same way that we would any other stored procedure to see the result.

Appendix B — Snowflake Advanced Topics

Source: advanced.md converted from the corresponding uploaded Word document.

Contents

Calling snowflake stored proc from power bi:

https://docs.snowflake.com/en/developer-guide/stored-procedure/stored-procedures-selecting-from

case 1 : proc with out ddl and dml

Select * from (table (cdp_aq.rpt.TEST_VMAC_NEW()) .. we need to use this query in power bi to get the results from snowflake procedure.

Case 2 : strored proc with ddl and dml scenario

External tables

https://interworks.com/blog/2023/09/19/snowflake-external-tables-connect-efficiently-to-your-data-lake/

refer ch Pradeep video from udemy

What is a Snowflake External Table?

- Snowflake external table is a type of table in Snowflake that is not stored in the Snowflake storage area; but instead is located in an external storage provider such as Amazon AWS S3, Google Cloud Storage—GCP, or Azure Blob Storage.
- Snowflake external tables allow users to query files stored in the Snowflake external stage like a regular table without moving that data from files to Snowflake tables.
- Snowflake external tables store the metadata about the data files, but not the data itself
- External tables are read-only, so no DML (data manipulation language) operations can be performed on them
- they can be used for query and join operations.

Diff b/w normal tables and external tables

As external table is created using external stage first we should create storage integration then external stage after that w ecan create external table

Creating external table

CREATE OR REPLACE EXTERNAL TABLE ext_table

WITH LOCATION = @my_s3_stage

FILE_FORMAT = (TYPE = CSV);

As table will have only one column with variant data type .. the data is in json format. W ecan read data AS shown below.

Create external table with column names

External tables will store metadata

Whereas if we query data using only external stage .. in this scenario n snowflake don't maintain metadata. The main advantage of external table compare to querying directcyly from external stage is having metadata.

CASE 1: NEW FILES ADDED TO S3

If any new files added to s3 .. that data will not be reflected in external table.. to reflected that newly added files also we have to REFRESH external table so that metadata will be updated and we are able to see that newly added files data from external table

CASE 2: FILE DELETED FROM S3

If we delete the files from s3 .. but it wont reflect in external table as metadata is not updated . so we have to refresh metadata table then it will be reflected

From below image we can observe one file showing as unregistered .. which is deleted from s3.

**
**

Case 3: updating the s3 file then verify external table

As we updated s3 file.. and we did refresh external table…. Snowflake metadata will be updated..

The hash value for the updated file will change in external table so it will consider as file is updated so its showing as REGISTERED_UPDATE

PARTITIONS on external Tables

Manual partition in external tables (Add partitions manually)

. Use this option when you prefer to add and remove partitions selectively rather than automatically adding partitions for all new files in an external storage location that match an expression.

This option is generally chosen to synchronize external tables with other metastores (e.g. AWS Glue or Apache Hive).

Include the required PARTITION_TYPE = USER_SPECIFIED parameter.

The partition column definitions are expressions that parse the column metadata in the internal (hidden) METADATA$EXTERNAL_TABLE_PARTITION column.

The object owner adds partitions to the external table metadata manually by executing the ALTER EXTERNAL TABLE … ADD PARTITION command:

Note: External table REREFSH wont work for manual adding partions ..it will throw error

Refreshing external table metadata automatically

To create AWS notification :

Go to properties→ under advance settings →select events→click on add notifications→

Enter details as shown below

Here main noting point is we have to select SQS queue

In sqs ARN value we have to copy key from snow flake show pipe notification key value.

Sqs queue ARN value has to be taken from snow flake **show external tables** notification_channel column value

ICEBERG TABLES

Refer ch Pradeep udemy snowflake videos — this videos covers iceberg tables with snowflake has catalog ie. Read/write works

https://www.snowflake.com/blog/iceberg-tables-powering-open-standards-with-snowflake-innovations/

https://www.phdata.io/blog/what-are-iceberg-tables-in-snowflake-and-when-to-use-them/

https://www.snowflake.com/blog/5-reasons-apache-iceberg/

What is Apache Iceberg?

Apache Iceberg is a high-performance open table format designed to manage huge datasets at scale. This format determines how to manage, organize, and track all the data files stored in open file formats that make up a table.

Architecture Overview

The architecture of Apache Iceberg consists of three different layers:

- Catalog
- Metadata layer
- Data layer

COMPARIISON OF external tables vs snowflake tables vs iceberg table

Configure iceburg with snowflake

Iceberg Table Types in Snowflake : —–IMP FROM DOCUMENTATION

1. **Snowflake as the Iceberg catalog**
2. **Use an external catalog**

IMP: ICEBERG TABLE TYPE COMPARISION From snowflake documentation

Use Snowflake as the catalog

An Iceberg table that uses Snowflake as the Iceberg catalog provides full Snowflake platform support with read and write access. The table data and metadata are stored in external cloud storage, which Snowflake accesses using an **external volume.** Snowflake handles all life-cycle maintenance, such as compaction, for the table.

Use an external catalog

An Iceberg table that uses an external catalog provides limited Snowflake platform support with read-only access. With this table type, Snowflake uses a **catalog integration** to retrieve information about your Iceberg metadata and schema.

You can use this option to create an Iceberg table registered in the AWS Glue Data Catalog or to create a table from Iceberg metadata files in object storage.

Snowflake does not assume any life-cycle management on the table.

The table data and metadata are stored in external cloud storage, which Snowflake accesses using an external volume.

The following diagram shows how an Iceberg table uses a catalog integration with an external Iceberg catalog.

Iceberg Table Types in Snowflake

https://www.phdata.io/blog/what-are-iceberg-tables-in-snowflake-and-when-to-use-them/

Depending on where the catalog is managed for Iceberg tables, Snowflake can have two types of Iceberg tables:

1. **Snowflake Managed Iceberg Tables** – Snowflake manages the metadata and catalog for these tables. These tables can support all Snowflake features with read and write access.
2. **Externally Managed Iceberg Tables** – An external system such as AWS Glue manages the metadata and catalog. These tables can support read-only access in Snowflake.

Note: Tables that use Snowflake as the catalog wont support Cross-cloud/cross-region support

Cross-cloud/cross-region tables are supported when you use an external catalog

Hands on iceberg tables(from Pradeep video) catalog managed by snow flake

1. cretate external volume

3.create table

COPY ON WRITE (COW)

- There are two approaches to handle deletes and updates in the data lakehouse: copy-on-write (COW) and merge-on-read (MOR).

- **Copy-on-write (COW) is the default mode for writing Iceberg tables in Snowflake.**
- Copy-On-Write (COW) – **Best for tables with frequent reads**, infrequent writes/updates, or large batch updates
- Merge-On-Read (MOR) – **Best for tables with frequent writes/updates**
-
-
-
-

SNOWFLAKE HYBRID TABLES—IN PREVIEW jun 2024

https://medium.com/snowflake/snowflake-hybrid-tables-all-you-need-to-know-58fd73426698

it supports indexing, unique constraints and integrtity checks like oracle

One of the most significant differences between HYBRID and Standard Tables is that HYBRID uses row locking, allowing for much higher concurrency and throughput.

But let's look at a case with FOREIGN KEY and Secondary Indexes

Note that analytical queries inside HYBRID Tables are around 2 to 3X slower than Snowflake Standard Analytical queries

Query acceleration service

Its similar to increasing the warehouse size dynamically which is useful for slow running queries.

https://thinketl.com/query-acceleration-service-in-snowflake/

3. How Query Acceleration Service in Snowflake works?

Consider a huge query is running on an SMALL warehouse (2 credits per hour), it results in a poor query performance and also results in blocking all other queries from running.

While huge queries running on an X-LARGE warehouse will undoubtedly run faster, this is not an ideal solution when you have only a small number of large queries to run and many small queries. This leads to higher overall running costs as the smaller queries don't make full use of the compute resources, but the cluster is still charged at a rate of 16 credits per hour.

The Query Acceleration Service enabled on **an SMALL warehouse** can automatically detect when a huge query needs more resources and leases additional compute resources to complete the query, and then release the resources when the query is finished. There by reduces the overall cost compared to using a warehouse of bigger size.

The number of resources that the QAS can lease can be controlled using **Scale Factor** defined for QAS.

The above illustration shows that an SMALL warehouse with QAS enabled up to 8X scale factor detects a huge query. Then the QAS deploys additional resources which helps in extracting data faster.

The Query Acceleration Service effectively functions as a group of resources that are temporarily deployable alongside your current warehouse and when needed, takes on some of the heavy lifting.

3. How to enable Query Acceleration Service in Snowflake?
4.

Which queries are supported by Query Acceleration Service?

Currently the Query Acceleration Service can only accelerate certain parts of query which includes fetching data and filtering out the results. However, these are the parts of the query which take most of the execution time. Also the amount of data that is being extracted must be huge for the query to be eligible for query acceleration.

The queries which include the following SQL commands are supported by QAS.

- SELECT
- INSERT (when the statement contains a SELECT statement)

- CREATE TABLE AS SELECT (CTAS)

The parts of query which does aggregation, sorting and other steps are not supported by Query Acceleration Service. Although this may be supported in upcoming releases.

How to identify Queries that take advantage of Query Acceleration Service?

To identify if a query is eligible for Query Acceleration Service, you can use the **SYSTEM$ESTIMATE_QUERY_ACCELERATION** function as shown below.

Search optimization service

https://thinketl.com/search-optimization-service-in-snowflake/

The following are the Key points related to search access paths

- When search optimization is enabled on a table, the process of populating data into search access path for the first time might take significant time depending on the size of the data.
- When data in the table is modified, the search access path is automatically updated by Snowflake maintenance service.
- There is additional cost involved for the storage and compute resources for maintaining the search access path.

8. How to identify which columns are configured for Search Optimization in a table?

To identify the columns and their search optimization configuration in a table, use the **DESCRIBE SEARCH OPTIMIZATION** command.

The following statement shows the estimated costs of adding search optimization to a specific column of a table using EQUALITY search method.

Snowflake tags

https://thinketl.com/tagging-in-snowflake/

ROW ACCESS POLICY

https://thinketl.com/row-level-security-using-row-access-policies-in-snowflake/

3.3. Create a Row Access Policy

The below SQL statement creates a Row Access Policy with following two conditions.

1. User with SYSADMIN role can query all rows of the table.

2. User with DATA_ANALYST roles can query only rows belonging to their country based on the role mapping table.

In the above statement:

country_role_policy specifies the name of the policy.

country_name is the signature of the row access policy which specifies the field and data type of the mapping table to which it links.

returns boolean -> specifies the application of the row access policy.

'SYSADMIN' = current_role() is the first condition of row access policy which allows users with SYSDAMIN role to view all rows of the table.

or exists ... is the second condition of the row access policy expression which uses a subquery. The subquery requires the CURRENT_ROLE to be the custom role which specifies the country through role mapping table. This is used by row access policy to limit the rows to be returned for the query executed by user.

show row access policies;

8. How to Drop a Row Access Policy in Snowflake?

A Row Access Policy cannot be dropped successfully if it is currently attached to a resource. Before executing a DROP statement, detach the row access policy from the table or view.

Follow below steps to drop a row access policy in Snowflake

1. Find the objects on which the row access policy is attached.

The below SQL statement lists all the objects on which row access policy named *country_role_policy* is attached.

The below SQL statement drops row access policy named *country_role_policy*.

drop row access policy country_role_policy;

Query optimization

Dynamic tables

A Dynamic table materializes the result of a query that you specify. It can track the changes in the query data you specify and refresh the materialized results incrementally through an automated process.

To incrementally load data from a base table into a target table, define the target table as a dynamic table and specify the SQL statement that performs the transformation on the base table. The dynamic table eliminates the additional step of identifying and merging changes from the base table, as the entire process is automatically performed within the dynamic table.

Dynamic tables support **Time Travel**, **Masking**, **Tagging**, **Replication** etc. just like a standard Snowflake table

Consider the following example where Dynamic Table 2 (DT2) is defined based on Dynamic Table 1 (DT1). DT2 must read from DT1 to materialize its contents. In addition, a report consumes DT2 data via a query.

Differences Between Snowflake Dynamic Tables and Materialized Views

Materialized Views	Dynamic Tables
A Materialized View cannot use a complex SQL query with joins or nested views.	A Dynamic table can be based on a complex query, including one with joins and unions.
A Materialized View can be built using only a single base table.	A Dynamic table can be built using multiple base tables including other dynamic tables.
A Materialized View always returns the current data when executed.	A Materialized View returns the data latest up to the target lag time.

Dynamic tables General System Limitations

- **Account Limits:** A single Snowflake account is capped at a maximum of **50,000** dynamic tables.
- **No Temporary Tables:** You cannot create a TEMPORARY or VOLATILE dynamic table; they must be permanent or transient.
- **No Truncate:** You cannot use the TRUNCATE command on a dynamic table.

- **Source Restrictions:** Dynamic tables **cannot** read directly from:
  - External tables
  - Directory tables
  - Streams
  - Materialized Views

2. SQL & Query Constraints

Certain SQL constructs are unsupported or behave differently:

- **Dynamic SQL:** You cannot use session variables or unbound variables in the table definition.

Dynamic SQL (Session Variables)

Dynamic Tables must be "self-contained." If a table definition relied on a session variable, Snowflake wouldn't know what value to use when the background refresh process runs (since there is no active user session).

- ❌ **Unsupported:**

  – This fails because $region_name might change or not exist during background refresh

  CREATE OR REPLACE DYNAMIC TABLE sales_by_region

  TARGET_LAG = '5 minutes'

  WAREHOUSE = my_wh

  AS

  SELECT * FROM raw_sales WHERE region = $region_name;

  ✅ **Workaround: Hardcode the value or join against a "Configuration Table" that contains the metadata you need.**

- **User-Defined Functions (UDFs):** * UDFs written in Python, Java, or Javascript that are marked as VOLATILE are not supported.
  - SQL UDFs containing subqueries are blocked.
- **Subqueries:** Subqueries outside of the FROM clause (e.g., in a WHERE EXISTS or WHERE IN block) are generally not supported.

Snowflake currently requires the query structure to be very "flat" to map out the incremental data flow. Subqueries in the WHERE or SELECT clauses (Scalar Subqueries) are difficult to refresh incrementally.

- ❌ **Unsupported (In WHERE/SELECT):**

  SELECT

order_id,

(SELECT name FROM customers c WHERE c.id = o.cust_id) as cust_name – Scalar subquery in SELECT

FROM orders o

WHERE store_id IN (SELECT id FROM stores WHERE city = 'Atlanta'); – Subquery in WHERE

✅ Workaround (Use Joins): Almost any subquery can be rewritten as a JOIN, which *is* supported.

- **Shared Data:** If you are ingesting data from a share, your query cannot select from a shared dynamic table or a shared secure view that references another upstream dynamic table.

Snowpark

For csv files inferschema not supported

For parquet avro schems is attached so ingerschema will work

In below image which are highlighted in yellow colour are spark function that functions are not supported by snowpark version 1.3.0

CREATE SNOWPARK DATA FRAME USING PANDAS DATAFRAME

https://medium.com/snowflake/your-cheatsheet-to-snowflake-snowpark-dataframes-using-python-e5ec8709d5d7

Creating Snowpark Dataframe by reading data from Stage

Creating Snowpark Dataframe by Joining two tables/dataframes

Transforming a DataFrame

Using Select Method to create a new dataframe with specific columns from existing DF

Using **Filter Method** to filter data (Similar to Where Clause)

Using the **SORT Method** to Order the data

Using **AGG Method** to aggregate the data

Using **Group_by** for grouping aggregate results

Using **Window Method** as Window Function

Using **DataFrame NA Function** for Handling Missing Values

READING FROM S3 CSV

FOR CSV FILE WE HAVE TO PROVIDE SCHEMA using structtype and structfield

READING FROM S3 json

Note: employee_s3_json.cache_result() this command will throw error for semi structure data .. it will work for only csv data

We have to use select_expr

SNOWPARK WRITE OPERATIONS

WE USE save_as_table COMMOND TO WRITE THE DATA INTO SNOWFALKE TABLE

Write has different modes. Overwrite will create oe replace table evry time

Mode APPEND will add data .. i.e it will insert data into existing table

Appendix C — DWH & Dimensional Modeling

Source: dwh.md converted from the corresponding uploaded Word document.

Contents

Data Lake architecture [63](#data-lake-architecture)

Refer L Bhaskar Jogi youtube channel for DW

Refer: https://medium.com/@arjunagarwal_22593/building-a-data-warehouse-79923addab0c

https://www.youtube.com/watch?v=lImtOVc8C1E&list=PLskvRY5fTxgcW3m_hDhqmW0QJftoahL12&index=3

https://www.youtube.com/watch?v=Ddzn9siwx6Q&list=PLskvRY5fTxgf36tOnq9P82lGcu29EwpXm&index=5

edureka:

https://www.youtube.com/watch?v=J326LIUrZM8&list=PL9ooVrP1hQOEDSc5QEbI8WYVV_EbWKJwX

Erwin:

https://www.youtube.com/watch?v=IJ-5vC3pjHA

https://www.youtube.com/watch?v=YwVT_pdxbkE

https://www.youtube.com/watch?v=YwVT_pdxbkE

https://www.youtube.com/watch?v=0Vby5YHVkYY

https://www.youtube.com/watch?v=m4S4Mo8U_ao&list=PL2-GO-f-XvjBnTL9CDPBXVTDO_UT_k28O&index=11

dimensional modeling interview questions:

https://tekslate.com/interview-questions-on-dimensional-data-modeling

build datware house with Erwin in 4 steps

https://www.youtube.com/watch?v=YwVT_pdxbkE&t=5s

In OLTP master n transactional tables will be there

In OLAP → master table converted as dimension tables

- Transaction tables are converted as fact tables

Dimension tables are loaded using incremental loading

Scd(slowly changing domensions these methods for dimension tables not for facts tables)

For fact tables we can't use incremental loading

For fact table data load will be full load OR INREMENTAL LOAD

In the screen factAttendance is fact less fact table bcz it don't have any measures or facts

In above image in table Factsales if sales amount , salesqty column are not there then it will become fact less fact table.

Most of the traditional DWH will be snowflake schema

More indexes required in OLAP then reading query will be fast.

Where as in OLTP less indexex are required bcz as indexes are more then writing will be slowdown.

More indexes then DML operation will be slow .DML operations are more in OLTP,

DML operations are less in OLAP. We will use select clause more in OLAP

Star is recommended as speed is high.

In real time snowflake schema will be there

Conformed Dimensions

Conformed dimensions are those dimensions which have been designed in such a way that the dimension can be used across many fact tables in different subject areas of the warehouse. It is imperative that the designer plan for these dimensions as they will provide reporting consistency across subject areas and reduce the development costs of those subject areas via reuse of existing dimensions. The date dimension is an excellent example of a conformed dimension. Most warehouses only have a single date dimension used throughout the warehouse.

Confirmed dimension will be available in galaxy /double stat schema

Conformed Dimension- This is used in multiple locations**.** It helps in creating consistency so that the same can be maintained across the fact tables. Different tables can use the table across the fact table and it can help in creating different reports.

Role playing dimension:

More than 1 relationship b/w fact and dimension table. In below image dimTime Table has 2 relation ships with fact sales tableside sales date ky,delivery datakey

Below screnn shot is from this video :aroundbi channel

https://www.youtube.com/watch?v=gynrzgH6c78

Transactional business processes typically produce a number of miscellaneous, low-cardinality flags and indicators. Rather than making separate dimensions for each flag and attribute, you can create a single *junk dimension* combining them together. This dimension, frequently labeled as a *transaction profile dimension* in a schema, does not need to be the Cartesian product of all the attributes' possible values, but should only contain the combination of values that actually occur in the source data.

Remove unnessary details like payment details form fact table and create a differnet table that is called junk dimension

Historical data wil be maintained SCD i.e SCD1, SCD2,SCD3

Below fact came first ,dimension came late so we will follow 2 steps

1) **Either we will drop FK on fact table**

   Or

   We will update key val in dimension table with other values as null later we will update remaining columns

Types of Facts Tables:

Fully additive measure/ fact:

The numeric value in a fact table that is more flexible is an additive measure. For each dimension you can even sum up. If you want to know the total sales of your company you can easily sum up all the sales.

In below image sales amount we can calculate with all dimensions i.e location wise sales amount, customer wise sales amount, date wise sales amount and product wise sales

Semi-additive

Semi-additive measures can be summed across some dimensions, but not all; checking account or savings account balance amounts are common semi-additive facts.

balance amounts are common semi-additive facts because they are additive across all dimensions except time

ex: my bank balance on 01- aug s 1000

my balance on 01- sep 500

to find total balance we can't sum it bcz total balance available in my account is 500 only not 1500

Semi-Additive:
Semi-additive Facts are Facts that can be summed up for some of the dimensions in the Fact table, but not the others. One of the usual examples for this is are the current account or savings account balance amounts are common semi-additive facts. We can get/generate a balance amount from the overall transactions , but it doesn't make any sense to add (group) the balance amounts from different dates or months or across the time dimension.

U can aggregate semi-additive facts for some, but not all, dimensions.

You can aggregate department headcounts to give an organization total, but you cannot aggregate them over time, so the Sales department headcount for March 31 may be 20 employees, and for April 30 the headcount may be 23, but that does not mean that the total headcount at the end of April is 43.

Non additive measures

Non-Additive:

Certain measures/numbers are completely non-additive, such as ratios discounts passmarks. Non-additive Facts are Facts that cannot be summed up for any of the dimensions present in the Fact table.

Eg: Facts which have percentages, Ratios calculated.

Ex: one employee has sold an item with a 55% profit margin and the other has sold an item with a 45% profit margin, the profit margin for the department is not 100%.

In this example passmarks is non additive we can't perform any operations on that.

Derived measures/facts :

In below image profit is not part of db we will calculate on the fly.

Factless Fcat Table

A factless fact table is a fact table that does not have any measures.OR facts it contains only key attributes i.e FK

In above image Fact aatendace table is factless fact table. Form this table we can anlyze data like in total strength of given course , how many people are obsent in given course and given year, which student has max attendace

Textual Measures:

In below example bill no is textual measure

Surrogate key:

https://dwgeek.com/data-warehouse-surrogate-key-design-advantages-disadvantages.html/#:~:text=Data%20warehouse%20surrogate%20keys%20are,join%20dimension%20and%20fact%20tables.&text=It%20is%20just%20sequentially%20generated,better%20lookup%20and%20faster%20joins.

What are surrogate keys in Data warehouse?

If you are a data warehouse developer, that you might be thinking what is surrogate key? How and where it is being used? You will get answers to all your questions here.

Data warehouse surrogate keys are sequentially generated meaningless numbers associated with each and every record in the data warehouse. These surrogate keys are used to join dimension and fact tables.

- Usually, database sequences are used to generate surrogate key so it is always **unique number**
- Surrogate keys **cannot be NULLs**. Surrogate key are never populated with NULL values.
- It does not hold any meaning in data warehouse, often called meaningless numbers. It is just **sequentially generated INTEGER** number for better lookup and faster joins.

Why surrogate keys are used in Data warehouse?

Basically, surrogate key is an artificial key that is used as a substitute for natural key (NK) defined in data warehouse tables. We can use natural key or business keys as a primary key for tables. However, it is not recommended because of following reasons:

- **Natural keys (NK)** or **Business keys** are generally alphanumeric values that is not suitable for index as traversing become slower. For example, prod123, prod231 etc
- Business keys are often reused after sometime. It will cause the problem as in data warehouse we maintain historic data as well as current data.

For example, product codes can be revised and reused after few years. It will become difficult to differentiate current products and historic products. To avoid such a situation, surrogate keys are used.

Data Warehouse Surrogate Key examples

Surrogate Keys are integers that are assigned sequentially in the dimension table which can be used as primary key. The surrogate key column could be identity column or database sequences are used.

Below is the sample example of surrogate key:

PATIENT_SK	PATIENT_ID	PATIENT_NAME	PATIENT_AGE
1	P001	ABC	20
2	P002	BCD	25
3	P003	CDE	19
4	P004	DEF	45

Advantages of Surrogate Key

Below are some of advantages of using surrogate keys in data warehouse:

- With help of surrogate keys, you can integrate heterogeneous data sources to data warehouse if they don't have natural or business keys.

- Joining tables (fact and dimensions) using surrogate key is faster hence better performance
- Surrogate keys are very helpful for ETL transformations.
- Data warehouse Surrogate keys are usually small integer numbers that makes smaller index and better performance
- Surrogate keys are required if you are implementing slowly changing dimension (SCD)

Disadvantages of Surrogate Key

Below are some of disadvantages of using surrogate keys in data warehouse:

- Surrogate key generation and assignment takes unnecessary burden on ETL framework
- You should not over use the surrogate keys as they don't have any meaning in data warehouse tables.
- Data migration becomes difficult if you have database sequence associated with surrogate key columns. You should carefully take care of number surrogate key generation in new database otherwise you may end up with duplicate surrogate keys.

Grain:

When you identify the grain, you specify exactly what a fact table record contains. The grain conveys the level of detail that is associated with the fact table measurements. When you identify the grain, you also decide on the level of detail you want to make available in the dimensional model. If more detail is included, the level of granularity is lower. If less detail is included, the level of granularity is higher.

Identifying the level of detail

The level of detail that is available in a star schema is known as the *grain*. Each fact and dimension table has its own grain or granularity. Each table (either fact or dimension) contains some level of detail that is associated with it. The grain of the dimensional model is the finest level of detail that is implied when the fact and dimension tables are joined. For example, the granularity of a dimensional model that consists of the dimensions Date, Store, and Product is *product sold in store by day*.

Data flow

. if we write query directly from DWH the qury performace is slow. cube will perform all permutations and combinations of data aggregations so we will get o/p very fast

olap

OLAP works as a fundamental stone to many types of business apps pertaining to

- Business Performance Management
- Planning, Forecasting, Budgeting
- Financial Reporting
- Simulation Models
- Knowledge Discovery
- Data Warehouse Reporting

Features

- Cube Design
- MOLAP, ROLAP (Multidimensional and Relational OLAP)
- MDX Queries
- Slice and Dice
- Drill down
- Ad-hoc Analysis

Major Advantages

- In-depth analysis because of multiple dimensions

- Empowers complicated analysis of bulky data volumes as against intricate business-queries

Cubes

Cubes are data processing units composed of fact tables and dimensions from the data warehouse. They provide multidimensional views of data, querying and analytical capabilities to clients. A cube can be stored on a single analysis server and then defined as a linked cube on other Analysis servers. End users connected to any of these analysis servers can then access the cube. This arrangement avoids the more costly alternative of storing and maintaining copies of a cube on multiple analysis servers. linked cubes can be connected using TCP/IP or HTTP. To end users a linked cube looks like a regular cube.Linked cube are cubes in which a sub-set of the data can be analysed into great detail. The linking ensures that the data in the cubes remain consistent.

https://intellipaat.com/blog/tutorial/data-warehouse-tutorial/what-is-olap-and-multidimensional-model/

Peeling back 3 layers of BI and Analytics software

by Ken Gnazdowsky, B.Sc. | Posted on October 17, 2018

Three inter-related layers or processes in a robust BI analytics solution can include:

1. a Symantic (meta-data) layer;
2. OLAP cubes;
3. Extract, Transform, Load (ETL) jobs.

Let's take a closer look at each of these and how small changes within any one of them can affect your critical business reports at the top of your BI stack.

(1) Semantic layer – an abstracted (meta-data) layer that contains simplified business views mapped to underlying relational database models and data warehouses. This layer is a pre-requisite for some WSYWIG, drag and drop design reporting tools.

A **semantic layer** is a business representation of corporate data that helps non-technical end users access data using common business terms. The semantic layer is configured by a person who has knowledge of both the data store and the reporting needs of the business. The person creating the semantic layer chooses to expose appropriate fields in the data store as "business fields," and to hide any fields that are not relevant. Each business field is given a friendly, meaningful name, and the business fields are organized in a way that will make sense to business users.

By using common business terms, rather than data language, to access, manipulate, and organize information, a semantic layer simplifies the complexity of business data. This is claimed to be core business intelligence (BI) technology that frees users from IT while ensuring

correct results when creating and analysing their own reports & dashboards. In reality, however, *IT staff are typically needed to create and maintain the* ***semantic layer*** *(business view definitions) which maps tables to classes and columns to objects.*

(2) OLAP cubes – the arrangement of data into Cubes overcomes a limitation of relational databases, which are not well suited for near instantaneous analysis and display of large amounts of data. OLAP data is typically stored in a star schema or snowflake schema within a relational data warehouse or in a special-purpose data management system.

An **OLAP cube** is a term that typically refers to multi-dimensional array of data. *OLAP* is an acronym for online analytical processing, which is a computer-based technique of analyzing data to look for insights. A cube can be considered a multi-dimensional generalization of a two- or three-dimensional spreadsheet. Each cell of the cube holds a number that represents some *measure* of the business, such as sales, profits, expenses, budget and forecast. A Slice represents two-dimensional view of an **OLAP Cube** that arranges data in a grid, similar to a spreadsheet; a Slice functions much like a report or a query in an **RDBMS** in that it returns data based on a request for what to see

Although many report-writing tools exist for relational databases, these are slow when the whole database must be summarized, and present great difficulties when users wish to re-orient reports or analyses according to different, multidimensional perspectives, aka, **Slices**. The use of Cubes facilitate this kind of fast end-user interaction with data. Some reporting tools like Crystal Reports include the ability to define simple two-dimentional OLAP cubes for generating graphs for example.

(3) ETL Software – import clean data from virtually any source into permanent, structured relational database tables.

ALSO CALLED: Data Extraction Software, Database Extraction Software

DEFINITION: In managing databases, ETL refers to three separate functions combined into a single programming tool: Extract, Transform, Load

1. First, the **extract** function reads data from a specified source database and extracts a desired subset of data.

2. Next, the **transform** function works with the acquired data – using rules or lookup tables, or creating combinations with other data – to convert it to the desired state.

3. Finally, the **load** function is used to write the resulting data (either all of the subset or just the changes) to a target database, which may or may not previously exist.

ETL *can be used to acquire a temporary subset of data from virtually any file for reports* or other purposes, or a more permanent data set may be acquired for other purposes such as: the population of a data mart or data warehouse; conversion from one database type to another; and the migration of data from one database or platform to another.

A change to the underlying source database, or at any one of these BI layers can affect the hundreds or in some organizations, thousands of reports that are dependent on the underlying data model. *Something as simple as renaming a field or variable name can essentially break the chain, and stop reports from working or producing accurate results the business depends on.*

Now imagine a change impact analysis tool that can search your entire BI process flow to show you all affected references throughout your Business Intelligence stack.

Data Lake architecture

Because data that goes into data warehouses needs to go through a strict governance process before it gets stored, adding new data elements to a data warehouse means changing the design, implementing or refactoring structured storage for the data and the corresponding ETL to load the data. With a massive amount of data, this process could require significant time and resources. This is where a data lake concept comes into the picture and becomes a game-changer in big data management.

The concept of data lake emerges in the 2010s, which, in a simple language, is the idea that all enterprise's structured, unstructured and semi-structured data can and should be stored in the same place. Apache Hadoop is an example of data infrastructure that allows to store and process massive amounts of data, both structured and unstructured; which enables the Data Lake architecture.

Example of DWH and Data Lake architecture. Illustration by author based on MS Azure document and Daniel Linstedt 's book.

Data lake has schema on read approach. It stores raw data and is set up in a way that does not require defining the data structure and schema in the first place. Put differently, when we move data to data lake, we just bring it in without any gate keeping rules and when we need to read the data, we apply the rule to the code that reads the data rather than configuring the structure of data ahead of time. Instead of the typical Extract, Transform and Load in data warehousing, in the world of data lake, the process is Extract, Load and Transform. Data Lake is utilized for cost efficiency and exploration purpose. As such, a Data Lake architecture enables business to gain insights not only from the processed and governed data but also from raw data that was not available for analysis before. From that, raw data exploration can potentially trigger business questions. However, the biggest concern with data lake is that, without appropriate governance, data lakes can quickly turn into unmanageable data swamps. **Put it differently, without knowing how the water is in a lake, who would want to go swim in it? Business users can't not utilize the data lake if they don't trust the data quality of that lake .**

Source; Datavercity through slideshare

Recently the trend of companies that want to benefit from a data lake architecture in a more conservative way has emerged. These companies are stepping away from the ungoverned “free-entry” approach and instead developing a more governed data lake.

The data lake can contain two environments: an exploration/development and production environment. Data will be explored, cleansed, transformed in order to build machine learning model, build functions and other analytics purposes. Data such as metrics, functions that have been generated by the transformation process will be stored in the production part of data lake.

Another trend is, rather than pouring all raw data into the lake, the governed data lake only allows ‘verified’ data to get into it. Essentially, a governed data lake architecture does not restrict the types of data that are stored in it, meaning that governed data lakes still comprise multiple data types including unstructured and semi-structured data like XML, JSON, CSV. However, the key is to make sure that no data is stored in the lake without being described and documented in business glossary, which will give some confidence to the users about the quality and meaning of data.

To provide this layer of governance, a business glossary tool has to be in place to document the meaning of the data. More importantly, there needs to be a governance process around this — which is all about roles and responsibilities, for instance, who owns the data, who defines it, who will be responsible for any data quality issues. Going for this approach will be time-consuming because defining data itself can be a long process since it involves people from different disciplines across an enterprise.

Data Lake and Data Warehouse comparison. Illustration by author.

Defining Identifying and Non-Identifying Relationships

In database terms, relationships between two entities may be classified as being either identifying or non-identifying. Identifying relationships exist when the primary key of the parent entity is included in the primary key of the child entity. On the other hand, a non-identifying relationship exists when the primary key of the parent entity is included in the child entity but not as part of the child entity’s primary key. In addition, non-identifying relationships may be further classified as being either mandatory or optional. A “**mandatory**” non-identifying relationship exists when the value in **the child table cannot be null**. On the other hand, an “**optional**” non-identifying relationship exists when the value in the **child table can be null.**

Appendix D — Snowflake Interview Questions

Source: interview.md converted from the corresponding uploaded Word document.

Athena transform function wont work in snowflake

ATHENA VS SNOWFLAKE

HOW TO FIND TABLE size

1) Tasks
2) Streams cdc changes
3) Oracle analytical function
4) Joins
5) LEAD, LAG analytical function
6) Scheduling tool used to schedule snowflake scripts as part of batch load
7) Time taken for migration – 4 to 5 months
8) Which tool is used for ajile – https://jiraagile.emirates.com/
9) Table design best practices (refer below links or attached NTT document)
10) **What is CTE (Common Table Expressions) or** or subquery factoring clause, or with clause
11) →Diff B/w oracle and snowflake data base objects
12) **how to store filename on each record when copying files' data into snowflake?**
13) **Ans: user these metadata columns** metadata$filename, metadata$file_row_number
14) **IF WE CHNAGE THE upper to lower in query whther it will use result cache ?**

Ans: No. Because snowflake will check the executed queries in cloud service layer using hash. If you change the case the hash of query will change. Hence snowflake will think this as new query.

15) **IF WE ADD SPACE IN THE QUERY then it will use result cache?**

Ans : NO

17) **Bcz result cache will be used when** The new query matches the previously-executed query (with an exception for spaces).
18) **TYPES OF TABLES → DIFF B/W TEMPORARY,TRANSIENT, PERMANNET TABLES**
19) **if we use where cluase extra whether it will use same result cache**
20) **max no clusters in vartual warehouse— more than 10 possible? Max is 10**
21) **→when virtual warehouse is spinup to multiple virtual then is RAM also added for these warehouses?**

Ans: in snowflake **virtual warehouse uses shared nothing architecture. So it will add RAM also**

when virtual warehouse is spinup to multiple virtual then is storage also added for these warehouses ? ans :it wont add storage bcz in snowflake warehouse is for compute purpouse and it is de coupled with storage

Scaling policy: Standard /Economy

→If source has multiple commas in csv then how to handle that

Ans: we have to create file format with option as Filed optionally enclosed by

19) is it possible to create multiple not null constraints on single table

A Snowflake table can have multiple NOT NULL columns.

Snowflake supports defining and maintaining constraints, but does not enforce them, except for NOT NULL constraints, which are always enforced.

20) handling json data with STRIP_OUTER_ARRAY

The VARIANT data type has a 16 MB (compressed) size limit on the individual rows

What is column security or dynamic data masking

- **Retention period questions →CTS**

Note: retention period we can set at database level, schema level, table lavel.

Means in SAME database and same schema different table may have different retention periods

Scale up vs scale out → CTS

Scaling up is all about increasing the compute power of the existing warehouse node. This should assist long-running queries, queries that require a lot of bytes scanned and queries with storage spillage. This would be done by increasing the size property of the warehouse.

Scaling Out is the process of adding more clusters to an existing warehouse

Scaling up →ware house size increasing like from Large to extra large. scaling up is used when we are running complex queries

Scaling out→ adding more clusters i.e multi cluster . multi clustering is use full when queries large no of queries are queued in same warehouse

→what is file size suggested for Bulk load

→what is file suggested for snow pipe

→Is it possible to use function in copy command like concat →yes→what is metadata in snowflake

Imp points : snowflake uses Foundationdb for metadata

Copy command

Options force=true

Purge=true

Error handling using try cast

what is use of information schema in snowflake

3) while loading data error occurred few records are error out. Then how to load error records

Ans:

Fix Errors and Load Again

In regular use, you would fix the problematic records manually and write them to a new data file. Alternatively, you could regenerate a new data file from the data source containing only the records that did not load.

You would then stage the fixed data files to the S3 bucket and attempt to reload the data from the files.

-→ loading an dunloading data from snowflake from local system

- Smaller Virtual Warehouse = fewer files with a bigger file size
- Bigger Virtual Warehouse = more files with a smaller file size

Example:

Small WH = 2 nodes * 8 servers = 16 servers which will unload 16 files in parallel at any time
Large WH = 8 nodes * 8 servers = 64 servers which will unload 64 files in parallel at any time

Imp questions on cache:

(1) Which of these are snowflake cache layers? → result cache,local disk cache, remote disk

(2) **Which cache layer holds query result for 24 hours? →result cache**

(3) Result cache belongs to ? → service layer

(4) Local disk cache belongs to-→ Compute layer

(5) Remote disk is nothing but, → blob storage area like aws s3

(6) Cold , Hot , Warm ` match these cache layers with below sequence → remote disk,result cache,local disk cache

Cold→ remote disk cache

Warm→Local disk cache

Hot→ result cache

(7) alter session set use_cached_result=flase — this command will disable Result Cache

(8) You can not disable remote disk cache, Local disk cache

(9) When underlying data for table changes, we use → remote disk cache

(10) I can use Result cache even if i suspend my warehouse → True

Cluster key creation syntax:

Cluster key CTS interview question

- For one million unique records data if we add cluster key is performance will be improved. ANS: NO
- If above table is joined with any other table on clustering key then it will improve performace? Yes
- If all records are unique you should not be adding any clustering key. As it will not add any benefit.
- If you cluster the table based on the joining key(assuming it's having less cardinality) then you can see performance improvement.
- **1)When user submits query, snowflake submits it to→ Cloud service layer**
- **(2)Which of these are part of query life cycle in cloud service layer?–> parsing,object resolution,access control and plan optimization**
- **(3) Query execution plan will be submitted to ? → worker nodes in virtual warehouse**
- **(4) All query information and statistics are stored for audits and performance analysis. → in cloud service layer**
- **(5) Once execution plan is submitted to virtual warehouse layer, what will happen?–> Nodes will download HEADER FILE remote disk and based on it,scans meta data**
- **(6) select name , department from employee. This query will download ?→ only name and department columns from remote disk based on HEADER FILE information**
- **(7) Micro-partitioning is performed automatically on all snowflake tables**
- **(8) Tables are partitioned based on the ordering of data as it is inserted or loaded → TRUE**
- **(9) Micro partitions size when uncompressed → 50-500 MB**
- **(10) In micro-partition each column is stored independently → TRUE**

- **(11) Degree of overlap of micro-partitions in snowflake is called → Micro partition depth**
- **(12) Constant micro-partition will have a depth of → 1**
- **Important point for clustering (Pradeep ch udemy topic 27 how clustering works)**
- When we are loading data into a table, snowflake will not organize micro partitions while loading data. After data load completes Snowflake will reorganize the partitions (it will be done in back ground).
- Please remember that snowflake will charge for that reorganizing partitions

Topic 28. Improve performance without applying clustering

As Snowflake will charge to arrange partitions to avoid this billing , if we are sure that on which key clustering is required then while loading data from source table itself we can use **order by that column** so that partitions will be organized without creating cluster key on that column which will save cost

Course Curriculum

Snowflake Overview and Architecture

Explain about Snowflake architecture?

What is the Snowflake Data Warehouse?

How does Snowflake Work?

What are the three layers of Snowflake architecture?

Is Snowflake an MPP database?

Explain about the different table Types available in Snowflake?

Which Snowflake edition should you use if you want to enable time travel for up to 90 days?

What are Micro-partitions?

Can you create Transient Views in Snowflake?

Explain about the differences and similarities between Transient and Temporary tables?

By default, clustering keys are created for every table, how can you disable this option?

What is the default type of table created in Snowflake?

How many servers are present in X-Large Warehouse

As Snowflake should use one of the cloud provider (like AWS or Azure) as part of its architecture, why can't the AWS database Amazon Redshift can be used instead of the Snowflake warehouse?

What view types can be created in Snowflake but not in traditional databases?

Is Snowflake a Data Lake?

What are the key benefits you have noticed after migrating to Snowflake from a traditional on-premise database?

When you execute a query, how does Snowflake retrieves the data as compared to the traditional databases?

Explain the difference between External Stages and Internal Name Stages?

Explain the difference between User and Table Stages?

You are working in a Investment Bank and you want to explore on Snowflake and decided to create a free Snowflake account. Your Manager has Instructed you to use Virtual Private Snowflake Edition for the free trail as this edition provides dedicated servers for your company? What do you suggest?

What are the constraints which are enforced in Snowflake?

Real-Time Scenarios

You have created a Virtual Warehouse with a warehouse size of 2X-Large, and extracted data from different data sources applied Address Validation standardization using third-party tools before loading the data into the warehouse. As part of the compliance requirement, you need to make sure to compare the source data address with the address loaded in the target after applying the address standardization and load the records which don't match into a temporary table. The temporary table which is created in the Prod environment has to be cloned to the test environment for the development team to review the data and then provide the results to the Business Team? What is your recommendation?

You have recently created your Snowflake account, and created few jobs which extract data from an SAP HANA, and during one of the Product Release testing, there are some failures due to which some of the virtual warehouses (2X-Large) are not available? What do you recommend?

You have observed that a store procedure which is getting executed daily at 7 AM as part of your batch process is consuming resources and the CPU I/O is showing as 90%, and the other jobs which are getting executed are impacted due to the store procedure. How can you quickly resolve the issue with the store procedure?

You have migrated from Teradata to Snowflake, one of the main issues which you faced in the old system is with the MPI system data. The MPI data is sent by the source system daily at 01:00 AM and the ETL Process will take around 5-6 hours and loads the data into the target tables between 06:00 AM to 07:00 AM. After the ETL process is complete no other process will modify the data in these tables until the business users check and confirm the Ledger Transactions. After the users confirm, the Indicators in the target table are updated and data will be loaded to the downstream tables. The Issue which is faced by the business users while accessing the data in Teradata is, each user has to wait for 01-02 hours to get the required General Ledger Stats, and sometimes when multiple queries are executed the CPU I/O usage was high. Business users want to get the query results immediately as these are

static SQL's which they use on daily basis. How these issues can be fixed in the newly migrated Snowflake database?

You have migrated from Teradata to Snowflake, in the old system (Teradata) few ETL batch loads are scheduled to execute using Teradata Tpump load utility, TPump uses row hash locks, meaning users can run queries while it's updating the Teradata Warehouse. In the new system(Snowflake), you should also use above approach. i.e. Users should be able to access the data, while the loads are being executed on the Snowflake Warehouses without any issues. What is your recommendation?

You are using the Snowflake connector in Informatica Cloud (Data Integration tool) to process some data as per your batch requirements, you have extracted data from different data sets and loaded the data into the stage tables, from stage tables the data will be loaded to your warehouse. The data in the stage tables are always truncated and reloaded for every load. In Snowflake, you can define the stage tables type as

Some queries are getting executed on a warehouse and you have executed Alter Warehouse statement to resize the warehouse, how this will effect the queries which are already in execution state?

Its a best practice to disable the fail-safe for temporary tables, these tables exist only for the duration of a session and are not queriable by any other user, disabling fail-safe will help in reducing failsafe storage for temporary tables? What do you recommend?

Your company has recently procured Snowflake Standard edition, as per the Initial plan you have planned to migrate applications one by one and then upgrade the Snowflake to Enterprise Edition, but all the applications are dependent on each other, so have migrated all the applications at the same time to Snowflake Standard Edition. As queries are submitted to the warehouse, Snowflake has queued most of the queries due to Insufficient resources which are causing issues in users accessing different applications. How can you resolve the above issue at the earliest?

A new business analyst has joined your project, as part of the on-boarding process you have sent him some queries to generate some reports, the query took around 5 minutes to get executed, the same query, when executed by other business analyst's, has returned the results immediately? What could be the Issue?

You are working in an Insurance company, and you have planned a major deployment on the weekend which includes extracting historical data from PowerExchange and load it into one of the Snowflake database tables, the load took around 20 hours to complete and the data is validated to be released to the users on Monday morning so that the users can complete the review as part of the compliance process for the newly launched MedSupp Policies. One of the Incremental ETL Jobs which you have deployed as part of this major deployment has the Truncate Target Table option enabled and the data which is loaded PowerExchange is deleted when the job executed on Monday. What is the best approach to recover the historical data at the earliest which was accidentally deleted?

You have created a warehouse using the command create or replace warehouse OriginalWH initially_suspended=true; What will be the size of the warehouse?

You are changing the scaling policy for a warehouse from Standard to Economy. You want to make sure the SQL statements from the application can be queued for only 180 seconds and if there are any queries which run for more than 360 seconds should be canceled by the system. Which parameters should you configure for this requirement?

You are executing some queries on Medium size Warehouse, the queries are getting executed for a longer period of time than expected. You are planning to re-size the warehouse to X-Large size? can you resize the warehouse when the queries are still executing?

You have created an External Table(E_Prev_MPI) in which you have loaded all the Historical data from an MPI source system, you need to join the E_Prev_MPI table with one of the tables in Warehouse (W_Curr_MPI) which has the current snapshot of data, and if there are any matching records, you need to update the E_Prev_MPI.Matched column to 'Yes' There are a lot of performance issues while performing the update so you have created Partition on the E_Prev_MPI table. Is this the best approach?

You are working in a medical services company, as per the guidelines of the legal team, any objects containing PII data must not be visible to those who do not require access to the PII data. What is the best approach for the above requirement?

You are working in a major telecom company, you collect transactional data from different switches which generate huge(1 TB) CDR (call detail records) volume every day. All the CDR records are loaded to a summary table (which is present in the Snowflake warehouse of size 4X_Large) and different reports are generated based on the daily revenue generated on the calls for each region. The queries are getting executed for a long time to generate the daily reports which are based on the transaction date and region, what is the best approach to optimize the queries to generate the reports faster

Your organization has planned to procure Snowflake and has decided to migrate the application in a phase by phase manner. In the first phase, you have planned to Include some non-critical applications and the requests from these applications can be queued up to 24 hours till they are processed. You want to keep the costs low in the first phase which can be increased going forward. Select the best Warehouse for the above requirement.

Tableau reports are configured to query the data based on the Joins from multiple Snowflake tables which have a size of 120 GB each, the reports will be accessed by the senior management to make critical decisions. Some of the users who are accessing the reports, have faced severe performance issue to load the Dashboard and access the reports. What can be done so that the users can get fast response time for accessing the dashboards and reports?

You are trying to debug a production issue due to which some of the reports are showing incorrect numbers. The issue is with data loaded in a summary table for a particular policy(AMT00877TR5) is showing Incorrectly, the table from which the data is fetched is of SCD-Type 1, by the time you checked the data, the tables were

already updated with the latest data. You are not sure about the root cause of the issue. How can you check the data before it was updated?

You have created some reports which access huge volume of data, the reports are configured to perform range and equality searches and business users generate the report on every first business day of the month. The reports are generated based on the data available in the Snowflake tables, and the report generation process is very slow. You have been Instructed to not add any more storage costs due to some project constraints. What can be done to improve the performance of the reports?

A query is executed from the client and the query result exists in the result cache and the underlying data is not changed. The query results are returned from

A manufacturing company decided to implement data sharing and share data about the progress of orders directly to the consumers of their products. However, a customer must only be able to see the orders they have placed.

You have scheduled a job to re-cluster a table on the weekend, but the DBA Team will suspended all the virtual warehouse's on the weekend. What error will you get when your job is triggered?

You have a ETL process, which is currently getting executed on Oracle which is installed on a single cluster, the process s at 7 PM and it will take around 10 hours to complete. You are now migrating to Snowflake, you should provide clear estimate on how much time the job will take to complete in the new Snowflake Prod environment, your manager is concerned on the credits ($$) charged that will be charged by Snowflake. How will you provide the stats for this?

You have created a network policy but the policy is not enforced in Snowflake, what could be the issue?

You have recently joined a company which is using Teradata database, some of the Architects proposed that they should migrate their database from Teradata to Snowflake but the customer is not clear on the benefits they get by moving to Snowflake as both Teradata and Snowflake are Massive Parallel processing (MPP) systems. What is your recommendation to the customer on using Snowflake compared to the Teradata database?

Why Snowflake has the option to create the Primary and Foreign key constraints when these can not be enforced?

You have created a standard multi-cluster warehouse with Maximum clusters as 10 and Minimum Clusters as 3, lets say the warehouse is using 8 clusters and users have executed several queries which all are cached and users are able to see the cache results faster, now lets say the warehouse has scaled down from using 8 clusters to 4 clusters, will the cache files will be reused when the users execute the same query?

You are trying to perform some data loads using Snowpipe, the load is taking longer than excepted so you stopped the existing load and increased the size of Virtual Warehouse to X-Large, when you re the load does the load resume from the point where it was last stopped?

Billing

How Snowflake charges for Data Storage?

What is the difference between Replication and Cloning?

In Secure Data Sharing, who pays for the compute and storage resources?

You are a Data Provider, and you are sharing your data to non-Snowflake users. How can you control the data usage by the Data Consumers?

You have just procured Snowflake and created 4 warehouses, how much will snowflake charge for this?

You are using Snowflake throughout the year every day for 3 hours, and loading the data into Virtual Warehouse which has a size of X-Small. You are now trying to estimate the yearly credits of the Snowflake usage, your ETL analyst based on the ETL schedule has specified that the snowflake was used for 260 days. How many credits are charged by Snowflake for this usage?

Does Snowflake charge any credits when you perform re-clustering on a table?

You have used Snowflake trail account, and decided to proceed with migrating your data to Snowflake Enterprise Edition. You have clear stats on the required storage. Which purchase plan do you recommend?

How did you optimize the incurred costs in Snowflake?

Explain how on-Demand vs Pre-Purchased Capacity will impact your Project Budget?

You need to provide high level estimates on the cost of using Snowflake with different Cloud providers using different regions? Which utility can you use to get these stats?

How choosing the Incorrect storage type will Impact your budget?

You have selected AWS as your cloud provider, where can you check on how much AWS is charging your account for the Snowflake usage?

How can you check the consumed credits in your account?